Low-Level Design

150 Interview Q&A — SDE2 Edition

SDE2 Interview Preparation Series

iPad Edition · Dark Code Blocks · Syntax Highlighting

Low-Level Design (LLD) SDE2 Interview Guide

100 Questions & Answers with Go Code Examples

> **Prepared for:** SDE2 role interviews | **Focus:** Object-Oriented Design, Design Patterns, Go Implementation | **Level:** Mid-level (2–4 years)

Table of Contents

---

1. OOP & SOLID Principles

Q1

What is the Single Responsibility Principle (SRP) and how do you apply it in Go?

Difficulty: Easy

A struct/type should have only one reason to change — it should do one thing. In Go, keep structs focused, split concerns into separate types, and avoid god structs.

```go
// BAD — UserService does too many things
type UserService struct{}
func (u *UserService) GetUser(id int) (*User, error)    { /* DB query */ }
func (u *UserService) SendWelcomeEmail(u *User) error   { /* SMTP */ }
func (u *UserService) GenerateReport(u *User) []byte    { /* PDF */ }
func (u *UserService) LogUserActivity(u *User)          { /* Logging */ }

// GOOD — each type has one responsibility
type UserRepository struct{ db *sql.DB }
func (r *UserRepository) GetUser(id int) (*User, error) { /* DB only */ }

type EmailSender struct{ smtp *SMTPClient }
func (e *EmailSender) SendWelcome(u *User) error { /* email only */ }

type UserReporter struct{ tmpl *template.Template }
func (r *UserReporter) Generate(u *User) []byte { /* report only */ }

// Compose them in service
type UserService struct {
    repo     *UserRepository
    email    *EmailSender
    reporter *UserReporter
}
```

Interview tip: "If you find yourself saying 'and' when describing what a struct does, it probably violates SRP."

Q2

What is the Open/Closed Principle (OCP) and how do you implement it?

Difficulty: Medium

Software entities should be open for extension but closed for modification. In Go, achieve this via interfaces — add new behaviour by implementing an interface, not by modifying existing code.

```go
// Closed for modification — Shape interface never changes
type Shape interface {
    Area() float64
}

// Open for extension — add new shapes without touching existing code
type Circle struct{ Radius float64 }
func (c Circle) Area() float64 { return math.Pi * c.Radius * c.Radius }

type Rectangle struct{ Width, Height float64 }
func (r Rectangle) Area() float64 { return r.Width * r.Height }

type Triangle struct{ Base, Height float64 }
func (t Triangle) Area() float64 { return 0.5 * t.Base * t.Height }

// This function never needs to change when we add new shapes
func TotalArea(shapes []Shape) float64 {
    var total float64
    for _, s := range shapes { total += s.Area() }
    return total
}

// Adding Hexagon doesn't require modifying TotalArea
type Hexagon struct{ Side float64 }
func (h Hexagon) Area() float64 { return 2.598 * h.Side * h.Side }
```

Interview tip: "OCP is why Go interfaces are defined at the point of use, not at the point of definition. The consumer defines what it needs."

Q3
What is the Liskov Substitution Principle (LSP)?

Difficulty: Medium

Objects of a subtype should be substitutable for their supertype without altering program correctness. In Go, this means: any concrete type implementing an interface must honour the interface's contract — not just its method signatures.

```go
type Bird interface {
    Move() string
    Eat() string
}

// VIOLATES LSP — Duck can fly, Penguin cannot
// If code expects Bird.Move() to mean "fly", Penguin breaks it

// BETTER — separate the contracts
type Mover interface{ Move() string }
type Flyer interface{ Fly() string }
type Swimmer interface{ Swim() string }

type Duck struct{}
func (d Duck) Move() string  { return "waddle" }
func (d Duck) Fly() string   { return "flap flap" }
func (d Duck) Swim() string  { return "paddle" }

type Penguin struct{}
func (p Penguin) Move() string  { return "waddle" }
func (p Penguin) Swim() string  { return "fast paddle" }
// Penguin does NOT implement Flyer — correct!

// Functions accept only what they need
func makeItMove(m Mover) string { return m.Move() }
func makeItFly(f Flyer) string  { return f.Fly() }
```

Interview tip: "LSP is violated when implementing an interface forces you to panic or return an error for an unsupported operation. Split the interface instead."

Q4

What is the Interface Segregation Principle (ISP)?

Difficulty: Easy

Clients should not be forced to depend on interfaces they do not use. Prefer many small, focused interfaces over one large, fat interface.

```go
// BAD — fat interface forces all implementors to have all methods
type Worker interface {
    Work()
    Eat()
    Sleep()
    Drive()
    Code()
}

// GOOD — small, focused interfaces
type Coder interface{ Code() }
type Eater interface{ Eat() }
type Driver interface{ Drive() }

type SoftwareDev struct{}
func (s SoftwareDev) Code() { fmt.Println("typing...") }
func (s SoftwareDev) Eat()  { fmt.Println("eating...") }

// SoftwareDev does NOT need to implement Drive()

type DeliveryDriver struct{}
func (d DeliveryDriver) Drive() { fmt.Println("driving...") }
func (d DeliveryDriver) Eat()   { fmt.Println("eating...") }

// Functions accept precisely what they need
func doWork(c Coder)   { c.Code() }
func lunch(e Eater)    { e.Eat() }
func deliver(d Driver) { d.Drive() }
```

Interview tip: "Go interfaces are naturally small. The standard library's io.Reader and io.Writer are the best examples — one method each."

Q5
What is the Dependency Inversion Principle (DIP)?

Difficulty: Medium

High-level modules should not depend on low-level modules — both should depend on abstractions. In Go, high-level code depends on interfaces, not concrete types.

```go
// WRONG — high-level UserService depends on low-level MySQL
type UserService struct {
    db *mysql.DB // concrete dependency — hard to test, swap
}

// CORRECT — both depend on abstraction
type UserStore interface {
    GetUser(ctx context.Context, id int) (*User, error)
    SaveUser(ctx context.Context, u *User) error
}

// High-level module depends on interface
type UserService struct {
    store UserStore // abstract dependency
}

func NewUserService(store UserStore) *UserService {
    return &UserService{store: store}
}

// Low-level MySQL implementation
type MySQLUserStore struct{ db *sql.DB }
func (m *MySQLUserStore) GetUser(ctx context.Context, id int) (*User, error) { /* ... */ }
func (m *MySQLUserStore) SaveUser(ctx context.Context, u *User) error        { /* ... */ }

// Low-level in-memory (for tests)
type InMemoryUserStore struct{ users map[int]*User }
func (m *InMemoryUserStore) GetUser(ctx context.Context, id int) (*User, error) {
    return m.users[id], nil
}
func (m *InMemoryUserStore) SaveUser(ctx context.Context, u *User) error {
    m.users[u.ID] = u; return nil
}
```

Interview tip: "DIP is why constructor injection (passing interfaces) beats `new(ConcreteType)` inside constructors. It makes code testable and swappable."

Q6
What is composition over inheritance and how does Go enforce it?

Difficulty: Easy

Go has no class inheritance. It uses embedding (composition) to share behaviour. This avoids the fragile base class problem and the diamond inheritance problem.

```go
type Logger struct{ prefix string }
func (l Logger) Log(msg string) { fmt.Printf("[%s] %s\n", l.prefix, msg) }

type Metrics struct{ name string }
func (m Metrics) Record(key string, val float64) {
    fmt.Printf("metric %s.%s = %f\n", m.name, key, val)
}

// Compose via embedding — no inheritance
type UserService struct {
    Logger          // promoted: UserService.Log() works
    Metrics         // promoted: UserService.Record() works
    store UserStore
}

func NewUserService(store UserStore) *UserService {
    return &UserService{
        Logger:  Logger{"UserService"},
        Metrics: Metrics{"user_service"},
        store:   store,
    }
}

svc := NewUserService(store)
svc.Log("user service started")      // from Logger
svc.Record("requests", 1.0)          // from Metrics
```

Interview tip: "Embedding is NOT inheritance — there's no polymorphism. If Dog embeds Animal and both have Speak(), Dog.Speak() calls Dog's, not Animal's through Dog."

Q7

What is the difference between an abstract class and an interface in Go?

Difficulty: Easy

Go has no abstract classes. Interfaces define behaviour (method signatures only). Structs define data. You combine them: embed a struct for shared state/implementation, use an interface for the contract.

```go
// Simulate abstract class with interface + base struct
type Notifier interface {
    Send(to, message string) error
    Channel() string
}

// Shared implementation (like abstract base)
type BaseNotifier struct {
    retries int
    timeout time.Duration
}

func (b *BaseNotifier) withRetry(fn func() error) error {
    for i := 0; i < b.retries; i++ {
        if err := fn(); err == nil { return nil }
        time.Sleep(time.Second)
    }
    return errors.New("max retries exceeded")
}

// Concrete implementations embed base
type EmailNotifier struct {
    BaseNotifier
    smtpHost string
}

func (e *EmailNotifier) Send(to, msg string) error {
    return e.withRetry(func() error {
        return sendSMTP(e.smtpHost, to, msg)
    })
}
func (e *EmailNotifier) Channel() string { return "email" }

type SMSNotifier struct {
    BaseNotifier
    apiKey string
}
func (s *SMSNotifier) Send(to, msg string) error { /* ... */ return nil }
func (s *SMSNotifier) Channel() string { return "sms" }
```

Interview tip: "Go's answer to abstract classes: interface for contract + embedded struct for shared implementation. It's more flexible than single-inheritance."

Q8

How do you design for testability in Go?

Difficulty: Medium

Inject dependencies through interfaces, avoid global state, use constructors, keep functions pure where possible.

```go
// Testable design
type Clock interface{ Now() time.Time }
type RealClock struct{}
func (RealClock) Now() time.Time { return time.Now() }

type MockClock struct{ t time.Time }
func (m MockClock) Now() time.Time { return m.t }

type TokenService struct {
    clock  Clock
    secret []byte
    ttl    time.Duration
}

func NewTokenService(clock Clock, secret []byte, ttl time.Duration) *TokenService {
    return &TokenService{clock: clock, secret: secret, ttl: ttl}
}

func (ts *TokenService) Generate(userID int) string {
    now := ts.clock.Now()
    expiry := now.Add(ts.ttl)
    // create JWT with fixed time — deterministic!
    return createJWT(userID, expiry, ts.secret)
}

// Test with mock clock — fully deterministic
func TestTokenService(t *testing.T) {
    fixedTime := time.Date(2024, 1, 1, 0, 0, 0, 0, time.UTC)
    svc := NewTokenService(MockClock{fixedTime}, []byte("secret"), time.Hour)
    token := svc.Generate(42)
    // token is always the same — no flaky tests
    assert.Equal(t, expectedToken, token)
}
```

Interview tip: "If you can't test it without a real DB, network, or clock — the design needs fixing. Inject all external dependencies."

Q9
What is the difference between coupling and cohesion?

Difficulty: Easy

Cohesion: degree to which elements of a module belong together. Coupling: degree to which modules depend on each other. Goal: high cohesion, low coupling.

```go
// LOW cohesion — Order struct does unrelated things
type Order struct{}
func (o *Order) CalculateTotal() float64 { /* belongs here */ }
func (o *Order) SendConfirmationEmail()   { /* belongs in EmailService */ }
func (o *Order) GenerateInvoicePDF()      { /* belongs in InvoiceService */ }
func (o *Order) UpdateInventory()         { /* belongs in InventoryService */ }

// HIGH cohesion — each type has related responsibilities
type Order struct {
    Items    []OrderItem
    Discount float64
}
func (o *Order) Total() float64       { /* calculation only */ }
func (o *Order) ApplyDiscount(d float64) { /* discount logic only */ }
func (o *Order) IsValid() bool           { /* validation only */ }

// LOW coupling — services communicate through interfaces
type OrderProcessor struct {
    emailer  Emailer      // interface — not concrete EmailService
    invoicer Invoicer     // interface — not concrete InvoiceService
    inventory Inventory   // interface — not concrete InventoryService
}
```

Interview tip: "High cohesion means the struct makes sense as a unit. Low coupling means you can replace one piece without breaking others."

Q10

What is the Law of Demeter (LoD)?

Difficulty: Medium

A unit should only talk to its immediate friends — don't chain method calls through multiple levels of objects. "Don't talk to strangers."

```go
// VIOLATES LoD — knows too much about internal structure
func processOrder(order *Order) {
    city := order.Customer.Address.City.Name // chain of dots = LoD violation
    tax := order.Customer.Account.TaxRate    // knows too much
}

// FOLLOWS LoD — ask the object for what you need
type Order struct {
    customer *Customer
    items    []Item
}

func (o *Order) CustomerCity() string  { return o.customer.City() }
func (o *Order) TaxRate() float64      { return o.customer.TaxRate() }

type Customer struct {
    address *Address
    account *Account
}

func (c *Customer) City() string     { return c.address.City() }
func (c *Customer) TaxRate() float64 { return c.account.TaxRate() }

// Clean — only talks to direct collaborator
func processOrder(order *Order) {
    city := order.CustomerCity()
    tax := order.TaxRate()
}
```

Interview tip: "Each dot is a dependency. `a.b.c.d()` means your code knows about a, b, c, and d's internal structure. One change breaks everything."

Q11

What is DRY (Don't Repeat Yourself) and when should you violate it?

Difficulty: Easy

DRY says every piece of knowledge should have a single, unambiguous representation. But premature abstraction is worse than duplication — wait for the third repetition before abstracting.

```go
// Two similar functions — don't abstract yet (rule of three)
func validateEmail(email string) error {
    if !strings.Contains(email, "@") {
        return errors.New("invalid email")
    }
    return nil
}

func validatePhone(phone string) error {
    if len(phone) < 10 {
        return errors.New("invalid phone")
    }
    return nil
}

// Third similar case — NOW abstract
type Validator func(string) error

func validateField(value string, validators ...Validator) error {
    for _, v := range validators {
        if err := v(value); err != nil { return err }
    }
    return nil
}

emailValidator := func(s string) error {
    if !strings.Contains(s, "@") { return errors.New("invalid email") }
    return nil
}
phoneValidator := func(s string) error {
    if len(s) < 10 { return errors.New("invalid phone") }
    return nil
}
```

Interview tip: "WET code (Write Everything Twice) is sometimes correct. DRY coupling can be worse than duplication. Prefer duplication over the wrong abstraction."

Q12
What is the Tell Don't Ask principle?

Difficulty: Medium

Tell objects what to do, don't ask them for their state and then act on it. Logic belongs with the data.

```go
// ASK style — violates Tell Don't Ask
type Account struct{ balance float64 }
func (a Account) Balance() float64 { return a.balance }

// Caller asks for state and acts on it — business logic leaks out
if account.Balance() >= amount {
    account.balance -= amount
    sendMoney(amount)
}

// TELL style — logic inside the object
type Account struct{ balance float64 }

func (a *Account) Withdraw(amount float64) error {
    if a.balance < amount {
        return errors.New("insufficient funds")
    }
    a.balance -= amount
    return nil
}

func (a *Account) Transfer(amount float64, to *Account) error {
    if err := a.Withdraw(amount); err != nil { return err }
    to.balance += amount
    return nil
}

// Caller tells the object what to do
if err := account.Transfer(amount, recipient); err != nil {
    return fmt.Errorf("transfer failed: %w", err)
}
```

Interview tip: "If you're calling `.GetX()` and then making a decision based on X, move that decision inside the object."

---

2. Creational Patterns

Q13

Implement the Singleton pattern in Go.

Difficulty: Easy

Use `sync.Once` to guarantee one instance regardless of concurrent calls.

```go
type Database struct {
    conn *sql.DB
    dsn  string
}

var (
    dbInstance *Database
    dbOnce     sync.Once
)

func GetDatabase() *Database {
    dbOnce.Do(func() {
        db, err := sql.Open("postgres", os.Getenv("DATABASE_URL"))
        if err != nil { panic(fmt.Sprintf("failed to connect: %v", err)) }
        db.SetMaxOpenConns(25)
        db.SetConnMaxLifetime(5 * time.Minute)
        dbInstance = &Database{conn: db}
    })
    return dbInstance
}

// For testing — allow reset (use a separate Once per test)
func newTestDatabase(dsn string) *Database {
    db, _ := sql.Open("postgres", dsn)
    return &Database{conn: db, dsn: dsn}
}
```

Interview tip: "Never use global var with mutex nil-check (double-checked locking). sync.Once is the idiomatic Go singleton — it's also race-condition-free."

Q14

Implement the Factory Method pattern in Go.

Difficulty: Medium

Define an interface for creating objects. Let implementations decide which concrete type to instantiate.

```go
type Notification interface {
    Send(to, message string) error
    Channel() string
}

// Factory function
type NotificationFactory func(config map[string]string) (Notification, error)

var factories = map[string]NotificationFactory{}

func Register(channel string, factory NotificationFactory) {
    factories[channel] = factory
}

func Create(channel string, config map[string]string) (Notification, error) {
    factory, ok := factories[channel]
    if !ok { return nil, fmt.Errorf("unknown channel: %s", channel) }
    return factory(config)
}

// Email implementation
type EmailNotification struct{ host string }
func (e *EmailNotification) Send(to, msg string) error { /* SMTP */ return nil }
func (e *EmailNotification) Channel() string { return "email" }

// SMS implementation
type SMSNotification struct{ apiKey string }
func (s *SMSNotification) Send(to, msg string) error { /* Twilio */ return nil }
func (s *SMSNotification) Channel() string { return "sms" }

func init() {
    Register("email", func(cfg map[string]string) (Notification, error) {
        return &EmailNotification{host: cfg["host"]}, nil
    })
    Register("sms", func(cfg map[string]string) (Notification, error) {
        return &SMSNotification{apiKey: cfg["api_key"]}, nil
    })
}

// Usage
notif, err := Create("email", map[string]string{"host": "smtp.gmail.com"})
```

Interview tip: "The registry pattern (map of factories) lets you add new types without modifying existing code — OCP in action."

Q15

Implement the Abstract Factory pattern in Go.

Difficulty: Hard

Create families of related objects without specifying their concrete classes.

```go
// Abstract products
type Button interface { Render() string }
type Checkbox interface { Check() string }

// Abstract factory
type UIFactory interface {
    CreateButton() Button
    CreateCheckbox() Checkbox
}

// Windows family
type WindowsButton struct{}
func (w WindowsButton) Render() string { return "<WindowsButton>" }

type WindowsCheckbox struct{}
func (w WindowsCheckbox) Check() string { return "[x] Windows" }

type WindowsFactory struct{}
func (f WindowsFactory) CreateButton() Button     { return WindowsButton{} }
func (f WindowsFactory) CreateCheckbox() Checkbox { return WindowsCheckbox{} }

// Mac family
type MacButton struct{}
func (m MacButton) Render() string { return "<MacButton rounded>" }

type MacCheckbox struct{}
func (m MacCheckbox) Check() string { return "✓ Mac" }

type MacFactory struct{}
func (f MacFactory) CreateButton() Button     { return MacButton{} }
func (f MacFactory) CreateCheckbox() Checkbox { return MacCheckbox{} }

// Application uses factory — doesn't care about concrete types
type Application struct{ factory UIFactory }
func (a *Application) BuildUI() {
    btn := a.factory.CreateButton()
    chk := a.factory.CreateCheckbox()
    fmt.Println(btn.Render(), chk.Check())
}

func getFactory(os string) UIFactory {
    if os == "mac" { return MacFactory{} }
    return WindowsFactory{}
}
```

Interview tip: "Abstract Factory is for when you have families of objects that must be used together. If you just need one type, use Factory Method."

Q16

Implement the Builder pattern in Go.

Difficulty: Medium

Separate construction of a complex object from its representation.

```go
type HTTPRequest struct {
    method  string
    url     string
    headers map[string]string
    body    []byte
    timeout time.Duration
    retries int
}

type HTTPRequestBuilder struct {
    req HTTPRequest
    err error
}

func NewRequest(method, url string) *HTTPRequestBuilder {
    return &HTTPRequestBuilder{
        req: HTTPRequest{
            method:  method,
            url:     url,
            headers: make(map[string]string),
            timeout: 30 * time.Second,
        },
    }
}

func (b *HTTPRequestBuilder) Header(k, v string) *HTTPRequestBuilder {
    b.req.headers[k] = v; return b
}

func (b *HTTPRequestBuilder) Body(data []byte) *HTTPRequestBuilder {
    b.req.body = data; return b
}

func (b *HTTPRequestBuilder) Timeout(d time.Duration) *HTTPRequestBuilder {
    if d <= 0 { b.err = errors.New("timeout must be positive"); return b }
    b.req.timeout = d; return b
}

func (b *HTTPRequestBuilder) Retries(n int) *HTTPRequestBuilder {
    b.req.retries = n; return b
}

func (b *HTTPRequestBuilder) Build() (*HTTPRequest, error) {
    if b.err != nil { return nil, b.err }
    if b.req.url == "" { return nil, errors.New("url required") }
    return &b.req, nil
}

// Usage
req, err := NewRequest("POST", "https://api.example.com/users").
    Header("Content-Type", "application/json").
    Header("Authorization", "Bearer token").
    Body(jsonData).
```

```go
    Timeout(10 * time.Second).
    Retries(3).
    Build()
```

Interview tip: "Return the builder from each method for fluent chaining. Accumulate errors in the builder and return them from Build() — don't panic on invalid state."

Q17

Implement the Prototype pattern in Go.

Difficulty: Medium

Create new objects by copying an existing object (prototype). Useful when object creation is expensive.

```go
type Document struct {
    Title    string
    Content  string
    Metadata map[string]string
    Tags     []string
}

// Cloner interface
type Cloner interface { Clone() Cloner }

func (d *Document) Clone() Cloner {
    // Deep copy
    metaCopy := make(map[string]string, len(d.Metadata))
    for k, v := range d.Metadata { metaCopy[k] = v }

    tagsCopy := make([]string, len(d.Tags))
    copy(tagsCopy, d.Tags)

    return &Document{
        Title:    d.Title,
        Content:  d.Content,
        Metadata: metaCopy,
        Tags:     tagsCopy,
    }
}

// Registry of prototypes
type DocumentRegistry struct {
    prototypes map[string]*Document
}

func (r *DocumentRegistry) Register(name string, doc *Document) {
    r.prototypes[name] = doc
}

func (r *DocumentRegistry) Create(name string) (*Document, error) {
    proto, ok := r.prototypes[name]
    if !ok { return nil, fmt.Errorf("prototype %s not found", name) }
    return proto.Clone().(*Document), nil
}

// Usage
registry := &DocumentRegistry{prototypes: make(map[string]*Document)}
registry.Register("report", &Document{
    Title:    "Monthly Report",
    Metadata: map[string]string{"author": "system"},
    Tags:     []string{"report", "monthly"},
})

// Create report from prototype
report, _ := registry.Create("report")
report.Title = "January Report" // customise clone
```

Interview tip: "Deep copy vs shallow copy matters here. If the prototype has maps or slices, shallow copy leads to shared state bugs. Always deep copy."

Q18

What is the Object Pool pattern and when do you use it?

Difficulty: Medium

Reuse expensive objects (DB connections, goroutines, buffers) instead of creating/destroying them repeatedly.

```go
type ConnPool struct {
    pool    chan *sql.Conn
    db      *sql.DB
    maxSize int
}

func NewConnPool(db *sql.DB, size int) *ConnPool {
    p := &ConnPool{pool: make(chan *sql.Conn, size), db: db, maxSize: size}
    for i := 0; i < size; i++ {
        conn, _ := db.Conn(context.Background())
        p.pool <- conn
    }
    return p
}

func (p *ConnPool) Acquire(ctx context.Context) (*sql.Conn, error) {
    select {
    case conn := <-p.pool:
        return conn, nil
    case <-ctx.Done():
        return nil, ctx.Err()
    }
}

func (p *ConnPool) Release(conn *sql.Conn) {
    select {
    case p.pool <- conn:
    default:
        conn.Close() // pool full — discard
    }
}

// Usage with defer
func queryUser(pool *ConnPool, id int) (*User, error) {
    conn, err := pool.Acquire(context.Background())
    if err != nil { return nil, err }
    defer pool.Release(conn)
    // use conn...
    return &User{}, nil
}
```

Interview tip: "sync.Pool is built-in but has no size limit and objects can be GC'd. Use channel-based pools for DB connections where you need strict size limits."

Q19

Implement the functional options pattern.

Difficulty: Medium

Pass optional configuration without changing function signatures. Backward-compatible when adding new options.

```go
type Server struct {
    host         string
    port         int
    timeout      time.Duration
    maxConns     int
    tlsEnabled   bool
    certFile     string
    logger       *log.Logger
}

type ServerOption func(*Server)

func WithPort(p int) ServerOption {
    return func(s *Server) { s.port = p }
}

func WithTimeout(d time.Duration) ServerOption {
    return func(s *Server) { s.timeout = d }
}

func WithMaxConns(n int) ServerOption {
    return func(s *Server) { s.maxConns = n }
}

func WithTLS(cert, key string) ServerOption {
    return func(s *Server) {
        s.tlsEnabled = true
        s.certFile = cert
    }
}

func WithLogger(l *log.Logger) ServerOption {
    return func(s *Server) { s.logger = l }
}

func NewServer(host string, opts ...ServerOption) *Server {
    s := &Server{
        host:     host,
        port:     8080,
        timeout:  30 * time.Second,
        maxConns: 100,
        logger:   log.Default(),
    }
    for _, opt := range opts { opt(s) }
    return s
}

// Usage
srv := NewServer("0.0.0.0",
    WithPort(9090),
    WithTimeout(60*time.Second),
    WithTLS("cert.pem", "key.pem"),
    WithMaxConns(500),
```

```go
)
```

Interview tip: "This is the most important Go-specific pattern. Found in gRPC-Go, uber-zap, go-redis. Know it inside-out."

Q20
What is a Multiton pattern?

Difficulty: Medium

Like Singleton, but manages a fixed set of named instances. Useful for database connection pools per tenant, config per environment.

```go
type DBPool struct {
    db *sql.DB
}

var (
    pools   = map[string]*DBPool{}
    poolsMu sync.RWMutex
    poolsOnce = map[string]*sync.Once{}
)

func GetPool(name string) *DBPool {
    poolsMu.RLock()
    if p, ok := pools[name]; ok {
        poolsMu.RUnlock()
        return p
    }
    poolsMu.RUnlock()

    poolsMu.Lock()
    defer poolsMu.Unlock()

    if _, ok := poolsOnce[name]; !ok {
        poolsOnce[name] = &sync.Once{}
    }

    poolsOnce[name].Do(func() {
        dsn := os.Getenv("DB_" + strings.ToUpper(name))
        db, err := sql.Open("postgres", dsn)
        if err != nil { panic(err) }
        pools[name] = &DBPool{db: db}
    })

    return pools[name]
}

// Usage
primaryPool := GetPool("primary")
replicaPool := GetPool("replica")
```

Interview tip: "Multiton is common in multi-tenant SaaS — one DB pool per tenant. The key is ensuring thread-safe lazy initialisation per key."

Q21

Implement a Lazy Initialization pattern in Go.

Difficulty: Easy

Defer creation of an expensive object until it's first needed.

```go
type ExpensiveService struct {
    data []byte
}

func newExpensiveService() *ExpensiveService {
    time.Sleep(2 * time.Second) // expensive init
    data, _ := os.ReadFile("large_config.json")
    return &ExpensiveService{data: data}
}

type ServiceContainer struct {
    expensiveOnce sync.Once
    expensive     *ExpensiveService
}

func (c *ServiceContainer) GetExpensive() *ExpensiveService {
    c.expensiveOnce.Do(func() {
        c.expensive = newExpensiveService()
    })
    return c.expensive
}

// With generics (Go 1.18+)
type Lazy[T any] struct {
    once  sync.Once
    val   T
    init  func() T
}

func NewLazy[T any](init func() T) *Lazy[T] {
    return &Lazy[T]{init: init}
}

func (l *Lazy[T]) Get() T {
    l.once.Do(func() { l.val = l.init() })
    return l.val
}

// Usage
lazyDB := NewLazy(func() *sql.DB {
    db, _ := sql.Open("postgres", dsn)
    return db
})
db := lazyDB.Get() // initialises only on first call
```

Interview tip: "Lazy initialisation with sync.Once is safe for concurrent use. Without it, you'd need a mutex and nil check — error-prone."

Q22

What is the difference between Factory Method and Abstract Factory?

Difficulty: Medium

Factory Method: creates ONE product, subclasses decide which type. Abstract Factory: creates FAMILIES of related products, ensuring they work together.

```go
// Factory Method — one product
type Logger interface { Log(string) }

func NewLogger(logType string) Logger {
    switch logType {
    case "json": return &JSONLogger{}
    case "text": return &TextLogger{}
    default: return &TextLogger{}
    }
}

// Abstract Factory — families of products
type UIKit interface {
    Button() Button
    Input() Input
    Dialog() Dialog
}

// MaterialUIKit — all Material Design components
type MaterialUIKit struct{}
func (m MaterialUIKit) Button() Button { return &MaterialButton{} }
func (m MaterialUIKit) Input() Input   { return &MaterialInput{} }
func (m MaterialUIKit) Dialog() Dialog { return &MaterialDialog{} }

// BootstrapUIKit — all Bootstrap components
type BootstrapUIKit struct{}
func (b BootstrapUIKit) Button() Button { return &BootstrapButton{} }
func (b BootstrapUIKit) Input() Input   { return &BootstrapInput{} }
func (b BootstrapUIKit) Dialog() Dialog { return &BootstrapDialog{} }
```

Interview tip: "Use Factory Method when you have one product with variants. Use Abstract Factory when you have product families that must be consistent."

3. Structural Patterns

Q23

Implement the Adapter pattern in Go.

Difficulty: Easy

Convert an incompatible interface into the one the client expects.

```go
// Third-party logger with incompatible interface
type LegacyLogger struct{}
func (l *LegacyLogger) WriteLog(severity, msg string) {
    fmt.Printf("[%s] %s\n", severity, msg)
}

// Our system expects this interface
type AppLogger interface {
    Info(msg string)
    Warn(msg string)
    Error(msg string)
}

// Adapter wraps LegacyLogger
type LoggerAdapter struct{ legacy *LegacyLogger }

func NewLoggerAdapter(l *LegacyLogger) AppLogger {
    return &LoggerAdapter{legacy: l}
}

func (a *LoggerAdapter) Info(msg string)  { a.legacy.WriteLog("INFO", msg) }
func (a *LoggerAdapter) Warn(msg string)  { a.legacy.WriteLog("WARN", msg) }
func (a *LoggerAdapter) Error(msg string) { a.legacy.WriteLog("ERROR", msg) }

// Usage — client code uses AppLogger interface
var logger AppLogger = NewLoggerAdapter(&LegacyLogger{})
logger.Info("application started")
logger.Error("something went wrong")
```

Interview tip: "Adapters are the glue between 'what we have' and 'what we need.' They're especially common when integrating third-party SDKs."

Q24

Implement the Decorator pattern in Go.

Difficulty: Medium

Add responsibilities to an object dynamically. Wraps the original without modifying it.

```go
type DataProcessor interface {
    Process(data []byte) ([]byte, error)
}

// Base processor
type JSONProcessor struct{}
func (j *JSONProcessor) Process(data []byte) ([]byte, error) {
    var v any
    if err := json.Unmarshal(data, &v); err != nil { return nil, err }
    return json.Marshal(v) // normalise JSON
}

// Encryption decorator
type EncryptedProcessor struct {
    wrapped DataProcessor
    key     []byte
}

func (e *EncryptedProcessor) Process(data []byte) ([]byte, error) {
    processed, err := e.wrapped.Process(data)
    if err != nil { return nil, err }
    return encrypt(processed, e.key), nil
}

// Compression decorator
type CompressedProcessor struct{ wrapped DataProcessor }
func (c *CompressedProcessor) Process(data []byte) ([]byte, error) {
    processed, err := c.wrapped.Process(data)
    if err != nil { return nil, err }
    return compress(processed), nil
}

// Logging decorator
type LoggedProcessor struct {
    wrapped DataProcessor
    logger  *log.Logger
}
func (l *LoggedProcessor) Process(data []byte) ([]byte, error) {
    start := time.Now()
    result, err := l.wrapped.Process(data)
    l.logger.Printf("processed %d bytes in %v, err=%v",
        len(data), time.Since(start), err)
    return result, err
}

// Compose decorators
processor := &LoggedProcessor{
    wrapped: &CompressedProcessor{
        wrapped: &EncryptedProcessor{
            wrapped: &JSONProcessor{},
            key:     encKey,
        },
    },
```

```go
    logger: log.Default(),
}
```

Interview tip: "Each decorator adds exactly one concern. This is the Open/Closed Principle in action — extend without modifying."

Q25

Implement the Proxy pattern in Go.

Difficulty: Medium

Provide a surrogate that controls access to another object. Used for caching, access control, lazy loading.

```go
type ImageLoader interface {
    Load(path string) []byte
}

// Real implementation — expensive
type RealImageLoader struct{}
func (r *RealImageLoader) Load(path string) []byte {
    fmt.Printf("Loading image from disk: %s\n", path)
    data, _ := os.ReadFile(path)
    return data
}

// Caching proxy
type CachedImageLoader struct {
    real  ImageLoader
    cache map[string][]byte
    mu    sync.RWMutex
}

func NewCachedLoader(real ImageLoader) *CachedImageLoader {
    return &CachedImageLoader{real: real, cache: make(map[string][]byte)}
}

func (c *CachedImageLoader) Load(path string) []byte {
    c.mu.RLock()
    if data, ok := c.cache[path]; ok {
        c.mu.RUnlock()
        fmt.Printf("Cache hit: %s\n", path)
        return data
    }
    c.mu.RUnlock()

    data := c.real.Load(path) // expensive

    c.mu.Lock()
    c.cache[path] = data
    c.mu.Unlock()
    return data
}

// Access control proxy
type AuthImageLoader struct {
    real    ImageLoader
    allowed map[string]bool
}
func (a *AuthImageLoader) Load(path string) []byte {
    if !a.allowed[path] { panic("access denied") }
    return a.real.Load(path)
}
```

Interview tip: "Three proxy types: virtual proxy (lazy loading), caching proxy (memoisation), protection proxy (access control). Know all three."

Q26

Implement the Facade pattern in Go.

Difficulty: Easy

Provide a simplified interface to a complex subsystem.

```go
// Complex subsystems
type AudioDecoder struct{}
func (a *AudioDecoder) Decode(file string) []byte { return nil }

type VideoDecoder struct{}
func (v *VideoDecoder) Decode(file string) []byte { return nil }

type AudioMixer struct{}
func (a *AudioMixer) Mix(audio []byte) []byte { return audio }

type VideoRenderer struct{}
func (v *VideoRenderer) Render(video, audio []byte) []byte { return nil }

type FileWriter struct{}
func (f *FileWriter) Write(data []byte, path string) error { return nil }

// Facade — simple interface hides complexity
type VideoConverter struct {
    audioDecoder *AudioDecoder
    videoDecoder *VideoDecoder
    audioMixer   *AudioMixer
    renderer     *VideoRenderer
    writer       *FileWriter
}

func NewVideoConverter() *VideoConverter {
    return &VideoConverter{
        audioDecoder: &AudioDecoder{},
        videoDecoder: &VideoDecoder{},
        audioMixer:   &AudioMixer{},
        renderer:     &VideoRenderer{},
        writer:       &FileWriter{},
    }
}

func (vc *VideoConverter) Convert(input, output string) error {
    audio := vc.audioDecoder.Decode(input)
    video := vc.videoDecoder.Decode(input)
    mixedAudio := vc.audioMixer.Mix(audio)
    rendered := vc.renderer.Render(video, mixedAudio)
    return vc.writer.Write(rendered, output)
}

// Client only needs to know this
converter := NewVideoConverter()
converter.Convert("input.avi", "output.mp4")
```

Interview tip: "Facades reduce complexity for callers but don't restrict access to subsystems. Clients can still use subsystems directly if they need fine-grained control."

Q27

Implement the Composite pattern in Go.

Difficulty: Medium

Compose objects into tree structures to represent part-whole hierarchies. Treat individual objects and compositions uniformly.

```go
type FileSystemNode interface {
    Name() string
    Size() int64
    Print(indent string)
}

// Leaf
type File struct {
    name string
    size int64
}
func (f *File) Name() string   { return f.name }
func (f *File) Size() int64    { return f.size }
func (f *File) Print(indent string) {
    fmt.Printf("%s■ %s (%d bytes)\n", indent, f.name, f.size)
}

// Composite
type Directory struct {
    name     string
    children []FileSystemNode
}
func (d *Directory) Name() string { return d.name }
func (d *Directory) Size() int64 {
    var total int64
    for _, c := range d.children { total += c.Size() }
    return total
}
func (d *Directory) Add(node FileSystemNode) { d.children = append(d.children, node) }
func (d *Directory) Print(indent string) {
    fmt.Printf("%s■ %s (%d bytes)\n", indent, d.name, d.Size())
    for _, c := range d.children { c.Print(indent + "  ") }
}

// Usage
root := &Directory{name: "root"}
src := &Directory{name: "src"}
src.Add(&File{"main.go", 1024})
src.Add(&File{"utils.go", 512})
root.Add(src)
root.Add(&File{"README.md", 256})
root.Print("")
fmt.Printf("Total: %d bytes\n", root.Size())
```

Interview tip: "Composite is perfect for: file systems, UI component trees, organizational charts, expression trees. Any hierarchy where leaves and nodes are treated the same."

Q28

Implement the Bridge pattern in Go.

Difficulty: Hard

Decouple an abstraction from its implementation so both can vary independently.

```go
// Implementation interface
type Renderer interface {
    RenderCircle(x, y, radius float64)
    RenderSquare(x, y, side float64)
}

// Concrete implementations
type VectorRenderer struct{}
func (v *VectorRenderer) RenderCircle(x, y, r float64) {
    fmt.Printf("Drawing circle at (%.0f,%.0f) r=%.0f as vector\n", x, y, r)
}
func (v *VectorRenderer) RenderSquare(x, y, s float64) {
    fmt.Printf("Drawing square at (%.0f,%.0f) side=%.0f as vector\n", x, y, s)
}

type RasterRenderer struct{}
func (r *RasterRenderer) RenderCircle(x, y, rad float64) {
    fmt.Printf("Drawing circle at (%.0f,%.0f) r=%.0f as pixels\n", x, y, rad)
}
func (r *RasterRenderer) RenderSquare(x, y, s float64) {
    fmt.Printf("Drawing square at (%.0f,%.0f) side=%.0f as pixels\n", x, y, s)
}

// Abstraction
type Shape interface {
    Draw()
    Resize(factor float64)
}

type Circle struct {
    renderer Renderer
    x, y, radius float64
}
func (c *Circle) Draw()               { c.renderer.RenderCircle(c.x, c.y, c.radius) }
func (c *Circle) Resize(f float64)    { c.radius *= f }

type Square struct {
    renderer Renderer
    x, y, side float64
}
func (s *Square) Draw()           { s.renderer.RenderSquare(s.x, s.y, s.side) }
func (s *Square) Resize(f float64) { s.side *= f }

// Can mix any shape with any renderer
c1 := &Circle{renderer: &VectorRenderer{}, x: 0, y: 0, radius: 5}
c2 := &Circle{renderer: &RasterRenderer{}, x: 0, y: 0, radius: 5}
c1.Draw() // vector circle
c2.Draw() // raster circle
```

Interview tip: "Bridge prevents 2×N class explosion. Without it, you'd need VectorCircle, RasterCircle, VectorSquare, RasterSquare... With Bridge: 2 shapes + 2 renderers = 4 combinations."

Q29

Implement the Flyweight pattern in Go.

Difficulty: Hard

Share common state among many fine-grained objects to save memory.

```go
// Intrinsic state (shared) — immutable
type CharStyle struct {
    font  string
    size  int
    color string
}

// Flyweight factory
type StyleFactory struct {
    styles map[string]*CharStyle
    mu     sync.RWMutex
}

func (f *StyleFactory) GetStyle(font string, size int, color string) *CharStyle {
    key := fmt.Sprintf("%s-%d-%s", font, size, color)
    f.mu.RLock()
    if s, ok := f.styles[key]; ok {
        f.mu.RUnlock()
        return s
    }
    f.mu.RUnlock()

    f.mu.Lock()
    defer f.mu.Unlock()
    style := &CharStyle{font: font, size: size, color: color}
    f.styles[key] = style
    return style
}

// Extrinsic state (unique per object) — position
type Character struct {
    char  rune
    x, y  int
    style *CharStyle // shared flyweight
}

// In a document with 1M characters, there may only be 10 unique styles
// Memory: 1M * (char+x+y+pointer) + 10 * CharStyle
// vs without Flyweight: 1M * (char+x+y+font+size+color)
factory := &StyleFactory{styles: make(map[string]*CharStyle)}
boldStyle := factory.GetStyle("Arial", 14, "black")

doc := make([]Character, 0, 1000)
for i, ch := range "Hello World" {
    doc = append(doc, Character{char: ch, x: i * 10, y: 0, style: boldStyle})
}
```

Interview tip: "Flyweight trades CPU (factory lookup) for memory. Use when you have thousands of objects with mostly shared state — game entities, text editors, map tiles."

Q30
What is the Null Object pattern?

Difficulty: Easy

Provide a default object with do-nothing behaviour instead of nil checks.

```go
type Logger interface {
    Info(msg string)
    Error(msg string)
}

// Real logger
type ConsoleLogger struct{}
func (c *ConsoleLogger) Info(msg string)  { fmt.Println("[INFO]", msg) }
func (c *ConsoleLogger) Error(msg string) { fmt.Println("[ERROR]", msg) }

// Null object — does nothing
type NullLogger struct{}
func (n NullLogger) Info(msg string)  {}
func (n NullLogger) Error(msg string) {}

type Service struct{ logger Logger }

func NewService(logger Logger) *Service {
    if logger == nil { logger = NullLogger{} } // safe default
    return &Service{logger: logger}
}

// Usage — no nil checks in service code
svc := NewService(nil) // uses NullLogger, no panic
svc.logger.Info("started") // safe call

// Tests can use NullLogger to suppress output
testSvc := NewService(NullLogger{})
```

Interview tip: "Null Object eliminates defensive nil checks throughout your code. The object 'does nothing' instead of crashing. Common for loggers, metrics, event handlers."

Q31
Implement the Module pattern in Go.

Difficulty: Easy

Encapsulate related functionality with controlled access using packages and unexported identifiers.

```go
// payment/payment.go — module with private state
package payment

type processor struct {
    apiKey    string
    endpoint  string
    client    *http.Client
}

var defaultProcessor *processor

func Init(apiKey, endpoint string) {
    defaultProcessor = &processor{
        apiKey:   apiKey,
        endpoint: endpoint,
        client:   &http.Client{Timeout: 30 * time.Second},
    }
}

// Public API — exported
func Charge(amount float64, token string) (*Receipt, error) {
    if defaultProcessor == nil {
        return nil, errors.New("payment not initialised: call Init()")
    }
    return defaultProcessor.charge(amount, token)
}

// Private implementation — unexported
func (p *processor) charge(amount float64, token string) (*Receipt, error) {
    // calls payment API
    return &Receipt{Amount: amount}, nil
}

// main.go
// payment.Init(os.Getenv("STRIPE_KEY"), "https://api.stripe.com")
// receipt, err := payment.Charge(99.99, "tok_visa")
```

Interview tip: "Go packages are modules. Unexported identifiers are the module's private state. This is the Go-idiomatic way to encapsulate."

Q32

What is the Repository pattern vs DAO pattern?

Difficulty: Medium

Repository: domain-focused, returns domain objects, hides all persistence details. DAO (Data Access Object): database-focused, often maps directly to DB tables, may expose query methods.

```go
// DAO — database-centric, exposes DB operations
type UserDAO struct{ db *sql.DB }
func (d *UserDAO) FindByID(id int) (*UserRow, error)       { /* SELECT */ }
func (d *UserDAO) FindByEmail(email string) (*UserRow, error) { /* SELECT */ }
func (d *UserDAO) Insert(row *UserRow) error               { /* INSERT */ }
func (d *UserDAO) Update(row *UserRow) error               { /* UPDATE */ }
func (d *UserDAO) Delete(id int) error                     { /* DELETE */ }

// Repository — domain-centric, returns domain objects
type UserRepository interface {
    GetByID(ctx context.Context, id int) (*User, error)
    GetByEmail(ctx context.Context, email string) (*User, error)
    Save(ctx context.Context, user *User) error
    Delete(ctx context.Context, id int) error
    ListActive(ctx context.Context) ([]*User, error) // domain concept
}

type postgresUserRepository struct{ db *sql.DB }

func (r *postgresUserRepository) GetByID(ctx context.Context, id int) (*User, error) {
    row := r.db.QueryRowContext(ctx, "SELECT * FROM users WHERE id=$1 AND deleted_at IS NULL", id)
    return scanUser(row)
}

func (r *postgresUserRepository) ListActive(ctx context.Context) ([]*User, error) {
    rows, err := r.db.QueryContext(ctx,
        "SELECT * FROM users WHERE active=true AND deleted_at IS NULL ORDER BY created_at DESC")
    // ...
    return users, err
}
```

Interview tip: "Repository abstracts persistence completely. Change from PostgreSQL to MongoDB without touching business logic. DAO is just a thin wrapper over SQL."

---

4. Behavioural Patterns

Q33

Implement the Observer pattern in Go.

Difficulty: Medium

Define a one-to-many dependency. When one object changes state, all dependents are notified automatically.

```go
type Event struct {
    Type    string
    Payload any
}

type Observer interface { OnEvent(e Event) }

type EventBus struct {
    mu          sync.RWMutex
    subscribers map[string][]Observer
}

func NewEventBus() *EventBus {
    return &EventBus{subscribers: make(map[string][]Observer)}
}

func (eb *EventBus) Subscribe(eventType string, obs Observer) {
    eb.mu.Lock(); defer eb.mu.Unlock()
    eb.subscribers[eventType] = append(eb.subscribers[eventType], obs)
}

func (eb *EventBus) Unsubscribe(eventType string, obs Observer) {
    eb.mu.Lock(); defer eb.mu.Unlock()
    subs := eb.subscribers[eventType]
    for i, s := range subs {
        if s == obs {
            eb.subscribers[eventType] = append(subs[:i], subs[i+1:]...)
            return
        }
    }
}

func (eb *EventBus) Publish(e Event) {
    eb.mu.RLock()
    subs := make([]Observer, len(eb.subscribers[e.Type]))
    copy(subs, eb.subscribers[e.Type])
    eb.mu.RUnlock()

    for _, sub := range subs { go sub.OnEvent(e) } // async notify
}

// Concrete observers
type EmailObserver struct{ addr string }
func (eo *EmailObserver) OnEvent(e Event) {
    fmt.Printf("Email to %s: %v\n", eo.addr, e.Payload)
}

type AuditObserver struct{ log []Event }
func (ao *AuditObserver) OnEvent(e Event) { ao.log = append(ao.log, e) }
```

Interview tip: "Sync vs async notify: async (goroutines) means one slow observer doesn't block others. But error handling is harder. Discuss trade-offs."

Q34

Implement the Strategy pattern in Go.

Difficulty: Easy

Define a family of algorithms, make them interchangeable.

```go
type SortStrategy interface {
    Sort(data []int) []int
    Name() string
}

type BubbleSort struct{}
func (b BubbleSort) Name() string { return "bubble" }
func (b BubbleSort) Sort(data []int) []int {
    a := append([]int{}, data...)
    n := len(a)
    for i := 0; i < n-1; i++ {
        for j := 0; j < n-i-1; j++ {
            if a[j] > a[j+1] { a[j], a[j+1] = a[j+1], a[j] }
        }
    }
    return a
}

type MergeSort struct{}
func (m MergeSort) Name() string { return "merge" }
func (m MergeSort) Sort(data []int) []int {
    if len(data) <= 1 { return data }
    mid := len(data) / 2
    left := m.Sort(data[:mid])
    right := m.Sort(data[mid:])
    return merge(left, right)
}

type Sorter struct {
    strategy SortStrategy
}
func (s *Sorter) SetStrategy(st SortStrategy) { s.strategy = st }
func (s *Sorter) Sort(data []int) []int {
    fmt.Printf("Using %s sort\n", s.strategy.Name())
    return s.strategy.Sort(data)
}

sorter := &Sorter{strategy: MergeSort{}}
result := sorter.Sort([]int{5, 3, 1, 4, 2})

// Switch strategy based on data size
if len(data) < 10 {
    sorter.SetStrategy(BubbleSort{})
} else {
    sorter.SetStrategy(MergeSort{})
}
```

Interview tip: "In Go, every interface is a strategy. Whenever you inject a dependency, you're using the Strategy pattern."

Q35

Implement the Command pattern in Go.

Difficulty: Medium

Encapsulate a request as an object, allowing undo, queuing, logging.

```go
type Command interface {
    Execute() error
    Undo() error
    Description() string
}

// Concrete command
type TransferMoneyCommand struct {
    from   *Account
    to     *Account
    amount float64
}

func (t *TransferMoneyCommand) Execute() error {
    if err := t.from.Withdraw(t.amount); err != nil { return err }
    t.to.Deposit(t.amount)
    return nil
}

func (t *TransferMoneyCommand) Undo() error {
    t.to.Withdraw(t.amount)
    t.from.Deposit(t.amount)
    return nil
}

func (t *TransferMoneyCommand) Description() string {
    return fmt.Sprintf("Transfer %.2f from %s to %s", t.amount, t.from.ID, t.to.ID)
}

// Command history — supports undo
type CommandHistory struct {
    history []Command
    mu      sync.Mutex
}

func (ch *CommandHistory) Execute(cmd Command) error {
    ch.mu.Lock(); defer ch.mu.Unlock()
    if err := cmd.Execute(); err != nil { return err }
    ch.history = append(ch.history, cmd)
    return nil
}

func (ch *CommandHistory) Undo() error {
    ch.mu.Lock(); defer ch.mu.Unlock()
    if len(ch.history) == 0 { return errors.New("nothing to undo") }
    last := ch.history[len(ch.history)-1]
    ch.history = ch.history[:len(ch.history)-1]
    return last.Undo()
}

func (ch *CommandHistory) Log() {
    for i, cmd := range ch.history {
        fmt.Printf("%d: %s\n", i+1, cmd.Description())
```

```go
    }
}
```

Interview tip: "Command pattern is in every text editor (Ctrl+Z), transactional systems (rollback), and CLI frameworks. The Undo() method is what distinguishes it."

Q36

Implement the Chain of Responsibility pattern in Go.

Difficulty: Medium

Pass a request along a chain of handlers. Each handler decides to handle it or pass it on.

```go
type Request struct {
    Amount  float64
    UserID  int
    Country string
}

type Handler interface {
    Handle(req Request) (bool, error)
    SetNext(h Handler)
}

type BaseHandler struct{ next Handler }
func (b *BaseHandler) SetNext(h Handler) { b.next = h }
func (b *BaseHandler) PassToNext(req Request) (bool, error) {
    if b.next != nil { return b.next.Handle(req) }
    return true, nil // end of chain — approve
}

// Fraud check
type FraudCheckHandler struct{ BaseHandler }
func (f *FraudCheckHandler) Handle(req Request) (bool, error) {
    if req.Amount > 10000 && req.Country == "XX" {
        return false, errors.New("fraud detected")
    }
    return f.PassToNext(req)
}

// Limit check
type LimitCheckHandler struct {
    BaseHandler
    limit float64
}
func (l *LimitCheckHandler) Handle(req Request) (bool, error) {
    if req.Amount > l.limit {
        return false, fmt.Errorf("amount %.2f exceeds limit %.2f", req.Amount, l.limit)
    }
    return l.PassToNext(req)
}

// KYC check
type KYCHandler struct{ BaseHandler }
func (k *KYCHandler) Handle(req Request) (bool, error) {
    if !isKYCVerified(req.UserID) {
        return false, errors.New("KYC not verified")
    }
    return k.PassToNext(req)
}

// Build chain
fraud := &FraudCheckHandler{}
limit := &LimitCheckHandler{limit: 5000}
kyc := &KYCHandler{}
fraud.SetNext(limit)
```

```go
limit.SetNext(kyc)

approved, err := fraud.Handle(Request{Amount: 100, UserID: 1, Country: "US"})
```

Interview tip: "Chain of Responsibility is great for validation pipelines, middleware, and approval workflows. The order of handlers matters — put cheap checks first."

Q37

Implement the Iterator pattern in Go.

Difficulty: Easy

Provide sequential access to elements without exposing the underlying structure.

```go
// Generic iterator interface
type Iterator[T any] interface {
    HasNext() bool
    Next() T
}

// Slice iterator
type SliceIterator[T any] struct {
    data  []T
    index int
}

func NewSliceIterator[T any](data []T) *SliceIterator[T] {
    return &SliceIterator[T]{data: data}
}

func (it *SliceIterator[T]) HasNext() bool { return it.index < len(it.data) }
func (it *SliceIterator[T]) Next() T {
    v := it.data[it.index]; it.index++; return v
}

// Tree in-order iterator
type TreeNode struct {
    Val   int
    Left  *TreeNode
    Right *TreeNode
}

type InOrderIterator struct{ stack []*TreeNode }

func NewInOrderIterator(root *TreeNode) *InOrderIterator {
    it := &InOrderIterator{}
    it.pushLeft(root)
    return it
}

func (it *InOrderIterator) pushLeft(node *TreeNode) {
    for node != nil { it.stack = append(it.stack, node); node = node.Left }
}

func (it *InOrderIterator) HasNext() bool { return len(it.stack) > 0 }
func (it *InOrderIterator) Next() int {
    n := it.stack[len(it.stack)-1]
    it.stack = it.stack[:len(it.stack)-1]
    it.pushLeft(n.Right)
    return n.Val
}

// Usage
it := NewInOrderIterator(root)
for it.HasNext() { fmt.Println(it.Next()) }
```

Interview tip: "In Go, iterators are often implemented as channel generators or closures with range. The Go 1.22+ range-over-func feature makes custom iterators ergonomic."

Q38

Implement the State pattern in Go.

Difficulty: Medium

Allow an object to alter its behaviour when its internal state changes.

```go
type OrderState interface {
    Name() string
    Cancel(*Order) error
    Ship(*Order) error
    Deliver(*Order) error
}

type Order struct {
    ID    string
    state OrderState
}

func (o *Order) SetState(s OrderState) { o.state = s }
func (o *Order) Cancel() error         { return o.state.Cancel(o) }
func (o *Order) Ship() error           { return o.state.Ship(o) }
func (o *Order) Deliver() error        { return o.state.Deliver(o) }
func (o *Order) State() string         { return o.state.Name() }

// States
type PendingState struct{}
func (p PendingState) Name() string { return "pending" }
func (p PendingState) Cancel(o *Order) error {
    o.SetState(CancelledState{}); return nil
}
func (p PendingState) Ship(o *Order) error {
    o.SetState(ShippedState{}); return nil
}
func (p PendingState) Deliver(o *Order) error {
    return errors.New("cannot deliver pending order")
}

type ShippedState struct{}
func (s ShippedState) Name() string { return "shipped" }
func (s ShippedState) Cancel(*Order) error { return errors.New("cannot cancel shipped order") }
func (s ShippedState) Ship(*Order) error   { return errors.New("already shipped") }
func (s ShippedState) Deliver(o *Order) error {
    o.SetState(DeliveredState{}); return nil
}

type DeliveredState struct{}
func (d DeliveredState) Name() string              { return "delivered" }
func (d DeliveredState) Cancel(*Order) error       { return errors.New("already delivered") }
func (d DeliveredState) Ship(*Order) error         { return errors.New("already delivered") }
func (d DeliveredState) Deliver(*Order) error      { return errors.New("already delivered") }

type CancelledState struct{}
func (c CancelledState) Name() string              { return "cancelled" }
func (c CancelledState) Cancel(*Order) error       { return errors.New("already cancelled") }
func (c CancelledState) Ship(*Order) error         { return errors.New("order is cancelled") }
func (c CancelledState) Deliver(*Order) error      { return errors.New("order is cancelled") }
```

Interview tip: "State vs Strategy: both use interfaces. State holds a reference back to the context (Order) and transitions. Strategy is stateless and interchangeable."

Q39

Implement the Template Method pattern in Go.

Difficulty: Medium

Define the skeleton of an algorithm. Let subclasses override specific steps without changing the structure.

```go
// Template (abstract)
type DataMigrator interface {
    ReadSource() ([]map[string]any, error)
    Transform(rows []map[string]any) ([]map[string]any, error)
    WriteDestination(rows []map[string]any) error
    Validate(rows []map[string]any) error
}

// Template method — the algorithm skeleton
func RunMigration(m DataMigrator) error {
    fmt.Println("Step 1: Reading source...")
    rows, err := m.ReadSource()
    if err != nil { return fmt.Errorf("read: %w", err) }

    fmt.Println("Step 2: Validating...")
    if err := m.Validate(rows); err != nil { return fmt.Errorf("validate: %w", err) }

    fmt.Println("Step 3: Transforming...")
    transformed, err := m.Transform(rows)
    if err != nil { return fmt.Errorf("transform: %w", err) }

    fmt.Println("Step 4: Writing destination...")
    if err := m.WriteDestination(transformed); err != nil {
        return fmt.Errorf("write: %w", err)
    }
    return nil
}

// Concrete implementation
type CSVToPostgresMigrator struct {
    csvPath string
    db      *sql.DB
}

func (m *CSVToPostgresMigrator) ReadSource() ([]map[string]any, error) {
    // read CSV
    return nil, nil
}
func (m *CSVToPostgresMigrator) Validate(rows []map[string]any) error {
    // validate required fields
    return nil
}
func (m *CSVToPostgresMigrator) Transform(rows []map[string]any) ([]map[string]any, error) {
    // normalise data
    return rows, nil
}
func (m *CSVToPostgresMigrator) WriteDestination(rows []map[string]any) error {
    // bulk insert to Postgres
    return nil
}
```

Interview tip: "Template Method is Go's alternative to inheritance-based method overriding. Define the algorithm skeleton via an interface, concrete types fill in the steps."

Q40

Implement the Mediator pattern in Go.

Difficulty: Medium

Define an object that encapsulates how a set of objects interact. Promotes loose coupling.

```go
type ChatMediator interface {
    Send(msg, from string)
    Register(user *ChatUser)
}

type ChatUser struct {
    name     string
    mediator ChatMediator
}

func (u *ChatUser) Send(msg string) { u.mediator.Send(msg, u.name) }
func (u *ChatUser) Receive(msg, from string) {
    fmt.Printf("[%s] received from %s: %s\n", u.name, from, msg)
}

type ChatRoom struct {
    users map[string]*ChatUser
    mu    sync.RWMutex
}

func NewChatRoom() *ChatRoom { return &ChatRoom{users: make(map[string]*ChatUser)} }

func (r *ChatRoom) Register(user *ChatUser) {
    r.mu.Lock(); defer r.mu.Unlock()
    r.users[user.name] = user
    user.mediator = r
}

func (r *ChatRoom) Send(msg, from string) {
    r.mu.RLock(); defer r.mu.RUnlock()
    for name, user := range r.users {
        if name != from { go user.Receive(msg, from) }
    }
}

// Usage
room := NewChatRoom()
alice := &ChatUser{name: "Alice"}
bob := &ChatUser{name: "Bob"}
charlie := &ChatUser{name: "Charlie"}
room.Register(alice)
room.Register(bob)
room.Register(charlie)
alice.Send("Hello everyone!")
```

Interview tip: "Without Mediator, each user would need a reference to every other user — $O(n^2)$ connections. With Mediator, each user knows only the mediator — $O(n)$

connections."

Q41

Implement the Visitor pattern in Go.

Difficulty: Hard

Add new operations to existing object structures without modifying them.

```go
type ShapeVisitor interface {
    VisitCircle(c *Circle)
    VisitRectangle(r *Rectangle)
    VisitTriangle(t *Triangle)
}

type Shape interface{ Accept(v ShapeVisitor) }

type Circle struct{ Radius float64 }
func (c *Circle) Accept(v ShapeVisitor) { v.VisitCircle(c) }

type Rectangle struct{ Width, Height float64 }
func (r *Rectangle) Accept(v ShapeVisitor) { v.VisitRectangle(r) }

type Triangle struct{ Base, Height float64 }
func (t *Triangle) Accept(v ShapeVisitor) { v.VisitTriangle(t) }

// Area visitor
type AreaVisitor struct{ Total float64 }
func (a *AreaVisitor) VisitCircle(c *Circle) {
    a.Total += math.Pi * c.Radius * c.Radius
}
func (a *AreaVisitor) VisitRectangle(r *Rectangle) {
    a.Total += r.Width * r.Height
}
func (a *AreaVisitor) VisitTriangle(t *Triangle) {
    a.Total += 0.5 * t.Base * t.Height
}

// Export visitor — adds a new operation without touching shapes
type SVGExporter struct{ buffer strings.Builder }
func (s *SVGExporter) VisitCircle(c *Circle) {
    s.buffer.WriteString(fmt.Sprintf(`<circle r="%.0f"/>`, c.Radius))
}
func (s *SVGExporter) VisitRectangle(r *Rectangle) {
    s.buffer.WriteString(fmt.Sprintf(`<rect w="%.0f" h="%.0f"/>`, r.Width, r.Height))
}
func (s *SVGExporter) VisitTriangle(t *Triangle) {
    s.buffer.WriteString(fmt.Sprintf(`<polygon base="%.0f"/>`, t.Base))
}

shapes := []Shape{&Circle{5}, &Rectangle{3, 4}, &Triangle{6, 8}}
area := &AreaVisitor{}
for _, s := range shapes { s.Accept(area) }
fmt.Printf("Total area: %.2f\n", area.Total)
```

Interview tip: "Visitor violates OCP for the element hierarchy (adding new shapes requires updating all visitors). It's the trade-off: easy to add operations, hard to add elements."

Q42

Implement the Memento pattern in Go.

Difficulty: Medium

Capture and restore an object's internal state without violating encapsulation.

```go
// Originator
type TextEditor struct {
    content string
    cursor  int
}

// Memento — snapshot of state
type EditorMemento struct {
    content string
    cursor  int
}

func (e *TextEditor) Type(text string) {
    e.content = e.content[:e.cursor] + text + e.content[e.cursor:]
    e.cursor += len(text)
}

func (e *TextEditor) Save() *EditorMemento {
    return &EditorMemento{content: e.content, cursor: e.cursor}
}

func (e *TextEditor) Restore(m *EditorMemento) {
    e.content = m.content
    e.cursor = m.cursor
}

// Caretaker — manages mementos
type History struct{ mementos []*EditorMemento }

func (h *History) Push(m *EditorMemento) {
    h.mementos = append(h.mementos, m)
}

func (h *History) Pop() *EditorMemento {
    if len(h.mementos) == 0 { return nil }
    m := h.mementos[len(h.mementos)-1]
    h.mementos = h.mementos[:len(h.mementos)-1]
    return m
}

// Usage — Ctrl+Z support
editor := &TextEditor{}
history := &History{}

editor.Type("Hello")
history.Push(editor.Save()) // save state 1

editor.Type(" World")
history.Push(editor.Save()) // save state 2

editor.Type("!!!")
fmt.Println(editor.content) // "Hello World!!!"
```

```go
editor.Restore(history.Pop()) // undo
fmt.Println(editor.content)   // "Hello World"

editor.Restore(history.Pop()) // undo again
fmt.Println(editor.content)   // "Hello"
```

Interview tip: "Memento is the pattern behind Ctrl+Z in any editor. Keep mementos small — store diffs instead of full state for large documents."

Q43

Implement the Interpreter pattern in Go.

Difficulty: Hard

Given a language, define a representation for its grammar and provide an interpreter.

```go
// Simple math expression interpreter
type Expression interface { Interpret() int }

type NumberExpr struct{ val int }
func (n NumberExpr) Interpret() int { return n.val }

type AddExpr struct{ left, right Expression }
func (a AddExpr) Interpret() int { return a.left.Interpret() + a.right.Interpret() }

type SubExpr struct{ left, right Expression }
func (s SubExpr) Interpret() int { return s.left.Interpret() - s.right.Interpret() }

type MulExpr struct{ left, right Expression }
func (m MulExpr) Interpret() int { return m.left.Interpret() * m.right.Interpret() }

// Simple parser for "3 + 4 * 2"
func parse(tokens []string) Expression {
    // For interview purposes — simplified left-to-right parsing
    if len(tokens) == 1 {
        n, _ := strconv.Atoi(tokens[0])
        return NumberExpr{n}
    }
    left, _ := strconv.Atoi(tokens[0])
    right, _ := strconv.Atoi(tokens[2])
    switch tokens[1] {
    case "+": return AddExpr{NumberExpr{left}, NumberExpr{right}}
    case "-": return SubExpr{NumberExpr{left}, NumberExpr{right}}
    case "*": return MulExpr{NumberExpr{left}, NumberExpr{right}}
    }
    return NumberExpr{left}
}

expr := AddExpr{
    NumberExpr{3},
    MulExpr{NumberExpr{4}, NumberExpr{2}},
}
fmt.Println(expr.Interpret()) // 11 (3 + 4*2)
```

Interview tip: "Interpreter is used in: rule engines, query parsers, config DSLs. In practice, most teams use a parser generator (goyacc, antlr4-go) instead of hand-rolling."

Q44
What is the Specification pattern?

Difficulty: Hard

Encapsulate business rules as combinable predicates.

```go
type Specification[T any] interface {
    IsSatisfiedBy(item T) bool
    And(other Specification[T]) Specification[T]
    Or(other Specification[T]) Specification[T]
    Not() Specification[T]
}

type BaseSpec[T any] struct {
    fn func(T) bool
}

func Spec[T any](fn func(T) bool) Specification[T] { return &BaseSpec[T]{fn: fn} }

func (s *BaseSpec[T]) IsSatisfiedBy(item T) bool { return s.fn(item) }
func (s *BaseSpec[T]) And(other Specification[T]) Specification[T] {
    return Spec(func(item T) bool { return s.fn(item) && other.IsSatisfiedBy(item) })
}
func (s *BaseSpec[T]) Or(other Specification[T]) Specification[T] {
    return Spec(func(item T) bool { return s.fn(item) || other.IsSatisfiedBy(item) })
}
func (s *BaseSpec[T]) Not() Specification[T] {
    return Spec(func(item T) bool { return !s.fn(item) })
}

// Usage
type Product struct{ Name string; Price float64; Category string }

affordable := Spec(func(p Product) bool { return p.Price < 100 })
electronics := Spec(func(p Product) bool { return p.Category == "electronics" })
affordable_electronics := affordable.And(electronics)

products := []Product{
    {"Laptop", 999, "electronics"},
    {"Cable", 9.99, "electronics"},
    {"Book", 29.99, "books"},
}

for _, p := range products {
    if affordable_electronics.IsSatisfiedBy(p) {
        fmt.Println(p.Name) // Cable
    }
}
```

Interview tip: "Specification is common in e-commerce filtering and DDD. It makes complex business rules composable and testable in isolation."

Q45

Implement a middleware pipeline pattern in Go.

Difficulty: Medium

Chain handlers where each can pre/post process and decide to continue.

```go
type Context struct {
    Request  *http.Request
    Response http.ResponseWriter
    UserID   int
    values   map[string]any
}

func (c *Context) Set(key string, val any) { c.values[key] = val }
func (c *Context) Get(key string) any      { return c.values[key] }

type HandlerFunc func(ctx *Context) error

type Middleware func(HandlerFunc) HandlerFunc

func Chain(h HandlerFunc, middlewares ...Middleware) HandlerFunc {
    for i := len(middlewares) - 1; i >= 0; i-- {
        h = middlewares[i](h)
    }
    return h
}

// Middlewares
func Logger(next HandlerFunc) HandlerFunc {
    return func(ctx *Context) error {
        start := time.Now()
        err := next(ctx)
        fmt.Printf("%s %s %v\n", ctx.Request.Method, ctx.Request.URL.Path, time.Since(start))
        return err
    }
}

func Auth(next HandlerFunc) HandlerFunc {
    return func(ctx *Context) error {
        token := ctx.Request.Header.Get("Authorization")
        userID, err := validateToken(token)
        if err != nil {
            ctx.Response.WriteHeader(http.StatusUnauthorized)
            return err
        }
        ctx.UserID = userID
        return next(ctx)
    }
}

func RateLimit(limit int) Middleware {
    limiter := NewRateLimiter(limit)
    return func(next HandlerFunc) HandlerFunc {
        return func(ctx *Context) error {
            ip := ctx.Request.RemoteAddr
            if !limiter.Allow(ip) {
                ctx.Response.WriteHeader(http.StatusTooManyRequests)
                return errors.New("rate limit exceeded")
            }
```

```go
            return next(ctx)
        }
    }
}

// Compose
handler := Chain(
    myBusinessLogic,
    Logger,
    Auth,
    RateLimit(100),
)
```

Interview tip: "This is exactly how net/http, Gin, and Echo middleware work. The reverse loop ensures middlewares execute in the order they're listed."

---

5. Caching Systems

Q46

Implement an LRU cache with O(1) operations.

Difficulty: Hard

HashMap for O(1) lookup + doubly linked list for O(1) move-to-front and eviction.

```go
type LRUNode struct {
    key, val   int
    prev, next *LRUNode
}

type LRUCache struct {
    cap        int
    cache      map[int]*LRUNode
    head, tail *LRUNode // sentinels
    mu         sync.RWMutex
}

func NewLRU(cap int) *LRUCache {
    h, t := &LRUNode{}, &LRUNode{}
    h.next = t; t.prev = h
    return &LRUCache{cap: cap, cache: make(map[int]*LRUNode), head: h, tail: t}
}

func (c *LRUCache) remove(n *LRUNode) {
    n.prev.next = n.next
    n.next.prev = n.prev
}

func (c *LRUCache) insertFront(n *LRUNode) {
    n.next = c.head.next; n.prev = c.head
    c.head.next.prev = n; c.head.next = n
}

func (c *LRUCache) Get(key int) (int, bool) {
    c.mu.Lock(); defer c.mu.Unlock()
    if n, ok := c.cache[key]; ok {
        c.remove(n); c.insertFront(n)
        return n.val, true
    }
    return 0, false
}

func (c *LRUCache) Put(key, val int) {
    c.mu.Lock(); defer c.mu.Unlock()
    if n, ok := c.cache[key]; ok {
        n.val = val; c.remove(n); c.insertFront(n); return
    }
    n := &LRUNode{key: key, val: val}
    c.cache[key] = n; c.insertFront(n)
    if len(c.cache) > c.cap {
        lru := c.tail.prev
        c.remove(lru); delete(c.cache, lru.key)
    }
}
```

Interview tip: "Two sentinel nodes (head/tail) eliminate nil checks in remove() and insertFront(). Without sentinels, every operation needs 4-6 edge case checks."

Q47

Implement an LFU (Least Frequently Used) cache.

Difficulty: Hard

Evict least frequently used. Track frequency per key; on tie, evict least recently used among equal-frequency keys.

```go
type LFUCache struct {
    cap, minFreq int
    keyToEntry   map[int]*lfuEntry
    freqToKeys   map[int]*list.List
    keyToElem    map[int]*list.Element
}

type lfuEntry struct{ key, val, freq int }

func NewLFU(cap int) *LFUCache {
    return &LFUCache{
        cap:        cap,
        keyToEntry: make(map[int]*lfuEntry),
        freqToKeys: make(map[int]*list.List),
        keyToElem:  make(map[int]*list.Element),
    }
}

func (c *LFUCache) increment(key int, e *lfuEntry) {
    f := e.freq
    c.freqToKeys[f].Remove(c.keyToElem[key])
    if c.freqToKeys[f].Len() == 0 && f == c.minFreq { c.minFreq++ }
    e.freq++
    if c.freqToKeys[e.freq] == nil { c.freqToKeys[e.freq] = list.New() }
    c.keyToElem[key] = c.freqToKeys[e.freq].PushFront(key)
}

func (c *LFUCache) Get(key int) int {
    e, ok := c.keyToEntry[key]
    if !ok { return -1 }
    c.increment(key, e)
    return e.val
}

func (c *LFUCache) Put(key, val int) {
    if c.cap == 0 { return }
    if e, ok := c.keyToEntry[key]; ok {
        e.val = val; c.increment(key, e); return
    }
    if len(c.keyToEntry) == c.cap {
        lst := c.freqToKeys[c.minFreq]
        evict := lst.Back()
        evictKey := lst.Remove(evict).(int)
        delete(c.keyToEntry, evictKey)
        delete(c.keyToElem, evictKey)
    }
    e := &lfuEntry{key: key, val: val, freq: 1}
    c.keyToEntry[key] = e
    if c.freqToKeys[1] == nil { c.freqToKeys[1] = list.New() }
    c.keyToElem[key] = c.freqToKeys[1].PushFront(key)
    c.minFreq = 1
}
```

Interview tip: "LFU is O(1) only with this two-map approach. A naive implementation scans all entries — O(n) eviction. The minFreq tracking is the key insight."

Q48

Design a thread-safe in-memory cache with TTL.

Difficulty: Medium

Items expire automatically after a duration. Background goroutine cleans up expired entries.

```go
type cacheEntry struct {
    value     any
    expiresAt time.Time
}

type TTLCache struct {
    mu      sync.RWMutex
    entries map[string]*cacheEntry
    stop    chan struct{}
}

func NewTTLCache(cleanupInterval time.Duration) *TTLCache {
    c := &TTLCache{
        entries: make(map[string]*cacheEntry),
        stop:    make(chan struct{}),
    }
    go c.cleanup(cleanupInterval)
    return c
}

func (c *TTLCache) Set(key string, val any, ttl time.Duration) {
    c.mu.Lock(); defer c.mu.Unlock()
    c.entries[key] = &cacheEntry{value: val, expiresAt: time.Now().Add(ttl)}
}

func (c *TTLCache) Get(key string) (any, bool) {
    c.mu.RLock(); defer c.mu.RUnlock()
    entry, ok := c.entries[key]
    if !ok || time.Now().After(entry.expiresAt) { return nil, false }
    return entry.value, true
}

func (c *TTLCache) Delete(key string) {
    c.mu.Lock(); defer c.mu.Unlock()
    delete(c.entries, key)
}

func (c *TTLCache) cleanup(interval time.Duration) {
    ticker := time.NewTicker(interval)
    defer ticker.Stop()
    for {
        select {
        case <-ticker.C:
            c.mu.Lock()
            now := time.Now()
            for k, e := range c.entries {
                if now.After(e.expiresAt) { delete(c.entries, k) }
            }
            c.mu.Unlock()
        case <-c.stop:
            return
        }
    }
```

```go
}

func (c *TTLCache) Close() { close(c.stop) }
```

Interview tip: "Lazy expiration (check on Get) + background cleanup is the standard approach. Lazy alone leads to memory bloat. Background alone adds CPU overhead."

Q49

Design a multi-level cache (L1 in-memory, L2 Redis).

Difficulty: Hard

Check L1 first (fast, small). On miss, check L2 (slower, large). On miss, fetch from source.

```go
type Cache interface {
    Get(ctx context.Context, key string) ([]byte, error)
    Set(ctx context.Context, key string, val []byte, ttl time.Duration) error
}

type MultiLevelCache struct {
    l1  Cache         // in-memory, small, fast
    l2  Cache         // Redis, large, medium
    ttl time.Duration
}

func (m *MultiLevelCache) Get(ctx context.Context, key string) ([]byte, error) {
    // Check L1
    if val, err := m.l1.Get(ctx, key); err == nil {
        return val, nil
    }

    // L1 miss — check L2
    val, err := m.l2.Get(ctx, key)
    if err == nil {
        // Backfill L1 with shorter TTL
        _ = m.l1.Set(ctx, key, val, m.ttl/10)
        return val, nil
    }

    return nil, ErrCacheMiss
}

func (m *MultiLevelCache) Set(ctx context.Context, key string, val []byte, ttl time.Duration) error {
    // Write to both levels
    if err := m.l1.Set(ctx, key, val, ttl/10); err != nil {
        return err
    }
    return m.l2.Set(ctx, key, val, ttl)
}

// Invalidation — delete from both levels
func (m *MultiLevelCache) Delete(ctx context.Context, key string) error {
    // Delete from both — ignore L1 errors (best effort)
    _ = m.l1.Delete(ctx, key)
    return m.l2.Delete(ctx, key)
}
```

Interview tip: "L1 TTL should be shorter than L2 TTL. Otherwise L1 may serve stale data after L2 is updated. L1 TTL = L2 TTL / 10 is a common heuristic."

Q50

How do you handle cache stampede (thundering herd) on cache miss?

Difficulty: Hard

When cache expires, many requests hit the DB simultaneously. Solutions: probabilistic early expiration, single-flight, or mutex-based coalescing.

```go
import "golang.org/x/sync/singleflight"

type SmartCache struct {
    cache *TTLCache
    sf    singleflight.Group
    db    Database
}

// Single-flight: only ONE goroutine fetches; others wait and share result
func (c *SmartCache) Get(ctx context.Context, key string) (any, error) {
    if val, ok := c.cache.Get(key); ok { return val, nil }

    // Deduplicate concurrent cache misses for the same key
    val, err, _ := c.sf.Do(key, func() (any, error) {
        // Check cache again — might have been filled while waiting
        if val, ok := c.cache.Get(key); ok { return val, nil }

        // Fetch from DB (only one goroutine does this)
        result, err := c.db.Query(ctx, key)
        if err != nil { return nil, err }

        c.cache.Set(key, result, 5*time.Minute)
        return result, nil
    })

    return val, err
}

// Probabilistic early expiration (PER)
// Refresh cache before it expires to prevent simultaneous misses
func (c *SmartCache) GetWithPER(key string, beta float64) (any, bool) {
    entry, ok := c.cache.GetWithExpiry(key)
    if !ok { return nil, false }

    // Probabilistically refresh before expiry
    remaining := time.Until(entry.ExpiresAt)
    if remaining < 0 { return nil, false }

    delta := -math.Log(rand.Float64()) * beta
    if time.Duration(delta)*time.Second > remaining {
        go c.refresh(key) // refresh in background
    }
    return entry.Value, true
}
```

Interview tip: "singleflight is the most practical solution. It's in golang.org/x/sync and used by many production systems. Mention it immediately."

Q51

Implement a write-through vs write-back cache strategy.

Difficulty: Medium

Write-through: write to cache and DB synchronously — strong consistency, higher write latency. Write-back: write to cache only, flush to DB asynchronously — fast writes, risk of data loss.

```go
// Write-through
type WriteThroughCache struct {
    cache *TTLCache
    db    Database
}

func (c *WriteThroughCache) Set(ctx context.Context, key string, val any) error {
    // Write to DB first (source of truth)
    if err := c.db.Save(ctx, key, val); err != nil { return err }
    // Then update cache
    c.cache.Set(key, val, 5*time.Minute)
    return nil
}

// Write-back (write-behind)
type WriteBackCache struct {
    cache    *TTLCache
    db       Database
    dirty    map[string]any
    dirtyMu  sync.Mutex
    flushCh  chan string
}

func (c *WriteBackCache) Set(ctx context.Context, key string, val any) error {
    // Write to cache immediately (fast)
    c.cache.Set(key, val, 5*time.Minute)

    // Mark dirty — will be flushed to DB asynchronously
    c.dirtyMu.Lock()
    c.dirty[key] = val
    c.dirtyMu.Unlock()

    c.flushCh <- key
    return nil
}

func (c *WriteBackCache) flushWorker(ctx context.Context) {
    ticker := time.NewTicker(500 * time.Millisecond)
    defer ticker.Stop()
    for {
        select {
        case <-ticker.C:
            c.dirtyMu.Lock()
            for k, v := range c.dirty {
                _ = c.db.Save(ctx, k, v)
                delete(c.dirty, k)
            }
            c.dirtyMu.Unlock()
        case <-ctx.Done():
            return
        }
    }
}
```

Interview tip: "Write-back is risky — if the cache crashes before flushing, data is lost. Always have a Write-Ahead Log (WAL) or use it only for non-critical data."

Q52

Design a distributed cache invalidation strategy.

Difficulty: Hard

When data changes in DB, invalidate caches across multiple nodes. Strategies: TTL-based, event-driven (pub/sub), version-based.

```go
// Event-driven invalidation via Redis pub/sub
type CacheInvalidator struct {
    localCache *TTLCache
    rdb        *redis.Client
    channel    string
}

func NewCacheInvalidator(cache *TTLCache, rdb *redis.Client) *CacheInvalidator {
    ci := &CacheInvalidator{
        localCache: cache,
        rdb:        rdb,
        channel:    "cache:invalidate",
    }
    go ci.listen()
    return ci
}

// Subscribe to invalidation events from other nodes
func (ci *CacheInvalidator) listen() {
    sub := ci.rdb.Subscribe(context.Background(), ci.channel)
    ch := sub.Channel()
    for msg := range ch {
        ci.localCache.Delete(msg.Payload) // invalidate local cache
    }
}

// Publish invalidation event to all nodes
func (ci *CacheInvalidator) Invalidate(ctx context.Context, key string) error {
    // Delete locally
    ci.localCache.Delete(key)
    // Broadcast to all other nodes
    return ci.rdb.Publish(ctx, ci.channel, key).Err()
}

// Version-based cache key (avoids invalidation entirely)
// key = "user:{id}:v{version}" — new version = new key
type VersionedCache struct {
    cache   *TTLCache
    versions map[string]int64
    mu      sync.RWMutex
}

func (c *VersionedCache) Key(entity string, id int) string {
    c.mu.RLock(); defer c.mu.RUnlock()
    ver := c.versions[fmt.Sprintf("%s:%d", entity, id)]
    return fmt.Sprintf("%s:%d:v%d", entity, id, ver)
}
```

Interview tip: "Cache invalidation is one of the two hard problems in CS (the other is naming things). In production: prefer short TTLs + event-driven invalidation over complex version schemes."

---

6. Rate Limiting

Q53

Implement a token bucket rate limiter.

Difficulty: Medium

Bucket holds N tokens. Refills at rate R/second. Each request consumes one token.

```go
type TokenBucket struct {
    rate     float64
    capacity float64
    tokens   float64
    lastTime time.Time
    mu       sync.Mutex
}

func NewTokenBucket(rate, capacity float64) *TokenBucket {
    return &TokenBucket{
        rate: rate, capacity: capacity,
        tokens: capacity, lastTime: time.Now(),
    }
}

func (tb *TokenBucket) Allow() bool {
    return tb.AllowN(1)
}

func (tb *TokenBucket) AllowN(n float64) bool {
    tb.mu.Lock(); defer tb.mu.Unlock()

    now := time.Now()
    elapsed := now.Sub(tb.lastTime).Seconds()
    tb.lastTime = now

    tb.tokens = math.Min(tb.capacity, tb.tokens+elapsed*tb.rate)

    if tb.tokens >= n {
        tb.tokens -= n
        return true
    }
    return false
}

func (tb *TokenBucket) Wait(ctx context.Context) error {
    for {
        if tb.Allow() { return nil }
        select {
        case <-ctx.Done():
            return ctx.Err()
        case <-time.After(time.Duration(1000/tb.rate) * time.Millisecond):
        }
    }
}
```

Interview tip: "Token bucket allows bursting up to capacity, then enforces the average rate. Good for APIs where occasional bursts are acceptable."

Q54

Implement a leaky bucket rate limiter.

Difficulty: Medium

Requests enter a bucket (queue). Bucket leaks at a constant rate. If bucket overflows, request is rejected.

```go
type LeakyBucket struct {
    rate     float64       // requests per second
    capacity int           // max queue size
    queue    chan struct{}
    ctx      context.Context
    cancel   context.CancelFunc
}

func NewLeakyBucket(rate float64, capacity int) *LeakyBucket {
    ctx, cancel := context.WithCancel(context.Background())
    lb := &LeakyBucket{
        rate:     rate,
        capacity: capacity,
        queue:    make(chan struct{}, capacity),
        ctx:      ctx,
        cancel:   cancel,
    }
    go lb.drain()
    return lb
}

func (lb *LeakyBucket) Allow() bool {
    select {
    case lb.queue <- struct{}{}: // entered queue
        return true
    default: // queue full — reject
        return false
    }
}

func (lb *LeakyBucket) drain() {
    ticker := time.NewTicker(time.Duration(float64(time.Second) / lb.rate))
    defer ticker.Stop()
    for {
        select {
        case <-ticker.C:
            select {
            case <-lb.queue: // process one request
            default: // empty queue
            }
        case <-lb.ctx.Done():
            return
        }
    }
}

func (lb *LeakyBucket) Close() { lb.cancel() }
```

Interview tip: "Leaky bucket smooths traffic — output is always constant rate regardless of burst. Token bucket allows bursts. Leaky = strict rate. Token = bursty-ok."

Q55

Implement a fixed window counter rate limiter.

Difficulty: Easy

Count requests in fixed time windows. Simple but has boundary issues.

```go
type FixedWindowLimiter struct {
    limit      int
    windowSize time.Duration
    mu         sync.Mutex
    windows    map[string]*windowEntry
}

type windowEntry struct {
    count     int
    windowEnd time.Time
}

func NewFixedWindowLimiter(limit int, windowSize time.Duration) *FixedWindowLimiter {
    return &FixedWindowLimiter{
        limit:      limit,
        windowSize: windowSize,
        windows:    make(map[string]*windowEntry),
    }
}

func (f *FixedWindowLimiter) Allow(key string) bool {
    f.mu.Lock(); defer f.mu.Unlock()

    now := time.Now()
    entry, ok := f.windows[key]

    if !ok || now.After(entry.windowEnd) {
        // New window
        f.windows[key] = &windowEntry{
            count:     1,
            windowEnd: now.Add(f.windowSize),
        }
        return true
    }

    if entry.count >= f.limit { return false }
    entry.count++
    return true
}

// Problem: boundary attack
// Limit: 100/min
// At 00:59, send 100 requests → allowed
// At 01:00 (new window), send 100 requests → allowed
// Effective: 200 requests in 2 seconds!
```

Interview tip: "Always mention the boundary problem when discussing fixed window. The interviewer will ask about it. Sliding window log or sliding window counter fixes it."

Q56

Implement a sliding window log rate limiter.

Difficulty: Medium

Keep a log of timestamps. On each request, prune old timestamps and count remaining.

```go
type SlidingWindowLog struct {
    limit    int
    window   time.Duration
    mu       sync.Mutex
    requests map[string][]time.Time
}

func NewSlidingWindowLog(limit int, window time.Duration) *SlidingWindowLog {
    return &SlidingWindowLog{
        limit:    limit,
        window:   window,
        requests: make(map[string][]time.Time),
    }
}

func (s *SlidingWindowLog) Allow(key string) bool {
    s.mu.Lock(); defer s.mu.Unlock()

    now := time.Now()
    cutoff := now.Add(-s.window)

    // Prune expired timestamps
    times := s.requests[key]
    valid := times[:0]
    for _, t := range times {
        if t.After(cutoff) { valid = append(valid, t) }
    }

    if len(valid) >= s.limit {
        s.requests[key] = valid
        return false
    }

    s.requests[key] = append(valid, now)
    return true
}
```

Interview tip: "Sliding window log is accurate but O(requests) memory per user. At 1M users × 1000 requests/min = 1B timestamps in memory. In production: use Redis sorted sets."

Q57

Implement a per-user rate limiter with cleanup.

Difficulty: Hard

Manage a rate limiter per user with background cleanup of idle users.

```go
type PerUserLimiter struct {
    mu       sync.Mutex
    limiters map[string]*userLimiter
    rate     float64
    capacity float64
}

type userLimiter struct {
    bucket   *TokenBucket
    lastSeen time.Time
}

func NewPerUserLimiter(rate, capacity float64) *PerUserLimiter {
    pul := &PerUserLimiter{
        limiters: make(map[string]*userLimiter),
        rate:     rate,
        capacity: capacity,
    }
    go pul.cleanup()
    return pul
}

func (p *PerUserLimiter) Allow(userID string) bool {
    p.mu.Lock()
    ul, ok := p.limiters[userID]
    if !ok {
        ul = &userLimiter{bucket: NewTokenBucket(p.rate, p.capacity)}
        p.limiters[userID] = ul
    }
    ul.lastSeen = time.Now()
    bucket := ul.bucket
    p.mu.Unlock()

    return bucket.Allow()
}

func (p *PerUserLimiter) cleanup() {
    ticker := time.NewTicker(5 * time.Minute)
    defer ticker.Stop()
    for range ticker.C {
        p.mu.Lock()
        cutoff := time.Now().Add(-10 * time.Minute)
        for id, ul := range p.limiters {
            if ul.lastSeen.Before(cutoff) {
                delete(p.limiters, id)
            }
        }
        p.mu.Unlock()
    }
}
```

Interview tip: "Without cleanup, limiters map grows unbounded for each unique user. Background cleanup prevents memory leak for short-lived sessions or bot IPs."

Q58

Design a distributed rate limiter using Redis.

Difficulty: Hard

Use Redis Lua scripts for atomic sliding window operations across multiple app servers.

```go
const luaScript = `
local key = KEYS[1]
local window = tonumber(ARGV[1])
local limit = tonumber(ARGV[2])
local now = tonumber(ARGV[3])

redis.call('ZREMRANGEBYSCORE', key, 0, now - window * 1000)
local count = redis.call('ZCARD', key)

if count >= limit then
    return 0
end

redis.call('ZADD', key, now, now)
redis.call('PEXPIRE', key, window * 1000)
return 1
`

type DistributedLimiter struct {
    rdb    *redis.Client
    script *redis.Script
    window time.Duration
    limit  int
}

func NewDistributedLimiter(rdb *redis.Client, window time.Duration, limit int) *DistributedLimiter {
    return &DistributedLimiter{
        rdb:    rdb,
        script: redis.NewScript(luaScript),
        window: window,
        limit:  limit,
    }
}

func (d *DistributedLimiter) Allow(ctx context.Context, key string) (bool, error) {
    now := time.Now().UnixMilli()
    result, err := d.script.Run(ctx, d.rdb,
        []string{"ratelimit:" + key},
        int(d.window.Seconds()),
        d.limit,
        now,
    ).Int()
    if err != nil { return false, err }
    return result == 1, nil
}

// With local fallback on Redis failure
func (d *DistributedLimiter) AllowWithFallback(ctx context.Context, key string, local *TokenBucket) bool {
    allowed, err := d.Allow(ctx, key)
    if err != nil {
        // Redis down — fall back to local limiter (fail open)
        return local.Allow()
    }
```

```go
    return allowed
}
```

Interview tip: "Lua scripts are atomic in Redis — no race conditions. ZRANGEBYSCORE+ZADD as separate commands would have a race window. Always use Lua for atomic rate limit operations."

---

7. Concurrency Designs

Q59

Design a concurrent pipeline with backpressure.

Difficulty: Hard

Upstream slows down when downstream is overwhelmed. Use bounded channels.

```go
type Stage func(in <-chan []byte) <-chan []byte

func NewPipeline(input <-chan []byte, stages ...Stage) <-chan []byte {
    out := input
    for _, stage := range stages { out = stage(out) }
    return out
}

// Each stage uses bounded channel — backpressure propagates upstream
func ParseStage(bufSize int) Stage {
    return func(in <-chan []byte) <-chan []byte {
        out := make(chan []byte, bufSize) // bounded = backpressure
        go func() {
            defer close(out)
            for data := range in {
                parsed := parseJSON(data)
                out <- parsed // blocks if out is full — slows upstream
            }
        }()
        return out
    }
}

func ValidateStage(bufSize int) Stage {
    return func(in <-chan []byte) <-chan []byte {
        out := make(chan []byte, bufSize)
        go func() {
            defer close(out)
            for data := range in {
                if validate(data) { out <- data }
                // invalid data dropped — no backpressure for dropped items
            }
        }()
        return out
    }
}

func TransformStage(bufSize int) Stage {
    return func(in <-chan []byte) <-chan []byte {
        out := make(chan []byte, bufSize)
        go func() {
            defer close(out)
            for data := range in { out <- transform(data) }
        }()
        return out
    }
}

// Build and run
rawData := make(chan []byte, 10)
result := NewPipeline(rawData,
    ParseStage(5),
    ValidateStage(5),
```

```go
    TransformStage(5),
)
```

Interview tip: "Backpressure is critical for production pipelines. Without bounded channels, a slow consumer causes unbounded memory growth. The bound is the safety valve."

Q60

Implement a concurrent map with sharding.

Difficulty: Hard

Reduce lock contention by splitting map into N shards, each with its own mutex.

```go
const shardCount = 32

type ShardedMap struct {
    shards [shardCount]mapShard
}

type mapShard struct {
    items map[string]any
    mu    sync.RWMutex
}

func NewShardedMap() *ShardedMap {
    sm := &ShardedMap{}
    for i := range sm.shards {
        sm.shards[i].items = make(map[string]any)
    }
    return sm
}

func (sm *ShardedMap) shard(key string) *mapShard {
    h := fnv.New32a()
    h.Write([]byte(key))
    return &sm.shards[h.Sum32()%shardCount]
}

func (sm *ShardedMap) Set(key string, val any) {
    shard := sm.shard(key)
    shard.mu.Lock(); defer shard.mu.Unlock()
    shard.items[key] = val
}

func (sm *ShardedMap) Get(key string) (any, bool) {
    shard := sm.shard(key)
    shard.mu.RLock(); defer shard.mu.RUnlock()
    val, ok := shard.items[key]
    return val, ok
}

func (sm *ShardedMap) Delete(key string) {
    shard := sm.shard(key)
    shard.mu.Lock(); defer shard.mu.Unlock()
    delete(shard.items, key)
}

func (sm *ShardedMap) Count() int {
    total := 0
    for i := range sm.shards {
        sm.shards[i].mu.RLock()
        total += len(sm.shards[i].items)
        sm.shards[i].mu.RUnlock()
    }
    return total
}
```

Interview tip: "sync.Map is good for read-heavy workloads with infrequent writes. Sharded map with RWMutex is better for write-heavy workloads. Sharding reduces contention by

1/N."

Q61

Implement a bounded goroutine pool.

Difficulty: Medium

Fixed number of goroutines, submit tasks to shared queue.

```go
type Task func() error

type Pool struct {
    tasks   chan Task
    results chan error
    wg      sync.WaitGroup
    once    sync.Once
    stop    chan struct{}
}

func NewPool(workers, queueSize int) *Pool {
    p := &Pool{
        tasks:   make(chan Task, queueSize),
        results: make(chan error, queueSize),
        stop:    make(chan struct{}),
    }

    for i := 0; i < workers; i++ {
        p.wg.Add(1)
        go func() {
            defer p.wg.Done()
            for {
                select {
                case task, ok := <-p.tasks:
                    if !ok { return }
                    p.results <- task()
                case <-p.stop:
                    return
                }
            }
        }()
    }
    return p
}

func (p *Pool) Submit(task Task) error {
    select {
    case p.tasks <- task:
        return nil
    case <-p.stop:
        return errors.New("pool stopped")
    }
}

func (p *Pool) SubmitTimeout(task Task, timeout time.Duration) error {
    select {
    case p.tasks <- task:
        return nil
    case <-time.After(timeout):
        return errors.New("queue full — submission timeout")
    case <-p.stop:
        return errors.New("pool stopped")
    }
```

```go
}

func (p *Pool) Stop() {
    p.once.Do(func() {
        close(p.stop)
        p.wg.Wait()
        close(p.results)
    })
}
```

Interview tip: "The queue size is as important as worker count. Too small = producers block. Too large = memory spike under load. Size to your P99 burst duration."

Q62

Design a concurrent job scheduler.

Difficulty: Hard

Schedule jobs with priorities, delays, and cron-like recurrence.

```go
type JobPriority int
const (High JobPriority = iota; Medium; Low)

type Job struct {
    ID        string
    Priority  JobPriority
    RunAt     time.Time
    Fn        func() error
    Recurring time.Duration // 0 = one-shot
}

type JobScheduler struct {
    mu      sync.Mutex
    heap    *jobHeap
    cond    *sync.Cond
    workers int
    stop    chan struct{}
}

func NewJobScheduler(workers int) *JobScheduler {
    h := &jobHeap{}
    heap.Init(h)
    js := &JobScheduler{heap: h, workers: workers, stop: make(chan struct{})}
    js.cond = sync.NewCond(&js.mu)
    for i := 0; i < workers; i++ { go js.worker() }
    return js
}

func (js *JobScheduler) Schedule(job *Job) {
    js.mu.Lock()
    heap.Push(js.heap, job)
    js.cond.Signal()
    js.mu.Unlock()
}

func (js *JobScheduler) worker() {
    for {
        js.mu.Lock()
        for js.heap.Len() == 0 {
            js.cond.Wait()
        }
        job := (*js.heap)[0]
        now := time.Now()
        if job.RunAt.After(now) {
            js.mu.Unlock()
            time.Sleep(job.RunAt.Sub(now))
            continue
        }
        heap.Pop(js.heap)
        js.mu.Unlock()

        go func(j *Job) {
            if err := j.Fn(); err != nil {
```

```go
                log.Printf("job %s failed: %v", j.ID, err)
            }
            // Re-schedule recurring jobs
            if j.Recurring > 0 {
                j.RunAt = time.Now().Add(j.Recurring)
                js.Schedule(j)
            }
        }(job)
    }
}
```

Interview tip: "Real schedulers (cron, Kubernetes CronJob) store jobs in DB for durability. In-memory scheduler loses jobs on restart. Discuss persistence as a follow-up."

Q63

Implement a publish-subscribe system in Go.

Difficulty: Medium

Topics, multiple subscribers, async delivery.

```go
type Message struct {
    Topic   string
    Payload any
}

type Subscriber struct {
    id   string
    ch   chan Message
    once sync.Once
}

func (s *Subscriber) Receive() <-chan Message { return s.ch }
func (s *Subscriber) Close() { s.once.Do(func() { close(s.ch) }) }

type PubSub struct {
    mu          sync.RWMutex
    subscribers map[string][]*Subscriber
}

func NewPubSub() *PubSub {
    return &PubSub{subscribers: make(map[string][]*Subscriber)}
}

func (ps *PubSub) Subscribe(topic string, bufSize int) *Subscriber {
    ps.mu.Lock(); defer ps.mu.Unlock()
    sub := &Subscriber{
        id: uuid.New().String(),
        ch: make(chan Message, bufSize),
    }
    ps.subscribers[topic] = append(ps.subscribers[topic], sub)
    return sub
}

func (ps *PubSub) Unsubscribe(topic string, sub *Subscriber) {
    ps.mu.Lock(); defer ps.mu.Unlock()
    subs := ps.subscribers[topic]
    for i, s := range subs {
        if s.id == sub.id {
            ps.subscribers[topic] = append(subs[:i], subs[i+1:]...)
            sub.Close()
            return
        }
    }
}

func (ps *PubSub) Publish(msg Message) int {
    ps.mu.RLock()
    subs := make([]*Subscriber, len(ps.subscribers[msg.Topic]))
    copy(subs, ps.subscribers[msg.Topic])
    ps.mu.RUnlock()

    delivered := 0
    for _, sub := range subs {
```

```go
        select {
        case sub.ch <- msg:
            delivered++
        default:
            // subscriber too slow — drop message (or handle differently)
        }
    }
    return delivered
}
```

Interview tip: "Slow subscriber handling: drop (default), block (no default), or use a dead letter queue. Discuss the trade-offs — blocking publish can cascade back to the publisher."

Q64

Implement a circuit breaker pattern.

Difficulty: Hard

Wraps external calls. Fails fast when service is down.

```go
type CircuitState int
const (StateClosed CircuitState = iota; StateOpen; StateHalfOpen)

type CircuitBreaker struct {
    mu            sync.Mutex
    state         CircuitState
    failures      int
    successes     int
    maxFailures   int
    successNeeded int
    timeout       time.Duration
    lastFailureAt time.Time
    onStateChange func(from, to CircuitState)
}

func NewCircuitBreaker(maxFailures, successNeeded int, timeout time.Duration) *CircuitBreaker {
    return &CircuitBreaker{
        maxFailures:   maxFailures,
        successNeeded: successNeeded,
        timeout:       timeout,
    }
}

func (cb *CircuitBreaker) Execute(fn func() error) error {
    cb.mu.Lock()
    state := cb.state
    if state == StateOpen {
        if time.Since(cb.lastFailureAt) > cb.timeout {
            cb.transition(StateHalfOpen)
        } else {
            cb.mu.Unlock()
            return errors.New("circuit open — fast fail")
        }
    }
    cb.mu.Unlock()

    err := fn()

    cb.mu.Lock()
    defer cb.mu.Unlock()

    if err != nil {
        cb.failures++
        cb.successes = 0
        cb.lastFailureAt = time.Now()
        if cb.state == StateHalfOpen || cb.failures >= cb.maxFailures {
            cb.transition(StateOpen)
        }
        return err
    }

    cb.successes++
    if cb.state == StateHalfOpen && cb.successes >= cb.successNeeded {
```

```go
        cb.failures = 0
        cb.transition(StateClosed)
    }
    return nil
}

func (cb *CircuitBreaker) transition(to CircuitState) {
    from := cb.state
    cb.state = to
    if cb.onStateChange != nil { go cb.onStateChange(from, to) }
}
```

Interview tip: "Half-open state is critical — it probes service health with limited traffic before fully re-opening. Without it, recovery is either too slow (manual) or too risky (all-at-once)."

Q65

Implement a retry mechanism with jitter.

Difficulty: Medium

Retry failed operations with exponential backoff + random jitter to prevent thundering herd.

```go
type RetryConfig struct {
    MaxAttempts int
    BaseDelay   time.Duration
    MaxDelay    time.Duration
    Multiplier  float64
    Jitter      float64
}

var DefaultRetry = RetryConfig{
    MaxAttempts: 5,
    BaseDelay:   100 * time.Millisecond,
    MaxDelay:    30 * time.Second,
    Multiplier:  2.0,
    Jitter:      0.1,
}

type RetryableError struct{ err error }
func (e RetryableError) Error() string { return e.err.Error() }
func IsRetryable(err error) bool {
    var re RetryableError
    return errors.As(err, &re)
}

func Retry(ctx context.Context, cfg RetryConfig, fn func() error) error {
    var lastErr error
    delay := cfg.BaseDelay

    for attempt := 0; attempt < cfg.MaxAttempts; attempt++ {
        if attempt > 0 {
            // Exponential backoff with jitter
            jitter := time.Duration(float64(delay) * cfg.Jitter * (2*rand.Float64() - 1))
            wait := delay + jitter
            if wait > cfg.MaxDelay { wait = cfg.MaxDelay }

            select {
            case <-ctx.Done():
                return ctx.Err()
            case <-time.After(wait):
            }

            delay = time.Duration(float64(delay) * cfg.Multiplier)
        }

        lastErr = fn()
        if lastErr == nil { return nil }
        if !IsRetryable(lastErr) { return lastErr } // don't retry non-retryable errors
    }
    return fmt.Errorf("after %d attempts: %w", cfg.MaxAttempts, lastErr)
}
```

Interview tip: "Jitter types: full jitter (0 to delay), equal jitter (delay/2 to delay), decorrelated jitter. AWS recommends 'full jitter' — most effective at spreading load."

Q66

Implement a graceful shutdown coordinator.

Difficulty: Hard

Coordinate shutdown of multiple components in the right order.

```go
type Component interface {
    Name() string
    Start(ctx context.Context) error
    Stop(ctx context.Context) error
}

type ShutdownCoordinator struct {
    components []Component
    started    []Component
    mu         sync.Mutex
}

func (sc *ShutdownCoordinator) Register(c Component) {
    sc.mu.Lock(); defer sc.mu.Unlock()
    sc.components = append(sc.components, c)
}

func (sc *ShutdownCoordinator) StartAll(ctx context.Context) error {
    for _, c := range sc.components {
        log.Printf("starting %s", c.Name())
        if err := c.Start(ctx); err != nil {
            // Roll back already-started components
            sc.StopAll(context.Background())
            return fmt.Errorf("failed to start %s: %w", c.Name(), err)
        }
        sc.mu.Lock()
        sc.started = append(sc.started, c)
        sc.mu.Unlock()
    }
    return nil
}

func (sc *ShutdownCoordinator) StopAll(ctx context.Context) {
    sc.mu.Lock()
    started := make([]Component, len(sc.started))
    copy(started, sc.started)
    sc.mu.Unlock()

    // Stop in reverse order (LIFO)
    for i := len(started) - 1; i >= 0; i-- {
        c := started[i]
        log.Printf("stopping %s", c.Name())
        if err := c.Stop(ctx); err != nil {
            log.Printf("error stopping %s: %v", c.Name(), err)
        }
    }
}

func (sc *ShutdownCoordinator) WaitForSignal(ctx context.Context) {
    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    select {
    case sig := <-quit:
```

```go
        log.Printf("received signal: %s", sig)
    case <-ctx.Done():
    }
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()
    sc.StopAll(shutdownCtx)
}
```

Interview tip: "Stop in reverse-start order — just like defer. Dependencies started first should be stopped last. E.g., HTTP server stops first (stop accepting), then DB connection pool."

---

8. Mini System Designs

Q67

Design a parking lot system.

Difficulty: Medium

```go
type VehicleType int
const (Motorcycle VehicleType = iota; Car; Truck)

type Vehicle interface {
    Type() VehicleType
    Plate() string
}

type Spot struct {
    ID       int
    Level    int
    Row      int
    SpotType VehicleType
    Vehicle  Vehicle
}

func (s *Spot) IsAvailable() bool    { return s.Vehicle == nil }
func (s *Spot) CanFit(v Vehicle) bool {
    return s.IsAvailable() && s.SpotType >= v.Type()
}
func (s *Spot) Park(v Vehicle)       { s.Vehicle = v }
func (s *Spot) Vacate() Vehicle      { v := s.Vehicle; s.Vehicle = nil; return v }

type Level struct {
    number int
    spots  []*Spot
}

func (l *Level) Park(v Vehicle) *Spot {
    for _, s := range l.spots {
        if s.CanFit(v) { s.Park(v); return s }
    }
    return nil
}

func (l *Level) Available() int {
    count := 0
    for _, s := range l.spots { if s.IsAvailable() { count++ } }
    return count
}

type ParkingLot struct {
    levels  []*Level
    tickets map[string]*Spot
    mu      sync.Mutex
}

func (pl *ParkingLot) Park(v Vehicle) (*Spot, error) {
    pl.mu.Lock(); defer pl.mu.Unlock()
    for _, level := range pl.levels {
        if spot := level.Park(v); spot != nil {
            pl.tickets[v.Plate()] = spot
            return spot, nil
```

```go
        }
    }
    return nil, errors.New("no available spots")
}

func (pl *ParkingLot) Unpark(plate string) (Vehicle, error) {
    pl.mu.Lock(); defer pl.mu.Unlock()
    spot, ok := pl.tickets[plate]
    if !ok { return nil, errors.New("ticket not found") }
    v := spot.Vacate()
    delete(pl.tickets, plate)
    return v, nil
}

func (pl *ParkingLot) AvailableSpots() map[VehicleType]int {
    pl.mu.Lock(); defer pl.mu.Unlock()
    counts := map[VehicleType]int{}
    for _, level := range pl.levels {
        for _, spot := range level.spots {
            if spot.IsAvailable() { counts[spot.SpotType]++ }
        }
    }
    return counts
}
```

Interview tip: "Extend this: add fee calculation (rate per hour), ticketing with entry time, handicap spots, electric charging spots. Each extension tests a new OOP concept."

Q68

Design an elevator system.

Difficulty: Hard

```go
type Direction int
const (DirIdle Direction = iota; DirUp; DirDown)

type ElevatorState int
const (StateIdle ElevatorState = iota; StateMoving; StateDoorOpen)

type Elevator struct {
    id       int
    floor    int
    dir      Direction
    state    ElevatorState
    stops    map[int]bool
    mu       sync.Mutex
}

func (e *Elevator) AddStop(floor int) {
    e.mu.Lock(); defer e.mu.Unlock()
    e.stops[floor] = true
    if e.dir == DirIdle {
        if floor > e.floor { e.dir = DirUp } else { e.dir = DirDown }
    }
}

func (e *Elevator) NextFloor() int {
    e.mu.Lock(); defer e.mu.Unlock()
    if e.dir == DirUp {
        for f := e.floor + 1; f <= 100; f++ {
            if e.stops[f] { return f }
        }
        e.dir = DirDown
    }
    for f := e.floor - 1; f >= 0; f-- {
        if e.stops[f] { return f }
    }
    e.dir = DirIdle
    return e.floor
}

func (e *Elevator) MoveTo(floor int) {
    e.mu.Lock(); defer e.mu.Unlock()
    e.floor = floor
    delete(e.stops, floor)
    e.state = StateDoorOpen
}

type ElevatorController struct {
    elevators []*Elevator
}

func (ec *ElevatorController) RequestElevator(fromFloor int, dir Direction) *Elevator {
    // Find best elevator (nearest idle or going in same direction)
    var best *Elevator
    bestScore := math.MaxInt64
```

```go
    for _, e := range ec.elevators {
        score := ec.score(e, fromFloor, dir)
        if score < bestScore { bestScore = score; best = e }
    }
    if best != nil { best.AddStop(fromFloor) }
    return best
}

func (ec *ElevatorController) score(e *Elevator, floor int, dir Direction) int {
    e.mu.Lock(); defer e.mu.Unlock()
    dist := abs(e.floor - floor)
    if e.dir == DirIdle { return dist }
    if e.dir == dir && ((dir == DirUp && e.floor <= floor) || (dir == DirDown && e.floor >= floor)) {
        return dist // same direction — good
    }
    return dist + 100 // penalise wrong direction
}
```

Interview tip: "The scoring function is the heart of the elevator algorithm. Discuss trade-offs: FCFS (simple, starvation risk), SCAN (efficient, may skip floors), LOOK (scan without going to end floors)."

Q69

Design a library management system.

Difficulty: Medium

```go
type Book struct {
    ISBN      string
    Title     string
    Author    string
    TotalCopy int
    Available int
}

type Member struct {
    ID       string
    Name     string
    Email    string
    BorrowedBooks []string
    MaxBooks int
}

type Loan struct {
    ID         string
    BookISBN   string
    MemberID   string
    BorrowedAt time.Time
    DueDate    time.Time
    ReturnedAt *time.Time
}

type Library struct {
    mu      sync.RWMutex
    books   map[string]*Book
    members map[string]*Member
    loans   map[string]*Loan
}

func (l *Library) AddBook(book *Book) {
    l.mu.Lock(); defer l.mu.Unlock()
    l.books[book.ISBN] = book
}

func (l *Library) Borrow(memberID, isbn string) (*Loan, error) {
    l.mu.Lock(); defer l.mu.Unlock()

    member, ok := l.members[memberID]
    if !ok { return nil, errors.New("member not found") }

    book, ok := l.books[isbn]
    if !ok { return nil, errors.New("book not found") }

    if len(member.BorrowedBooks) >= member.MaxBooks {
        return nil, errors.New("borrow limit reached")
    }
    if book.Available == 0 {
        return nil, errors.New("no copies available")
    }
```

```go
    loan := &Loan{
        ID:         uuid.New().String(),
        BookISBN:   isbn,
        MemberID:   memberID,
        BorrowedAt: time.Now(),
        DueDate:    time.Now().Add(14 * 24 * time.Hour),
    }

    book.Available--
    member.BorrowedBooks = append(member.BorrowedBooks, isbn)
    l.loans[loan.ID] = loan
    return loan, nil
}

func (l *Library) Return(loanID string) error {
    l.mu.Lock(); defer l.mu.Unlock()

    loan, ok := l.loans[loanID]
    if !ok { return errors.New("loan not found") }
    if loan.ReturnedAt != nil { return errors.New("already returned") }

    now := time.Now()
    loan.ReturnedAt = &now
    l.books[loan.BookISBN].Available++

    // Remove from member's borrowed list
    member := l.members[loan.MemberID]
    for i, isbn := range member.BorrowedBooks {
        if isbn == loan.BookISBN {
            member.BorrowedBooks = append(member.BorrowedBooks[:i], member.BorrowedBooks[i+1:]...)
            break
        }
    }

    // Calculate late fee
    if now.After(loan.DueDate) {
        days := int(now.Sub(loan.DueDate).Hours() / 24)
        fmt.Printf("Late fee: $%.2f\n", float64(days)*0.25)
    }
    return nil
}
```

Interview tip: "Extend: reservation queue (when a book is unavailable), notification on return, search by genre/author. Each adds a new design challenge."

Q70

Design a URL shortener (LLD level).

Difficulty: Medium

```go
const base62Chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

func toBase62(n int64) string {
    if n == 0 { return "0" }
    var b []byte
    for n > 0 { b = append([]byte{base62Chars[n%62]}, b...); n /= 62 }
    return string(b)
}

type URLRecord struct {
    ShortCode string
    LongURL   string
    UserID    string
    CreatedAt time.Time
    ExpiresAt *time.Time
    Clicks    int64
}

type URLShortener struct {
    mu      sync.RWMutex
    store   map[string]*URLRecord // shortCode → record
    reverse map[string]string     // longURL → shortCode
    counter int64
    baseURL string
}

func NewURLShortener(baseURL string) *URLShortener {
    return &URLShortener{
        store:   make(map[string]*URLRecord),
        reverse: make(map[string]string),
        baseURL: baseURL,
    }
}

func (u *URLShortener) Shorten(longURL, userID string, expiry *time.Time) string {
    u.mu.Lock(); defer u.mu.Unlock()

    if code, ok := u.reverse[longURL]; ok {
        return u.baseURL + "/" + code
    }

    id := atomic.AddInt64(&u.counter, 1)
    code := toBase62(id)

    u.store[code] = &URLRecord{
        ShortCode: code,
        LongURL:   longURL,
        UserID:    userID,
        CreatedAt: time.Now(),
        ExpiresAt: expiry,
    }
    u.reverse[longURL] = code
    return u.baseURL + "/" + code
```

```go
}

func (u *URLShortener) Expand(code string) (string, error) {
    u.mu.RLock(); defer u.mu.RUnlock()

    rec, ok := u.store[code]
    if !ok { return "", errors.New("short URL not found") }

    if rec.ExpiresAt != nil && time.Now().After(*rec.ExpiresAt) {
        return "", errors.New("short URL expired")
    }

    atomic.AddInt64(&rec.Clicks, 1)
    return rec.LongURL, nil
}

func (u *URLShortener) Analytics(code string) (*URLRecord, error) {
    u.mu.RLock(); defer u.mu.RUnlock()
    rec, ok := u.store[code]
    if !ok { return nil, errors.New("not found") }
    return rec, nil
}
```

Interview tip: "301 vs 302 redirect: 301 is permanent (browser caches, analytics miss it). 302 is temporary (every request hits your server — good for analytics). Discuss this trade-off."

Q71

Design a chess game.

Difficulty: Hard

```go
type Color int
const (White Color = iota; Black)

type PieceType int
const (King PieceType = iota; Queen; Rook; Bishop; Knight; Pawn)

type Position struct{ Row, Col int }

func (p Position) IsValid() bool {
    return p.Row >= 0 && p.Row < 8 && p.Col >= 0 && p.Col < 8
}

type Piece interface {
    Type() PieceType
    Color() Color
    ValidMoves(board *Board, from Position) []Position
}

type Board struct {
    squares [8][8]Piece
}

func (b *Board) Get(p Position) Piece     { return b.squares[p.Row][p.Col] }
func (b *Board) Set(p Position, pc Piece) { b.squares[p.Row][p.Col] = pc }

type Game struct {
    board       *Board
    turn        Color
    moveHistory []Move
    status      GameStatus
}

type Move struct {
    From, To Position
    Piece    Piece
    Captured Piece
}

func (g *Game) Move(from, to Position) error {
    piece := g.board.Get(from)
    if piece == nil { return errors.New("no piece at source") }
    if piece.Color() != g.turn { return errors.New("not your turn") }

    validMoves := piece.ValidMoves(g.board, from)
    valid := false
    for _, m := range validMoves {
        if m == to { valid = true; break }
    }
    if !valid { return errors.New("invalid move") }

    captured := g.board.Get(to)
    g.moveHistory = append(g.moveHistory, Move{from, to, piece, captured})
```

```go
    g.board.Set(to, piece)
    g.board.Set(from, nil)

    g.turn = 1 - g.turn // switch turns
    g.updateStatus()
    return nil
}

// Knight moves example
type KnightPiece struct {
    color PieceType
    c     Color
}
func (k *KnightPiece) Type() PieceType { return Knight }
func (k *KnightPiece) Color() Color    { return k.c }
func (k *KnightPiece) ValidMoves(board *Board, from Position) []Position {
    offsets := [][2]int{{2,1},{2,-1},{-2,1},{-2,-1},{1,2},{1,-2},{-1,2},{-1,-2}}
    var moves []Position
    for _, off := range offsets {
        pos := Position{from.Row + off[0], from.Col + off[1]}
        if !pos.IsValid() { continue }
        target := board.Get(pos)
        if target == nil || target.Color() != k.c { moves = append(moves, pos) }
    }
    return moves
}
```

Interview tip: "Focus on the board representation and piece hierarchy in the interview. Check/checkmate detection and castling/en-passant are follow-up complexity."

Q72

Design a hotel reservation system.

Difficulty: Hard

```go
type RoomType int
const (Single RoomType = iota; Double; Suite)

type Room struct {
    Number   int
    Type     RoomType
    Floor    int
    Price    float64
    Amenities []string
}

type Reservation struct {
    ID         string
    GuestID    string
    RoomNumber int
    CheckIn    time.Time
    CheckOut   time.Time
    TotalPrice float64
    Status     string
}

type Hotel struct {
    mu           sync.RWMutex
    rooms        map[int]*Room
    reservations map[string]*Reservation
    calendar     map[int]map[string]string // roomNum → date → reservationID
}

func NewHotel() *Hotel {
    return &Hotel{
        rooms:        make(map[int]*Room),
        reservations: make(map[string]*Reservation),
        calendar:     make(map[int]map[string]string),
    }
}

func (h *Hotel) SearchAvailable(checkIn, checkOut time.Time, roomType RoomType) []*Room {
    h.mu.RLock(); defer h.mu.RUnlock()

    var available []*Room
    for _, room := range h.rooms {
        if room.Type != roomType { continue }
        if h.isAvailable(room.Number, checkIn, checkOut) {
            available = append(available, room)
        }
    }
    return available
}

func (h *Hotel) isAvailable(roomNum int, checkIn, checkOut time.Time) bool {
    cal, ok := h.calendar[roomNum]
    if !ok { return true }
```

```go
    for d := checkIn; d.Before(checkOut); d = d.AddDate(0, 0, 1) {
        dateKey := d.Format("2006-01-02")
        if _, booked := cal[dateKey]; booked { return false }
    }
    return true
}

func (h *Hotel) Reserve(guestID string, roomNum int, checkIn, checkOut time.Time) (*Reservation, error) {
    h.mu.Lock(); defer h.mu.Unlock()

    if !h.isAvailable(roomNum, checkIn, checkOut) {
        return nil, errors.New("room not available for selected dates")
    }

    room := h.rooms[roomNum]
    nights := int(checkOut.Sub(checkIn).Hours() / 24)

    res := &Reservation{
        ID:         uuid.New().String(),
        GuestID:    guestID,
        RoomNumber: roomNum,
        CheckIn:    checkIn,
        CheckOut:   checkOut,
        TotalPrice: room.Price * float64(nights),
        Status:     "confirmed",
    }

    h.reservations[res.ID] = res
    if h.calendar[roomNum] == nil { h.calendar[roomNum] = make(map[string]string) }
    for d := checkIn; d.Before(checkOut); d = d.AddDate(0, 0, 1) {
        h.calendar[roomNum][d.Format("2006-01-02")] = res.ID
    }
    return res, nil
}

func (h *Hotel) Cancel(reservationID string) error {
    h.mu.Lock(); defer h.mu.Unlock()

    res, ok := h.reservations[reservationID]
    if !ok { return errors.New("reservation not found") }
    if res.Status == "cancelled" { return errors.New("already cancelled") }

    res.Status = "cancelled"
    for d := res.CheckIn; d.Before(res.CheckOut); d = d.AddDate(0, 0, 1) {
        delete(h.calendar[res.RoomNumber], d.Format("2006-01-02"))
    }
    return nil
}
```

Interview tip: "The calendar map (room → date → reservation) is the key data structure. It makes availability checks O(nights) instead of O(reservations). Interview focus: overbooking prevention with locks."

Q73

Design a vending machine.

Difficulty: Medium

```go
type VendingState int
const (Idle VendingState = iota; HasMoney; Dispensing; OutOfStock)

type Product struct {
    Code  string
    Name  string
    Price float64
    Stock int
}

type VendingMachine struct {
    mu           sync.Mutex
    state        VendingState
    products     map[string]*Product
    insertedAmt  float64
    selectedCode string
}

func NewVendingMachine(products map[string]*Product) *VendingMachine {
    return &VendingMachine{products: products, state: Idle}
}

func (vm *VendingMachine) InsertMoney(amount float64) error {
    vm.mu.Lock(); defer vm.mu.Unlock()
    if vm.state == Dispensing { return errors.New("please wait") }
    vm.insertedAmt += amount
    vm.state = HasMoney
    fmt.Printf("Inserted: $%.2f | Total: $%.2f\n", amount, vm.insertedAmt)
    return nil
}

func (vm *VendingMachine) SelectProduct(code string) error {
    vm.mu.Lock(); defer vm.mu.Unlock()
    if vm.state != HasMoney { return errors.New("insert money first") }

    product, ok := vm.products[code]
    if !ok { return errors.New("product not found") }
    if product.Stock == 0 { return errors.New("out of stock") }
    if vm.insertedAmt < product.Price {
        return fmt.Errorf("need $%.2f more", product.Price-vm.insertedAmt)
    }

    vm.selectedCode = code
    vm.state = Dispensing
    return nil
}

func (vm *VendingMachine) Dispense() (string, float64, error) {
    vm.mu.Lock(); defer vm.mu.Unlock()
    if vm.state != Dispensing { return "", 0, errors.New("select product first") }

    product := vm.products[vm.selectedCode]
    change := vm.insertedAmt - product.Price
```

```go
    product.Stock--

    vm.insertedAmt = 0
    vm.selectedCode = ""
    if vm.allOutOfStock() { vm.state = OutOfStock } else { vm.state = Idle }

    return product.Name, change, nil
}

func (vm *VendingMachine) Cancel() float64 {
    vm.mu.Lock(); defer vm.mu.Unlock()
    refund := vm.insertedAmt
    vm.insertedAmt = 0
    vm.state = Idle
    return refund
}

func (vm *VendingMachine) allOutOfStock() bool {
    for _, p := range vm.products { if p.Stock > 0 { return false } }
    return true
}
```

Interview tip: "State machine is the right pattern here. Draw the state diagram first: Idle→HasMoney→Dispensing→Idle. Invalid transitions return errors, never panic."

Q74

Design a food ordering system (like Swiggy/Zomato).

Difficulty: Hard

```go
type OrderStatus int
const (OrderPlaced OrderStatus = iota; OrderAccepted; Preparing; ReadyForPickup; OutForDelivery; Delivered

type MenuItem struct {
    ID       string
    Name     string
    Price    float64
    Category string
    InStock  bool
}

type Restaurant struct {
    ID        string
    Name      string
    Menu      map[string]*MenuItem
    IsOpen    bool
    Rating    float64
}

type OrderItem struct {
    MenuItemID string
    Name       string
    Quantity   int
    UnitPrice  float64
}

type Order struct {
    ID           string
    CustomerID   string
    RestaurantID string
    Items        []OrderItem
    Status       OrderStatus
    TotalAmount  float64
    DeliveryAddr string
    PlacedAt     time.Time
    UpdatedAt    time.Time
}

type OrderingSystem struct {
    mu          sync.RWMutex
    restaurants map[string]*Restaurant
    orders      map[string]*Order
    observers   []OrderObserver
}

type OrderObserver interface { OnStatusChange(order *Order) }

func (os *OrderingSystem) PlaceOrder(customerID, restaurantID string, items []OrderItem, addr string) (*Or
    os.mu.Lock(); defer os.mu.Unlock()

    rest, ok := os.restaurants[restaurantID]
    if !ok { return nil, errors.New("restaurant not found") }
    if !rest.IsOpen { return nil, errors.New("restaurant is closed") }
```

```go
    var total float64
    for i, item := range items {
        menuItem, ok := rest.Menu[item.MenuItemID]
        if !ok { return nil, fmt.Errorf("item %s not found", item.MenuItemID) }
        if !menuItem.InStock { return nil, fmt.Errorf("item %s out of stock", menuItem.Name) }
        items[i].UnitPrice = menuItem.Price
        items[i].Name = menuItem.Name
        total += menuItem.Price * float64(item.Quantity)
    }

    order := &Order{
        ID:           uuid.New().String(),
        CustomerID:   customerID,
        RestaurantID: restaurantID,
        Items:        items,
        Status:       OrderPlaced,
        TotalAmount:  total,
        DeliveryAddr: addr,
        PlacedAt:     time.Now(),
    }
    os.orders[order.ID] = order
    os.notify(order)
    return order, nil
}

func (os *OrderingSystem) UpdateStatus(orderID string, status OrderStatus) error {
    os.mu.Lock(); defer os.mu.Unlock()
    order, ok := os.orders[orderID]
    if !ok { return errors.New("order not found") }
    order.Status = status
    order.UpdatedAt = time.Now()
    os.notify(order)
    return nil
}

func (os *OrderingSystem) notify(order *Order) {
    for _, obs := range os.observers { go obs.OnStatusChange(order) }
}
```

Interview tip: "For a real system, discuss: payment integration, delivery partner assignment, real-time tracking (WebSocket), ETA calculation, and surge pricing."

Q75

Design a ride-sharing system (like Uber/Ola).

Difficulty: Hard

```go
type Location struct{ Lat, Lng float64 }

func (l Location) DistanceTo(other Location) float64 {
    // Haversine formula (simplified)
    dlat := l.Lat - other.Lat
    dlng := l.Lng - other.Lng
    return math.Sqrt(dlat*dlat + dlng*dlng) * 111 // km
}

type DriverStatus int
const (DriverAvailable DriverStatus = iota; DriverOnRide; DriverOffline)

type Driver struct {
    ID       string
    Name     string
    Location Location
    Status   DriverStatus
    Rating   float64
    Vehicle  string
}

type RideStatus int
const (RideRequested RideStatus = iota; RideAccepted; RideInProgress; RideCompleted; RideCancelled)

type Ride struct {
    ID        string
    RiderID   string
    DriverID  string
    Pickup    Location
    Dropoff   Location
    Status    RideStatus
    Fare      float64
    RequestedAt time.Time
    StartedAt   *time.Time
    EndedAt     *time.Time
}

type RideSystem struct {
    mu      sync.RWMutex
    drivers map[string]*Driver
    rides   map[string]*Ride
}

func (rs *RideSystem) RequestRide(riderID string, pickup, dropoff Location) (*Ride, error) {
    driver, err := rs.findNearestDriver(pickup)
    if err != nil { return nil, err }

    distance := pickup.DistanceTo(dropoff)
    fare := rs.calculateFare(distance)

    ride := &Ride{
        ID:          uuid.New().String(),
        RiderID:     riderID,
```

```go
        DriverID:    driver.ID,
        Pickup:      pickup,
        Dropoff:     dropoff,
        Status:      RideAccepted,
        Fare:        fare,
        RequestedAt: time.Now(),
    }

    rs.mu.Lock()
    rs.rides[ride.ID] = ride
    driver.Status = DriverOnRide
    rs.mu.Unlock()

    return ride, nil
}

func (rs *RideSystem) findNearestDriver(pickup Location) (*Driver, error) {
    rs.mu.RLock(); defer rs.mu.RUnlock()

    var nearest *Driver
    minDist := math.MaxFloat64

    for _, d := range rs.drivers {
        if d.Status != DriverAvailable { continue }
        dist := d.Location.DistanceTo(pickup)
        if dist < minDist { minDist = dist; nearest = d }
    }

    if nearest == nil { return nil, errors.New("no drivers available") }
    return nearest, nil
}

func (rs *RideSystem) calculateFare(distanceKm float64) float64 {
    baseFare := 2.0
    perKm := 1.5
    return baseFare + perKm*distanceKm
}

func (rs *RideSystem) CompleteRide(rideID string) error {
    rs.mu.Lock(); defer rs.mu.Unlock()
    ride, ok := rs.rides[rideID]
    if !ok { return errors.New("ride not found") }

    now := time.Now()
    ride.Status = RideCompleted
    ride.EndedAt = &now
    rs.drivers[ride.DriverID].Status = DriverAvailable
    return nil
}
```

Interview tip: "For scale: use a geospatial index (PostGIS, Redis GEOSEARCH) for nearest-driver lookup. The simple distance scan is O(drivers) — too slow for millions of drivers."

---

9. API & Data Modelling

Q76

Design a RESTful API for a blog system.

Difficulty: Easy

```go
// Resource models
type Post struct {
    ID          string    `json:"id"`
    Title       string    `json:"title"`
    Content     string    `json:"content"`
    AuthorID    string    `json:"author_id"`
    Tags        []string  `json:"tags"`
    PublishedAt *time.Time `json:"published_at,omitempty"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

type CreatePostRequest struct {
    Title   string   `json:"title" validate:"required,min=1,max=200"`
    Content string   `json:"content" validate:"required"`
    Tags    []string `json:"tags"`
}

type UpdatePostRequest struct {
    Title   *string  `json:"title,omitempty"`
    Content *string  `json:"content,omitempty"`
    Tags    []string `json:"tags,omitempty"`
}

// REST endpoints
// POST   /posts              → create post
// GET    /posts              → list posts (with pagination)
// GET    /posts/{id}         → get post by ID
// PUT    /posts/{id}         → full update
// PATCH  /posts/{id}         → partial update
// DELETE /posts/{id}         → delete post
// POST   /posts/{id}/publish → publish post
// GET    /posts/{id}/comments → list comments

// Handler example
type PostHandler struct{ svc PostService }

func (h *PostHandler) Create(w http.ResponseWriter, r *http.Request) {
    var req CreatePostRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, err.Error(), http.StatusBadRequest); return
    }
    userID := r.Context().Value("userID").(string)
    post, err := h.svc.CreatePost(r.Context(), userID, req)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError); return
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(http.StatusCreated)
    json.NewEncoder(w).Encode(post)
}

// Pagination response
```

```go
type PaginatedResponse[T any] struct {
    Data       []T    `json:"data"`
    Total      int    `json:"total"`
    Page       int    `json:"page"`
    PerPage    int    `json:"per_page"`
    TotalPages int    `json:"total_pages"`
    NextCursor string `json:"next_cursor,omitempty"`
}
```

Interview tip: "Cursor-based pagination (`?after=lastID`) is better than offset (`?page=2`) for large datasets. Offset has inconsistency issues when data changes between pages."

Q77

How do you design idempotent APIs?

Difficulty: Medium

Idempotent API: calling it multiple times with the same input produces the same result and has no additional side effects.

```go
type IdempotencyStore interface {
    Get(key string) (*IdempotencyRecord, error)
    Set(key string, rec *IdempotencyRecord, ttl time.Duration) error
}

type IdempotencyRecord struct {
    Status   int
    Body     []byte
    Created  time.Time
}

func IdempotencyMiddleware(store IdempotencyStore) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            idempotencyKey := r.Header.Get("Idempotency-Key")
            if idempotencyKey == "" {
                next.ServeHTTP(w, r); return
            }

            // Check if we've processed this key before
            if rec, err := store.Get(idempotencyKey); err == nil && rec != nil {
                w.WriteHeader(rec.Status)
                w.Write(rec.Body)
                return
            }

            // Process and record result
            rr := &responseRecorder{ResponseWriter: w, code: 200}
            next.ServeHTTP(rr, r)

            rec := &IdempotencyRecord{
                Status:  rr.code,
                Body:    rr.body,
                Created: time.Now(),
            }
            _ = store.Set(idempotencyKey, rec, 24*time.Hour)
        })
    }
}

// Idempotent operations:
// GET — naturally idempotent
// PUT — same data = same state
// DELETE — deleting twice still results in "deleted"
// POST — NOT naturally idempotent (use idempotency-key header)
```

Interview tip: "POST /payments must be idempotent. Client sends Idempotency-Key header. Server deduplicates using Redis. Same key = same response. Critical for payment systems."

Q78

Design a GraphQL-like query resolver in Go.

Difficulty: Hard

```go
type FieldResolver func(ctx context.Context, parent any, args map[string]any) (any, error)

type Schema struct {
    resolvers map[string]map[string]FieldResolver
}

func NewSchema() *Schema {
    return &Schema{resolvers: make(map[string]map[string]FieldResolver)}
}

func (s *Schema) AddResolver(typeName, fieldName string, resolver FieldResolver) {
    if s.resolvers[typeName] == nil {
        s.resolvers[typeName] = make(map[string]FieldResolver)
    }
    s.resolvers[typeName][fieldName] = resolver
}

func (s *Schema) Resolve(ctx context.Context, typeName, fieldName string, parent any, args map[string]any)
    typeResolvers, ok := s.resolvers[typeName]
    if !ok { return nil, fmt.Errorf("type %s not found", typeName) }
    resolver, ok := typeResolvers[fieldName]
    if !ok { return nil, fmt.Errorf("field %s not found on %s", fieldName, typeName) }
    return resolver(ctx, parent, args)
}

// Data loader — batch N+1 problem prevention
type DataLoader[K comparable, V any] struct {
    batchFn   func(keys []K) (map[K]V, error)
    cache     map[K]V
    mu        sync.Mutex
    waiters   map[K][]chan V
}

func (dl *DataLoader[K, V]) Load(key K) (V, error) {
    dl.mu.Lock()
    if val, ok := dl.cache[key]; ok {
        dl.mu.Unlock()
        return val, nil
    }
    // Add to pending batch
    ch := make(chan V, 1)
    dl.waiters[key] = append(dl.waiters[key], ch)
    dl.mu.Unlock()

    val := <-ch
    return val, nil
}
```

Interview tip: "The N+1 problem: fetching 10 posts then fetching author for each = 1+10 = 11 queries. DataLoader batches them into 1+1 = 2 queries. Essential for GraphQL performance."

Q79

Design a webhook delivery system.

Difficulty: Hard

```go
type Webhook struct {
    ID        string
    URL       string
    Secret    string
    Events    []string
    Active    bool
    CreatedAt time.Time
}

type WebhookEvent struct {
    ID        string
    Event     string
    Payload   map[string]any
    CreatedAt time.Time
}

type DeliveryAttempt struct {
    ID          string
    WebhookID   string
    EventID     string
    AttemptNum  int
    Status      int
    Error       string
    AttemptedAt time.Time
}

type WebhookDelivery struct {
    webhooks  []*Webhook
    queue     chan deliveryJob
    client    *http.Client
}

type deliveryJob struct {
    webhook *Webhook
    event   *WebhookEvent
    attempt int
}

func (wd *WebhookDelivery) Deliver(event *WebhookEvent) {
    for _, wh := range wd.webhooks {
        for _, e := range wh.Events {
            if e == event.Event || e == "*" {
                wd.queue <- deliveryJob{webhook: wh, event: event, attempt: 1}
            }
        }
    }
}

func (wd *WebhookDelivery) worker() {
    for job := range wd.queue {
        if err := wd.send(job); err != nil {
            // Retry with exponential backoff
            if job.attempt < 5 {
```

```go
                delay := time.Duration(math.Pow(2, float64(job.attempt))) * time.Second
                time.AfterFunc(delay, func() {
                    wd.queue <- deliveryJob{
                        webhook: job.webhook,
                        event:   job.event,
                        attempt: job.attempt + 1,
                    }
                })
            }
        }
    }
}

func (wd *WebhookDelivery) send(job deliveryJob) error {
    body, _ := json.Marshal(job.event.Payload)
    sig := hmacSignature(job.webhook.Secret, body)

    req, _ := http.NewRequest("POST", job.webhook.URL, bytes.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    req.Header.Set("X-Signature", sig)
    req.Header.Set("X-Event", job.event.Event)

    resp, err := wd.client.Do(req)
    if err != nil { return err }
    defer resp.Body.Close()
    if resp.StatusCode >= 300 {
        return fmt.Errorf("webhook returned %d", resp.StatusCode)
    }
    return nil
}
```

Interview tip: "Webhook delivery must be at-least-once. Use a persistent queue (DB or Kafka) so retries survive process restarts. HMAC signature prevents spoofing."

Q80

Design a real-time chat message model.

Difficulty: Medium

```go
type MessageType int
const (TextMessage MessageType = iota; ImageMessage; FileMessage; SystemMessage)

type Message struct {
    ID          string      `json:"id"`
    ChatID      string      `json:"chat_id"`
    SenderID    string      `json:"sender_id"`
    Type        MessageType `json:"type"`
    Content     string      `json:"content,omitempty"`
    MediaURL    string      `json:"media_url,omitempty"`
    ReplyTo     *string     `json:"reply_to,omitempty"`
    Reactions   map[string][]string `json:"reactions,omitempty"`
    ReadBy      []string    `json:"read_by"`
    EditedAt    *time.Time  `json:"edited_at,omitempty"`
    DeletedAt   *time.Time  `json:"deleted_at,omitempty"`
    SentAt      time.Time   `json:"sent_at"`
}

type Chat struct {
    ID          string    `json:"id"`
    Type        string    `json:"type"` // direct, group
    Name        string    `json:"name,omitempty"`
    Members     []string  `json:"members"`
    LastMessage *Message  `json:"last_message,omitempty"`
    CreatedAt   time.Time `json:"created_at"`
}

// WebSocket hub
type Hub struct {
    clients    map[string]*Client // userID → client
    broadcast  chan *Message
    register   chan *Client
    unregister chan *Client
    mu         sync.RWMutex
}

type Client struct {
    userID string
    conn   *websocket.Conn
    send   chan *Message
}

func (h *Hub) Run() {
    for {
        select {
        case client := <-h.register:
            h.mu.Lock()
            h.clients[client.userID] = client
            h.mu.Unlock()

        case client := <-h.unregister:
            h.mu.Lock()
            delete(h.clients, client.userID)
```

```go
            close(client.send)
            h.mu.Unlock()

        case msg := <-h.broadcast:
            h.mu.RLock()
            // Deliver to all members in the chat
            for _, memberID := range getChatMembers(msg.ChatID) {
                if client, ok := h.clients[memberID]; ok {
                    select {
                    case client.send <- msg:
                    default: // client too slow — disconnect
                        close(client.send)
                        delete(h.clients, memberID)
                    }
                }
            }
            h.mu.RUnlock()
        }
    }
}
```

Interview tip: "WebSocket hub pattern: one goroutine (Run) owns the clients map — no locks needed inside Run. All access goes through channels. Classic Go concurrency pattern."

Q81

Design an event sourcing system.

Difficulty: Hard

Store all changes as events instead of current state. State is derived by replaying events.

```go
type Event struct {
    ID          string
    AggregateID string
    Type        string
    Version     int
    Data        map[string]any
    OccurredAt  time.Time
}

type Aggregate interface {
    ID() string
    Version() int
    Apply(event Event)
}

// Order aggregate
type Order struct {
    id       string
    version  int
    status   string
    items    []OrderItem
    total    float64
    changes  []Event // uncommitted events
}

func (o *Order) ID() string   { return o.id }
func (o *Order) Version() int { return o.version }

func (o *Order) Apply(e Event) {
    switch e.Type {
    case "OrderCreated":
        o.id = e.AggregateID
        o.status = "created"
    case "ItemAdded":
        o.items = append(o.items, OrderItem{
            Name:  e.Data["name"].(string),
            Price: e.Data["price"].(float64),
        })
        o.total += e.Data["price"].(float64)
    case "OrderShipped":
        o.status = "shipped"
    case "OrderCancelled":
        o.status = "cancelled"
    }
    o.version++
}

// Event store
type EventStore struct {
    mu     sync.RWMutex
    events map[string][]Event
}
```

```go
func (es *EventStore) Append(aggregateID string, events []Event, expectedVersion int) error {
    es.mu.Lock(); defer es.mu.Unlock()
    existing := es.events[aggregateID]
    if len(existing) != expectedVersion {
        return errors.New("optimistic concurrency conflict")
    }
    es.events[aggregateID] = append(existing, events...)
    return nil
}

func (es *EventStore) Load(aggregateID string) (*Order, error) {
    es.mu.RLock(); defer es.mu.RUnlock()
    events, ok := es.events[aggregateID]
    if !ok { return nil, errors.New("aggregate not found") }
    order := &Order{}
    for _, e := range events { order.Apply(e) }
    return order, nil
}
```

Interview tip: "Event sourcing gives you a full audit log, time travel (replay to any point), and easy event-driven integration. Trade-off: complex queries require projections/read models."

Q82

How do you implement optimistic locking in Go?

Difficulty: Medium

Use version numbers to detect concurrent modifications without blocking.

```go
type VersionedEntity struct {
    ID      string
    Data    map[string]any
    Version int64
}

type Repository struct {
    mu    sync.RWMutex
    store map[string]*VersionedEntity
}

func (r *Repository) Get(id string) (*VersionedEntity, error) {
    r.mu.RLock(); defer r.mu.RUnlock()
    e, ok := r.store[id]
    if !ok { return nil, errors.New("not found") }
    // Return a copy
    copy := *e
    return &copy, nil
}

func (r *Repository) Update(entity *VersionedEntity) error {
    r.mu.Lock(); defer r.mu.Unlock()

    current, ok := r.store[entity.ID]
    if !ok { return errors.New("not found") }

    // Optimistic lock check
    if current.Version != entity.Version {
        return errors.New("version conflict: entity was modified by another process")
    }

    entity.Version++ // increment version
    r.store[entity.ID] = entity
    return nil
}

// Usage — read-modify-write with retry on conflict
func updateWithRetry(repo *Repository, id string, modify func(*VersionedEntity)) error {
    for attempts := 0; attempts < 3; attempts++ {
        entity, err := repo.Get(id)
        if err != nil { return err }

        modify(entity) // apply changes

        if err := repo.Update(entity); err != nil {
            if err.Error() == "version conflict: entity was modified by another process" {
                time.Sleep(time.Duration(attempts*10) * time.Millisecond)
                continue // retry
            }
            return err
        }
        return nil
    }
```

```go
    return errors.New("too many conflicts")
}
```

Interview tip: "Optimistic locking beats pessimistic locking (SELECT FOR UPDATE) when conflicts are rare. SQL equivalent: UPDATE ... WHERE version=? and check rows affected = 1."

---

10. Real-World LLD Problems

Q83

Design a notification dispatcher with multiple channels.

Difficulty: Hard

```go
type NotificationChannel int
const (ChannelEmail NotificationChannel = iota; ChannelSMS; ChannelPush; ChannelSlack)

type Notification struct {
    ID        string
    UserID    string
    Type      string
    Title     string
    Body      string
    Channels  []NotificationChannel
    Priority  int
    CreatedAt time.Time
}

type ChannelSender interface {
    Send(ctx context.Context, userID, title, body string) error
    Channel() NotificationChannel
}

type Dispatcher struct {
    senders  map[NotificationChannel]ChannelSender
    prefs    UserPreferenceStore
    queue    chan *Notification
    workers  int
}

func (d *Dispatcher) Dispatch(ctx context.Context, notif *Notification) error {
    prefs, err := d.prefs.Get(ctx, notif.UserID)
    if err != nil { return err }

    for _, ch := range notif.Channels {
        if !prefs.IsEnabled(ch) { continue }
        if prefs.InQuietHours() && notif.Priority < 10 { continue }

        sender, ok := d.senders[ch]
        if !ok { continue }

        go func(s ChannelSender) {
            if err := s.Send(ctx, notif.UserID, notif.Title, notif.Body); err != nil {
                log.Printf("failed to send via %v: %v", s.Channel(), err)
            }
        }(sender)
    }
    return nil
}

// Email sender
type EmailSender struct{ client *smtp.Client }
func (e *EmailSender) Channel() NotificationChannel { return ChannelEmail }
func (e *EmailSender) Send(ctx context.Context, userID, title, body string) error {
    email := lookupEmail(userID)
    return e.client.SendMail(email, title, body)
}
```

Q84

Design a file upload service.

Difficulty: Medium

```go
type FileMetadata struct {
    ID          string
    UserID      string
    FileName    string
    ContentType string
    Size        int64
    StoragePath string
    UploadedAt  time.Time
    Status      string
}

type FileStore interface {
    Upload(ctx context.Context, key string, r io.Reader, size int64, contentType string) error
    Download(ctx context.Context, key string) (io.ReadCloser, error)
    Delete(ctx context.Context, key string) error
    GetURL(key string) string
}

type FileUploadService struct {
    store    FileStore
    metadata MetadataStore
    maxSize  int64
    allowed  map[string]bool // allowed MIME types
}

func (s *FileUploadService) Upload(ctx context.Context, userID string, r io.Reader, filename string, size i
    if size > s.maxSize {
        return nil, fmt.Errorf("file too large: max %d bytes", s.maxSize)
    }
    if !s.allowed[contentType] {
        return nil, fmt.Errorf("content type %s not allowed", contentType)
    }

    id := uuid.New().String()
    ext := filepath.Ext(filename)
    key := fmt.Sprintf("uploads/%s/%s%s", userID, id, ext)

    if err := s.store.Upload(ctx, key, r, size, contentType); err != nil {
        return nil, fmt.Errorf("upload failed: %w", err)
    }

    meta := &FileMetadata{
        ID:          id,
        UserID:      userID,
        FileName:    filename,
        ContentType: contentType,
        Size:        size,
        StoragePath: key,
        UploadedAt:  time.Now(),
        Status:      "uploaded",
    }

    if err := s.metadata.Save(ctx, meta); err != nil {
```

```go
        _ = s.store.Delete(ctx, key) // rollback
        return nil, err
    }
    return meta, nil
}

func (s *FileUploadService) GetPresignedURL(ctx context.Context, fileID string, expiry time.Duration) (str
    meta, err := s.metadata.Get(ctx, fileID)
    if err != nil { return "", err }
    return s.store.GetURL(meta.StoragePath), nil
}
```

Interview tip: "Chunked upload for large files: split into 5MB chunks, upload concurrently, assemble on server (S3 multipart upload). Virus scanning before making public. Mention both."

Q85

Design a search autocomplete system.

Difficulty: Hard

```go
type TrieNode struct {
    children map[rune]*TrieNode
    isEnd    bool
    score    int // ranking score
    word     string
}

type Trie struct {
    root *TrieNode
}

func NewTrie() *Trie {
    return &Trie{root: &TrieNode{children: make(map[rune]*TrieNode)}}
}

func (t *Trie) Insert(word string, score int) {
    node := t.root
    for _, ch := range word {
        if node.children[ch] == nil {
            node.children[ch] = &TrieNode{children: make(map[rune]*TrieNode)}
        }
        node = node.children[ch]
    }
    node.isEnd = true
    node.score = score
    node.word = word
}

func (t *Trie) Search(prefix string, limit int) []string {
    node := t.root
    for _, ch := range prefix {
        if node.children[ch] == nil { return nil }
        node = node.children[ch]
    }
    return t.collectWords(node, limit)
}

func (t *Trie) collectWords(node *TrieNode, limit int) []string {
    type candidate struct{ word string; score int }
    var results []candidate

    var dfs func(n *TrieNode)
    dfs = func(n *TrieNode) {
        if n.isEnd { results = append(results, candidate{n.word, n.score}) }
        for _, child := range n.children { dfs(child) }
    }
    dfs(node)

    sort.Slice(results, func(i, j int) bool {
        return results[i].score > results[j].score
    })

    words := make([]string, 0, limit)
```

```go
    for i, r := range results {
        if i >= limit { break }
        words = append(words, r.word)
    }
    return words
}

// Concurrent-safe autocomplete with TTL for trending updates
type AutocompleteService struct {
    mu   sync.RWMutex
    trie *Trie
}

func (a *AutocompleteService) Suggest(prefix string) []string {
    a.mu.RLock(); defer a.mu.RUnlock()
    return a.trie.Search(strings.ToLower(prefix), 10)
}

func (a *AutocompleteService) UpdateScores(searches map[string]int) {
    newTrie := NewTrie()
    for word, score := range searches { newTrie.Insert(word, score) }
    a.mu.Lock(); a.trie = newTrie; a.mu.Unlock()
}
```

Interview tip: "Production: use Elasticsearch or a custom Trie stored in Redis. For ranking: click-through rate, recency, personal history. Trie in RAM works up to ~10M words."

Q86

Design a task queue with priorities and delayed execution.

Difficulty: Hard

```go
type TaskStatus int
const (TaskPending TaskStatus = iota; TaskRunning; TaskDone; TaskFailed)

type Task struct {
    ID        string
    Type      string
    Payload   []byte
    Priority  int
    RunAfter  time.Time
    MaxRetries int
    Retries   int
    Status    TaskStatus
}

type TaskQueue struct {
    mu      sync.Mutex
    heap    *taskHeap
    cond    *sync.Cond
    workers int
    handlers map[string]func([]byte) error
}

func NewTaskQueue(workers int) *TaskQueue {
    h := &taskHeap{}
    heap.Init(h)
    tq := &TaskQueue{heap: h, workers: workers, handlers: make(map[string]func([]byte) error)}
    tq.cond = sync.NewCond(&tq.mu)
    for i := 0; i < workers; i++ { go tq.runWorker() }
    return tq
}

func (tq *TaskQueue) RegisterHandler(taskType string, fn func([]byte) error) {
    tq.handlers[taskType] = fn
}

func (tq *TaskQueue) Enqueue(task *Task) {
    tq.mu.Lock()
    heap.Push(tq.heap, task)
    tq.cond.Signal()
    tq.mu.Unlock()
}

func (tq *TaskQueue) runWorker() {
    for {
        tq.mu.Lock()
        for tq.heap.Len() == 0 { tq.cond.Wait() }
        task := (*tq.heap)[0]

        if task.RunAfter.After(time.Now()) {
            tq.mu.Unlock()
            time.Sleep(100 * time.Millisecond)
            continue
        }
```

```go
        heap.Pop(tq.heap)
        task.Status = TaskRunning
        tq.mu.Unlock()

        handler, ok := tq.handlers[task.Type]
        if !ok {
            log.Printf("no handler for task type: %s", task.Type)
            continue
        }

        if err := handler(task.Payload); err != nil {
            task.Retries++
            if task.Retries <= task.MaxRetries {
                task.Status = TaskPending
                task.RunAfter = time.Now().Add(time.Duration(task.Retries) * time.Minute)
                tq.Enqueue(task)
            } else {
                task.Status = TaskFailed
            }
        } else {
            task.Status = TaskDone
        }
    }
}
```

Interview tip: "For production: persist tasks in DB (Postgres, Redis). In-memory queue loses tasks on restart. Libraries: asynq (Redis-backed), River (Postgres-backed)."

Q87

Design a permission/RBAC system.

Difficulty: Medium

```go
type Permission string
type Role string

const (
    PermRead   Permission = "read"
    PermWrite  Permission = "write"
    PermDelete Permission = "delete"
    PermAdmin  Permission = "admin"
)

type RBACSystem struct {
    mu          sync.RWMutex
    roles       map[Role][]Permission
    userRoles   map[string][]Role
    roleParents map[Role][]Role // role inheritance
}

func NewRBAC() *RBACSystem {
    return &RBACSystem{
        roles:       make(map[Role][]Permission),
        userRoles:   make(map[string][]Role),
        roleParents: make(map[Role][]Role),
    }
}

func (r *RBACSystem) DefineRole(role Role, perms []Permission, parents ...Role) {
    r.mu.Lock(); defer r.mu.Unlock()
    r.roles[role] = perms
    r.roleParents[role] = parents
}

func (r *RBACSystem) AssignRole(userID string, role Role) {
    r.mu.Lock(); defer r.mu.Unlock()
    r.userRoles[userID] = append(r.userRoles[userID], role)
}

func (r *RBACSystem) HasPermission(userID string, perm Permission) bool {
    r.mu.RLock(); defer r.mu.RUnlock()
    visited := map[Role]bool{}
    for _, role := range r.userRoles[userID] {
        if r.roleHasPerm(role, perm, visited) { return true }
    }
    return false
}

func (r *RBACSystem) roleHasPerm(role Role, perm Permission, visited map[Role]bool) bool {
    if visited[role] { return false }
    visited[role] = true

    for _, p := range r.roles[role] {
        if p == perm || p == PermAdmin { return true }
    }
    for _, parent := range r.roleParents[role] {
```

```go
        if r.roleHasPerm(parent, perm, visited) { return true }
    }
    return false
}

// Usage
rbac := NewRBAC()
rbac.DefineRole("reader", []Permission{PermRead})
rbac.DefineRole("editor", []Permission{PermRead, PermWrite}, "reader")
rbac.DefineRole("admin", []Permission{PermAdmin})

rbac.AssignRole("alice", "editor")
rbac.AssignRole("bob", "reader")

fmt.Println(rbac.HasPermission("alice", PermWrite)) // true
fmt.Println(rbac.HasPermission("bob", PermWrite))   // false
```

Interview tip: "RBAC vs ABAC: RBAC is role-based (simpler). ABAC is attribute-based (user, resource, environment attributes — more flexible but complex). RBAC handles 95% of use cases."

Q88

Design a shopping cart system.

Difficulty: Medium

```go
type CartItem struct {
    ProductID string
    Name      string
    Price     float64
    Quantity  int
}

func (item CartItem) Subtotal() float64 { return item.Price * float64(item.Quantity) }

type Cart struct {
    ID         string
    UserID     string
    Items      map[string]*CartItem
    CouponCode string
    UpdatedAt  time.Time
}

func NewCart(userID string) *Cart {
    return &Cart{
        ID:     uuid.New().String(),
        UserID: userID,
        Items:  make(map[string]*CartItem),
    }
}

func (c *Cart) AddItem(productID, name string, price float64, qty int) {
    if item, ok := c.Items[productID]; ok {
        item.Quantity += qty
    } else {
        c.Items[productID] = &CartItem{productID, name, price, qty}
    }
    c.UpdatedAt = time.Now()
}

func (c *Cart) RemoveItem(productID string) error {
    if _, ok := c.Items[productID]; !ok {
        return errors.New("item not in cart")
    }
    delete(c.Items, productID)
    c.UpdatedAt = time.Now()
    return nil
}

func (c *Cart) UpdateQuantity(productID string, qty int) error {
    item, ok := c.Items[productID]
    if !ok { return errors.New("item not in cart") }
    if qty <= 0 { delete(c.Items, productID); return nil }
    item.Quantity = qty
    c.UpdatedAt = time.Now()
    return nil
}

func (c *Cart) Subtotal() float64 {
```

```go
    var total float64
    for _, item := range c.Items { total += item.Subtotal() }
    return total
}

func (c *Cart) ApplyCoupon(code string, discount DiscountService) (float64, error) {
    d, err := discount.GetDiscount(code)
    if err != nil { return 0, err }
    c.CouponCode = code
    return c.Subtotal() * (1 - d.Rate), nil
}

func (c *Cart) ItemCount() int {
    count := 0
    for _, item := range c.Items { count += item.Quantity }
    return count
}
```

Interview tip: "Cart storage: session-based (Redis, TTL 30min) for guests, DB-backed for logged-in users. Merge carts on login. Stock validation at checkout, not add-to-cart — avoids over-locking."

Q89

Design a simple key-value store with TTL and persistence.

Difficulty: Hard

```go
type StoreEntry struct {
    Value     []byte
    ExpiresAt *time.Time
}

type KVStore struct {
    mu      sync.RWMutex
    data    map[string]*StoreEntry
    walPath string
    walFile *os.File
}

func NewKVStore(walPath string) (*KVStore, error) {
    store := &KVStore{data: make(map[string]*StoreEntry), walPath: walPath}
    if err := store.loadWAL(); err != nil { return nil, err }
    f, err := os.OpenFile(walPath, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
    if err != nil { return nil, err }
    store.walFile = f
    go store.cleanup()
    return store, nil
}

func (s *KVStore) Set(key string, val []byte, ttl *time.Duration) error {
    s.mu.Lock(); defer s.mu.Unlock()
    entry := &StoreEntry{Value: val}
    if ttl != nil { exp := time.Now().Add(*ttl); entry.ExpiresAt = &exp }
    s.data[key] = entry
    return s.writeWAL("SET", key, val, entry.ExpiresAt)
}

func (s *KVStore) Get(key string) ([]byte, bool) {
    s.mu.RLock(); defer s.mu.RUnlock()
    entry, ok := s.data[key]
    if !ok { return nil, false }
    if entry.ExpiresAt != nil && time.Now().After(*entry.ExpiresAt) { return nil, false }
    return entry.Value, true
}

func (s *KVStore) Delete(key string) {
    s.mu.Lock(); defer s.mu.Unlock()
    delete(s.data, key)
    _ = s.writeWAL("DEL", key, nil, nil)
}

func (s *KVStore) writeWAL(op, key string, val []byte, exp *time.Time) error {
    record := struct {
        Op  string `json:"op"`
        Key string `json:"key"`
        Val []byte `json:"val,omitempty"`
        Exp *time.Time `json:"exp,omitempty"`
    }{op, key, val, exp}
    data, _ := json.Marshal(record)
    data = append(data, '\n')
```

```go
    _, err := s.walFile.Write(data)
    return err
}

func (s *KVStore) cleanup() {
    ticker := time.NewTicker(time.Minute)
    for range ticker.C {
        s.mu.Lock()
        now := time.Now()
        for k, e := range s.data {
            if e.ExpiresAt != nil && now.After(*e.ExpiresAt) { delete(s.data, k) }
        }
        s.mu.Unlock()
    }
}
```

Interview tip: "WAL (Write-Ahead Log) provides durability. On startup, replay WAL to restore state. Compact WAL periodically to prevent unbounded growth. This is how Redis AOF works."

Q90

Design a log aggregation system.

Difficulty: Hard

```go
type LogLevel int
const (DEBUG LogLevel = iota; INFO; WARN; ERROR; FATAL)

type LogEntry struct {
    ID        string
    Service   string
    Level     LogLevel
    Message   string
    Fields    map[string]any
    Timestamp time.Time
    TraceID   string
}

type LogIngester struct {
    buffer   chan *LogEntry
    batch    []*LogEntry
    batchMu  sync.Mutex
    maxBatch int
    flushInterval time.Duration
    store    LogStore
}

func NewLogIngester(store LogStore, maxBatch int, flushInterval time.Duration) *LogIngester {
    li := &LogIngester{
        buffer:        make(chan *LogEntry, 10000),
        maxBatch:      maxBatch,
        flushInterval: flushInterval,
        store:         store,
    }
    go li.process()
    return li
}

func (li *LogIngester) Ingest(entry *LogEntry) {
    select {
    case li.buffer <- entry:
    default:
        // Buffer full — drop with metric increment
        log.Println("log buffer full, dropping entry")
    }
}

func (li *LogIngester) process() {
    ticker := time.NewTicker(li.flushInterval)
    defer ticker.Stop()
    for {
        select {
        case entry := <-li.buffer:
            li.batchMu.Lock()
            li.batch = append(li.batch, entry)
            shouldFlush := len(li.batch) >= li.maxBatch
            li.batchMu.Unlock()
            if shouldFlush { li.flush() }
```

```go
        case <-ticker.C:
            li.flush()
        }
    }
}

func (li *LogIngester) flush() {
    li.batchMu.Lock()
    if len(li.batch) == 0 { li.batchMu.Unlock(); return }
    toFlush := li.batch
    li.batch = make([]*LogEntry, 0, li.maxBatch)
    li.batchMu.Unlock()

    if err := li.store.BulkInsert(context.Background(), toFlush); err != nil {
        log.Printf("failed to flush %d logs: %v", len(toFlush), err)
    }
}

type LogQuery struct {
    Service   string
    Level     *LogLevel
    From, To  time.Time
    TraceID   string
    Message   string
    Limit     int
}
```

Interview tip: "Key design decisions: buffered channel prevents blocking callers, drop on full buffer (logs are lossy by design), batched writes reduce DB pressure, ticker ensures flush even on low volume."

Q91

Design a coupon/discount system.

Difficulty: Medium

```go
type DiscountType int
const (PercentOff DiscountType = iota; FixedAmountOff; FreeShipping; BuyXGetY)

type Coupon struct {
    Code          string
    Type          DiscountType
    Value         float64 // percent (0.2=20%) or fixed amount
    MinOrderValue float64
    MaxUses       int
    UsedCount     int
    PerUserLimit  int
    ValidFrom     time.Time
    ValidUntil    time.Time
    ApplicableTo  []string // product categories
    Active        bool
}

type CouponUsage struct {
    CouponCode string
    UserID     string
    OrderID    string
    UsedAt     time.Time
    Discount   float64
}

type CouponService struct {
    mu     sync.RWMutex
    coupons map[string]*Coupon
    usage   []CouponUsage
}

func (s *CouponService) Validate(code, userID string, orderValue float64) (*Coupon, error) {
    s.mu.RLock(); defer s.mu.RUnlock()

    coupon, ok := s.coupons[code]
    if !ok { return nil, errors.New("coupon not found") }
    if !coupon.Active { return nil, errors.New("coupon inactive") }

    now := time.Now()
    if now.Before(coupon.ValidFrom)   { return nil, errors.New("coupon not yet valid") }
    if now.After(coupon.ValidUntil)   { return nil, errors.New("coupon expired") }
    if coupon.UsedCount >= coupon.MaxUses { return nil, errors.New("coupon fully redeemed") }
    if orderValue < coupon.MinOrderValue {
        return nil, fmt.Errorf("minimum order $%.2f required", coupon.MinOrderValue)
    }

    // Check per-user limit
    userUses := 0
    for _, u := range s.usage {
        if u.CouponCode == code && u.UserID == userID { userUses++ }
    }
    if userUses >= coupon.PerUserLimit {
        return nil, errors.New("coupon usage limit reached for this user")
```

```go
    }

    return coupon, nil
}

func (s *CouponService) Apply(coupon *Coupon, orderValue float64) float64 {
    switch coupon.Type {
    case PercentOff:
        return orderValue * coupon.Value
    case FixedAmountOff:
        if coupon.Value > orderValue { return orderValue }
        return coupon.Value
    case FreeShipping:
        return 0 // shipping handled separately
    }
    return 0
}

func (s *CouponService) Redeem(code, userID, orderID string, discount float64) {
    s.mu.Lock(); defer s.mu.Unlock()
    s.coupons[code].UsedCount++
    s.usage = append(s.usage, CouponUsage{code, userID, orderID, time.Now(), discount})
}
```

Interview tip: "Concurrent coupon redemption: two users validating the same last-use coupon simultaneously. Use Redis INCR + check pattern or DB transaction with SELECT FOR UPDATE."

Q92

Design a content moderation system.

Difficulty: Hard

```go
type ContentType int
const (TextContent ContentType = iota; ImageContent; VideoContent)

type ModerationResult int
const (ModerationApproved ModerationResult = iota; ModerationRejected; ModerationPending; ModerationEscalat

type ContentItem struct {
    ID          string
    UserID      string
    Type        ContentType
    Content     string
    MediaURL    string
    SubmittedAt time.Time
    Status      ModerationResult
    ReviewedBy  string
    ReviewedAt  *time.Time
    Reason      string
}

type Moderator interface {
    Moderate(ctx context.Context, item *ContentItem) (ModerationResult, string, error)
    Priority() int
}

// Auto moderators
type ProfanityFilter struct{}
func (p *ProfanityFilter) Priority() int { return 1 }
func (p *ProfanityFilter) Moderate(ctx context.Context, item *ContentItem) (ModerationResult, string, erro
    if containsProfanity(item.Content) {
        return ModerationRejected, "contains profanity", nil
    }
    return ModerationApproved, "", nil
}

type SpamDetector struct{}
func (s *SpamDetector) Priority() int { return 2 }
func (s *SpamDetector) Moderate(ctx context.Context, item *ContentItem) (ModerationResult, string, error) {
    score := calculateSpamScore(item)
    if score > 0.9 { return ModerationRejected, "spam", nil }
    if score > 0.7 { return ModerationEscalated, "possible spam — human review", nil }
    return ModerationApproved, "", nil
}

type ModerationPipeline struct {
    auto       []Moderator
    humanQueue chan *ContentItem
}

func (mp *ModerationPipeline) Process(ctx context.Context, item *ContentItem) ModerationResult {
    sort.Slice(mp.auto, func(i, j int) bool {
        return mp.auto[i].Priority() < mp.auto[j].Priority()
    })
```

```go
    for _, mod := range mp.auto {
        result, reason, err := mod.Moderate(ctx, item)
        if err != nil { continue }
        if result == ModerationRejected {
            item.Status = ModerationRejected
            item.Reason = reason
            return result
        }
        if result == ModerationEscalated {
            item.Status = ModerationPending
            mp.humanQueue <- item // send to human review queue
            return ModerationPending
        }
    }

    item.Status = ModerationApproved
    return ModerationApproved
}
```

Interview tip: "Multi-stage pipeline: cheap fast checks first (profanity filter), expensive checks last (ML model). Escalation to human review for borderline cases. Record all decisions for appeal."

Q93

Design an audit log system.

Difficulty: Medium

```go
type AuditAction string
const (ActionCreate AuditAction = "create"; ActionUpdate = "update"; ActionDelete = "delete"; ActionAccess

type AuditLog struct {
    ID          string
    ActorID     string
    ActorType   string // user, service, admin
    Action      AuditAction
    Resource    string
    ResourceID  string
    OldValue    map[string]any
    NewValue    map[string]any
    IP          string
    UserAgent   string
    TraceID     string
    OccurredAt  time.Time
    Success     bool
    ErrorMsg    string
}

type AuditLogger struct {
    store  AuditStore
    buffer chan *AuditLog
}

func NewAuditLogger(store AuditStore) *AuditLogger {
    al := &AuditLogger{store: store, buffer: make(chan *AuditLog, 1000)}
    go al.flush()
    return al
}

func (al *AuditLogger) Log(log *AuditLog) {
    log.ID = uuid.New().String()
    log.OccurredAt = time.Now()
    select {
    case al.buffer <- log:
    default:
        // Buffer full — write synchronously (audit logs must not be lost)
        _ = al.store.Save(context.Background(), log)
    }
}

func (al *AuditLogger) flush() {
    var batch []*AuditLog
    ticker := time.NewTicker(100 * time.Millisecond)
    for {
        select {
        case log := <-al.buffer:
            batch = append(batch, log)
            if len(batch) >= 100 {
                al.store.BulkSave(context.Background(), batch)
                batch = batch[:0]
            }
```

```go
        case <-ticker.C:
            if len(batch) > 0 {
                al.store.BulkSave(context.Background(), batch)
                batch = batch[:0]
            }
        }
    }
}

// Middleware to auto-audit HTTP requests
func AuditMiddleware(logger *AuditLogger) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            rr := &responseRecorder{ResponseWriter: w}
            next.ServeHTTP(rr, r)
            logger.Log(&AuditLog{
                ActorID:  r.Context().Value("userID").(string),
                Action:   methodToAction(r.Method),
                Resource: r.URL.Path,
                IP:       r.RemoteAddr,
                Success:  rr.code < 400,
            })
        })
    }
}
```

Interview tip: "Audit logs must be append-only and tamper-evident. Use write-once storage (S3) or cryptographic chaining (each log entry includes hash of previous). GDPR: audit logs may need selective deletion."

Q94

Design a distributed lock manager.

Difficulty: Hard

```go
// Redis-based distributed lock
type DistributedLock struct {
    rdb     *redis.Client
    key     string
    token   string
    ttl     time.Duration
    stopCh  chan struct{}
}

func AcquireLock(ctx context.Context, rdb *redis.Client, key string, ttl time.Duration) (*DistributedLock,
    token := uuid.New().String()
    lockKey := "lock:" + key

    // SET NX PX — atomic set if not exists
    success, err := rdb.SetNX(ctx, lockKey, token, ttl).Result()
    if err != nil { return nil, err }
    if !success { return nil, errors.New("lock already held") }

    lock := &DistributedLock{rdb: rdb, key: lockKey, token: token, ttl: ttl, stopCh: make(chan struct{})}
    go lock.keepAlive() // auto-renew before expiry
    return lock, nil
}

// Lua script ensures atomic release — only release if we own the lock
const releaseScript = `
if redis.call("GET", KEYS[1]) == ARGV[1] then
    return redis.call("DEL", KEYS[1])
else
    return 0
end`

func (l *DistributedLock) Release(ctx context.Context) error {
    close(l.stopCh)
    result, err := l.rdb.Eval(ctx, releaseScript, []string{l.key}, l.token).Int()
    if err != nil { return err }
    if result == 0 { return errors.New("lock not owned or already expired") }
    return nil
}

func (l *DistributedLock) keepAlive() {
    ticker := time.NewTicker(l.ttl / 3)
    defer ticker.Stop()
    for {
        select {
        case <-ticker.C:
            ctx, cancel := context.WithTimeout(context.Background(), time.Second)
            l.rdb.Expire(ctx, l.key, l.ttl)
            cancel()
        case <-l.stopCh:
            return
        }
    }
}
```

```go
// Usage
func withLock(ctx context.Context, rdb *redis.Client, resource string, fn func() error) error {
    lock, err := AcquireLock(ctx, rdb, resource, 30*time.Second)
    if err != nil { return fmt.Errorf("cannot acquire lock: %w", err) }
    defer lock.Release(ctx)
    return fn()
}
```

Interview tip: "The Lua release script is critical — without it, a process could release another process's lock (if the original lock expired). Redlock algorithm for multi-node HA."

Q95

Design an API gateway (lightweight).

Difficulty: Hard

```go
type Route struct {
    Path       string
    Target     string
    Methods    []string
    Middleware  []string
}

type APIGateway struct {
    routes      []*Route
    rateLimiter *PerUserLimiter
    circuit     map[string]*CircuitBreaker
    client      *http.Client
    mu          sync.RWMutex
}

func NewAPIGateway(routes []*Route) *APIGateway {
    gw := &APIGateway{
        routes:      routes,
        rateLimiter: NewPerUserLimiter(100, 200),
        circuit:     make(map[string]*CircuitBreaker),
        client:      &http.Client{Timeout: 30 * time.Second},
    }
    for _, r := range routes {
        gw.circuit[r.Target] = NewCircuitBreaker(5, 2, 30*time.Second)
    }
    return gw
}

func (gw *APIGateway) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    route := gw.matchRoute(r.URL.Path, r.Method)
    if route == nil { http.NotFound(w, r); return }

    // Rate limiting
    userID := r.Header.Get("X-User-ID")
    if !gw.rateLimiter.Allow(userID) {
        http.Error(w, "rate limit exceeded", http.StatusTooManyRequests); return
    }

    // Circuit breaker
    cb := gw.circuit[route.Target]
    err := cb.Execute(func() error {
        return gw.proxy(w, r, route.Target)
    })
    if err != nil {
        if strings.Contains(err.Error(), "circuit open") {
            http.Error(w, "service temporarily unavailable", http.StatusServiceUnavailable)
        }
    }
}

func (gw *APIGateway) proxy(w http.ResponseWriter, r *http.Request, target string) error {
    url := target + r.URL.RequestURI()
    req, err := http.NewRequestWithContext(r.Context(), r.Method, url, r.Body)
```

```go
    if err != nil { return err }

    // Forward headers
    for k, v := range r.Header { req.Header[k] = v }
    req.Header.Set("X-Forwarded-For", r.RemoteAddr)

    resp, err := gw.client.Do(req)
    if err != nil { return err }
    defer resp.Body.Close()

    for k, v := range resp.Header { w.Header()[k] = v }
    w.WriteHeader(resp.StatusCode)
    _, err = io.Copy(w, resp.Body)
    return err
}

func (gw *APIGateway) matchRoute(path, method string) *Route {
    for _, r := range gw.routes {
        if strings.HasPrefix(path, r.Path) {
            for _, m := range r.Methods { if m == method { return r } }
        }
    }
    return nil
}
```

Interview tip: "Real API gateways add: authentication/JWT validation, request/response transformation, load balancing, SSL termination, API versioning, and observability. Kong, Nginx, Envoy do all this in production."

Q96

Design a search indexing system.

Difficulty: Hard

```go
type Document struct {
    ID      string
    Title   string
    Content string
    Tags    []string
}

type InvertedIndex struct {
    mu    sync.RWMutex
    index map[string]map[string]int // term → {docID → frequency}
    docs  map[string]*Document
}

func NewInvertedIndex() *InvertedIndex {
    return &InvertedIndex{
        index: make(map[string]map[string]int),
        docs:  make(map[string]*Document),
    }
}

func (idx *InvertedIndex) Index(doc *Document) {
    idx.mu.Lock(); defer idx.mu.Unlock()

    idx.docs[doc.ID] = doc
    terms := tokenize(doc.Title + " " + doc.Content)

    for _, term := range terms {
        if idx.index[term] == nil { idx.index[term] = make(map[string]int) }
        idx.index[term][doc.ID]++
    }
}

func (idx *InvertedIndex) Search(query string) []*SearchResult {
    idx.mu.RLock(); defer idx.mu.RUnlock()

    terms := tokenize(query)
    scores := make(map[string]float64)

    for _, term := range terms {
        docs, ok := idx.index[term]
        if !ok { continue }

        // TF-IDF scoring
        idf := math.Log(float64(len(idx.docs)) / float64(len(docs)))
        for docID, freq := range docs {
            tf := float64(freq) / float64(wordCount(idx.docs[docID]))
            scores[docID] += tf * idf
        }
    }

    var results []*SearchResult
    for docID, score := range scores {
        results = append(results, &SearchResult{Doc: idx.docs[docID], Score: score})
```

```go
    }
    sort.Slice(results, func(i, j int) bool { return results[i].Score > results[j].Score })
    return results
}

type SearchResult struct {
    Doc   *Document
    Score float64
}

func tokenize(text string) []string {
    text = strings.ToLower(text)
    words := strings.Fields(text)
    var tokens []string
    for _, w := range words {
        w = strings.Trim(w, ".,!?;:\"'")
        if len(w) > 2 && !isStopWord(w) { tokens = append(tokens, w) }
    }
    return tokens
}
```

Interview tip: "Production search: Elasticsearch uses inverted index + BM25 scoring + distributed sharding. Highlight why TF-IDF beats simple keyword matching in interviews."

Q97

Design a leaderboard system.

Difficulty: Medium

```go
type LeaderboardEntry struct {
    UserID   string
    Username string
    Score    float64
    Rank     int
}

type Leaderboard struct {
    mu       sync.RWMutex
    scores   map[string]float64
    userInfo map[string]string // userID → username
}

func NewLeaderboard() *Leaderboard {
    return &Leaderboard{
        scores:   make(map[string]float64),
        userInfo: make(map[string]string),
    }
}

func (l *Leaderboard) AddScore(userID, username string, delta float64) float64 {
    l.mu.Lock(); defer l.mu.Unlock()
    l.scores[userID] += delta
    l.userInfo[userID] = username
    return l.scores[userID]
}

func (l *Leaderboard) SetScore(userID, username string, score float64) {
    l.mu.Lock(); defer l.mu.Unlock()
    l.scores[userID] = score
    l.userInfo[userID] = username
}

func (l *Leaderboard) TopN(n int) []*LeaderboardEntry {
    l.mu.RLock(); defer l.mu.RUnlock()

    type entry struct{ id string; score float64 }
    all := make([]entry, 0, len(l.scores))
    for id, score := range l.scores { all = append(all, entry{id, score}) }
    sort.Slice(all, func(i, j int) bool { return all[i].score > all[j].score })

    result := make([]*LeaderboardEntry, 0, n)
    for i, e := range all {
        if i >= n { break }
        result = append(result, &LeaderboardEntry{
            UserID:   e.id,
            Username: l.userInfo[e.id],
            Score:    e.score,
            Rank:     i + 1,
        })
    }
    return result
}
```

```go
func (l *Leaderboard) GetRank(userID string) int {
    l.mu.RLock(); defer l.mu.RUnlock()
    myScore := l.scores[userID]
    rank := 1
    for id, score := range l.scores {
        if id != userID && score > myScore { rank++ }
    }
    return rank
}
```

Interview tip: "For production leaderboards: Redis ZADD/ZRANK/ZREVRANGE. O(log N) for add/update, O(log N + M) for top-M. Millions of players — in-memory sort is too slow."

Q98

Design a feature flag system.

Difficulty: Medium

```go
type RolloutType int
const (RolloutAll RolloutType = iota; RolloutPercentage; RolloutUserList; RolloutUserSegment)

type FeatureFlag struct {
    Name        string
    Enabled     bool
    Rollout     RolloutType
    Percentage  float64
    AllowList   []string
    Segments    []string
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

type FeatureFlagService struct {
    mu    sync.RWMutex
    flags map[string]*FeatureFlag
}

func NewFeatureFlagService() *FeatureFlagService {
    return &FeatureFlagService{flags: make(map[string]*FeatureFlag)}
}

func (s *FeatureFlagService) IsEnabled(flagName, userID string, userSegments []string) bool {
    s.mu.RLock(); defer s.mu.RUnlock()

    flag, ok := s.flags[flagName]
    if !ok || !flag.Enabled { return false }

    switch flag.Rollout {
    case RolloutAll:
        return true

    case RolloutPercentage:
        // Consistent hash — same user always gets same result
        hash := fnv32(userID + flagName)
        return float64(hash%100) < flag.Percentage

    case RolloutUserList:
        for _, id := range flag.AllowList {
            if id == userID { return true }
        }
        return false

    case RolloutUserSegment:
        for _, seg := range userSegments {
            for _, allowedSeg := range flag.Segments {
                if seg == allowedSeg { return true }
            }
        }
        return false
    }
    return false
```

```go
}

func (s *FeatureFlagService) SetFlag(flag *FeatureFlag) {
    s.mu.Lock(); defer s.mu.Unlock()
    flag.UpdatedAt = time.Now()
    s.flags[flag.Name] = flag
}

func fnv32(s string) uint32 {
    h := fnv.New32a()
    h.Write([]byte(s))
    return h.Sum32()
}
```

Interview tip: "Consistent hash for percentage rollout ensures the same user always gets the same experience — critical for A/B testing accuracy. Without it, users see flickering behaviour."

Q99

Design a money transfer system with transaction safety.

Difficulty: Hard

```go
type Account struct {
    ID       string
    OwnerID  string
    Balance  float64
    Currency string
    Version  int64
}

type Transaction struct {
    ID          string
    FromAccount string
    ToAccount   string
    Amount      float64
    Status      string // pending, completed, failed, reversed
    CreatedAt   time.Time
    CompletedAt *time.Time
}

type MoneyTransferService struct {
    mu       sync.Mutex
    accounts map[string]*Account
    txns     []*Transaction
}

func (s *MoneyTransferService) Transfer(fromID, toID string, amount float64) (*Transaction, error) {
    if amount <= 0 { return nil, errors.New("amount must be positive") }
    if fromID == toID { return nil, errors.New("cannot transfer to same account") }

    // Lock accounts in consistent order to prevent deadlock
    first, second := fromID, toID
    if first > second { first, second = second, first }

    s.mu.Lock()
    defer s.mu.Unlock()

    from, ok := s.accounts[fromID]
    if !ok { return nil, errors.New("source account not found") }

    to, ok := s.accounts[toID]
    if !ok { return nil, errors.New("destination account not found") }

    if from.Currency != to.Currency {
        return nil, errors.New("currency mismatch")
    }

    if from.Balance < amount {
        return nil, fmt.Errorf("insufficient funds: have %.2f, need %.2f", from.Balance, amount)
    }

    // Atomic transfer
    from.Balance -= amount
    to.Balance += amount
    from.Version++
```

```go
    to.Version++

    txn := &Transaction{
        ID:          uuid.New().String(),
        FromAccount: fromID,
        ToAccount:   toID,
        Amount:      amount,
        Status:      "completed",
        CreatedAt:   time.Now(),
    }
    now := time.Now()
    txn.CompletedAt = &now

    s.txns = append(s.txns, txn)
    return txn, nil
}

func (s *MoneyTransferService) GetBalance(accountID string) (float64, error) {
    s.mu.Lock(); defer s.mu.Unlock()
    acc, ok := s.accounts[accountID]
    if !ok { return 0, errors.New("account not found") }
    return acc.Balance, nil
}
```

Interview tip: "Lock ordering (sort IDs before locking) prevents deadlock between two concurrent reverse transfers. In SQL: use transactions with SELECT FOR UPDATE or serializable isolation."

Q100

Design a recommendation engine (collaborative filtering, LLD level).

Difficulty: Hard

```go
type Rating struct {
    UserID    string
    ItemID    string
    Score     float64 // 1-5
    Timestamp time.Time
}

type RecommendationEngine struct {
    mu      sync.RWMutex
    ratings map[string]map[string]float64 // userID → {itemID → score}
}

func NewRecommendationEngine() *RecommendationEngine {
    return &RecommendationEngine{
        ratings: make(map[string]map[string]float64),
    }
}

func (re *RecommendationEngine) AddRating(r Rating) {
    re.mu.Lock(); defer re.mu.Unlock()
    if re.ratings[r.UserID] == nil { re.ratings[r.UserID] = make(map[string]float64) }
    re.ratings[r.UserID][r.ItemID] = r.Score
}

// Cosine similarity between two users
func (re *RecommendationEngine) userSimilarity(userA, userB string) float64 {
    ratingsA := re.ratings[userA]
    ratingsB := re.ratings[userB]

    var dot, normA, normB float64
    for item, scoreA := range ratingsA {
        normA += scoreA * scoreA
        if scoreB, ok := ratingsB[item]; ok { dot += scoreA * scoreB }
    }
    for _, scoreB := range ratingsB { normB += scoreB * scoreB }

    if normA == 0 || normB == 0 { return 0 }
    return dot / (math.Sqrt(normA) * math.Sqrt(normB))
}

func (re *RecommendationEngine) Recommend(userID string, n int) []string {
    re.mu.RLock(); defer re.mu.RUnlock()

    userRatings := re.ratings[userID]
    scores := make(map[string]float64)
    weights := make(map[string]float64)

    for otherUser, otherRatings := range re.ratings {
        if otherUser == userID { continue }
        sim := re.userSimilarity(userID, otherUser)
        if sim <= 0 { continue }

        // Weighted sum of unrated items
```

```go
        for item, score := range otherRatings {
            if _, rated := userRatings[item]; !rated {
                scores[item] += sim * score
                weights[item] += sim
            }
        }
    }

    type recommendation struct{ item string; score float64 }
    var recs []recommendation
    for item, score := range scores {
        recs = append(recs, recommendation{item, score / weights[item]})
    }

    sort.Slice(recs, func(i, j int) bool { return recs[i].score > recs[j].score })

    result := make([]string, 0, n)
    for i, r := range recs {
        if i >= n { break }
        result = append(result, r.item)
    }
    return result
}
```

Interview tip: "This is O(users × items) — not scalable. Production: matrix factorisation (SVD), approximate nearest neighbours (FAISS), or deep learning embeddings. Always mention scalability limitations."

Quick Reference: Design Pattern Cheat Sheet

Pattern	Category	Problem Solved	Go Idiom
Singleton	Creational	One instance	sync.Once
Factory	Creational	Decouple creation	Interface + func
Builder	Creational	Complex construction	Method chaining
Functional Options	Creational (Go)	Optional config	...Option funcs
Adapter	Structural	Interface mismatch	Wrapper struct
Decorator	Structural	Add behaviour	Interface wrapper
Facade	Structural	Simplify subsystem	Simple API struct
Proxy	Structural	Control access	Wrapper + interface
Observer	Behavioural	Event notification	Channel / interface
Strategy	Behavioural	Interchangeable algos	Interface injection
Command	Behavioural	Encapsulate action	Interface + Execute/Undo

| State | Behavioural | State-dependent behaviour | Interface per state |

| Chain of Resp | Behavioural | Request pipeline | Linked handlers |

| Circuit Breaker | Resilience | Fail fast | State machine |

| Repository | Data Access | Abstract persistence | Interface |

Good luck with your SDE2 LLD interviews! Design → Code → Test in every answer. ■

LLD Additional Questions (Q101–Q150)

Q101

Design a thread-safe Singleton pattern in Go.

Difficulty: Medium

```go
// Singleton: only one instance of a type

// Using sync.Once (recommended, idiomatic Go)
type Config struct {
    DSN        string
    MaxConns   int
    Debug      bool
}

var (
    instance *Config
    once     sync.Once
)

func GetConfig() *Config {
    once.Do(func() {
        instance = &Config{
            DSN:      os.Getenv("DATABASE_URL"),
            MaxConns: 20,
            Debug:    os.Getenv("DEBUG") == "true",
        }
    })
    return instance
}

// Test-friendly singleton (allow reset)
type configManager struct {
    mu       sync.RWMutex
    instance *Config
}

var manager = &configManager{}

func (m *configManager) Get() *Config {
    m.mu.RLock()
    if m.instance != nil {
        defer m.mu.RUnlock()
        return m.instance
    }
    m.mu.RUnlock()

    m.mu.Lock()
    defer m.mu.Unlock()
    if m.instance == nil {
        m.instance = loadConfig()
    }
    return m.instance
}

func (m *configManager) Reset() {  // for testing
    m.mu.Lock()
    defer m.mu.Unlock()
    m.instance = nil
```

```go
}
```

Q102

Design an LRU (Least Recently Used) Cache in Go.

Difficulty: Hard

```go
// LRU Cache: O(1) get and put
// Data structures: doubly linked list + hash map

type LRUNode struct {
    key, value int
    prev, next *LRUNode
}

type LRUCache struct {
    capacity int
    cache    map[int]*LRUNode
    head     *LRUNode  // most recently used
    tail     *LRUNode  // least recently used
    mu       sync.RWMutex
}

func NewLRUCache(capacity int) *LRUCache {
    head := &LRUNode{}
    tail := &LRUNode{}
    head.next = tail
    tail.prev = head
    return &LRUCache{
        capacity: capacity,
        cache:    make(map[int]*LRUNode),
        head:     head,
        tail:     tail,
    }
}

func (c *LRUCache) Get(key int) (int, bool) {
    c.mu.Lock()
    defer c.mu.Unlock()
    if node, ok := c.cache[key]; ok {
        c.moveToFront(node)
        return node.value, true
    }
    return 0, false
}

func (c *LRUCache) Put(key, value int) {
    c.mu.Lock()
    defer c.mu.Unlock()
    if node, ok := c.cache[key]; ok {
        node.value = value
        c.moveToFront(node)
        return
    }
    if len(c.cache) >= c.capacity {
        lru := c.tail.prev
        c.removeNode(lru)
        delete(c.cache, lru.key)
    }
    node := &LRUNode{key: key, value: value}
```

```go
    c.cache[key] = node
    c.addToFront(node)
}

func (c *LRUCache) removeNode(n *LRUNode) {
    n.prev.next = n.next
    n.next.prev = n.prev
}

func (c *LRUCache) addToFront(n *LRUNode) {
    n.next = c.head.next
    n.prev = c.head
    c.head.next.prev = n
    c.head.next = n
}

func (c *LRUCache) moveToFront(n *LRUNode) {
    c.removeNode(n)
    c.addToFront(n)
}
```

Q103

Design a generic event bus in Go.

Difficulty: Hard

```go
type EventBus struct {
    mu          sync.RWMutex
    subscribers map[string][]chan interface{}
    done        chan struct{}
}

func NewEventBus() *EventBus {
    return &EventBus{
        subscribers: make(map[string][]chan interface{}),
        done:        make(chan struct{}),
    }
}

func (b *EventBus) Subscribe(topic string, bufSize int) <-chan interface{} {
    ch := make(chan interface{}, bufSize)
    b.mu.Lock()
    b.subscribers[topic] = append(b.subscribers[topic], ch)
    b.mu.Unlock()
    return ch
}

func (b *EventBus) Publish(topic string, payload interface{}) {
    b.mu.RLock()
    subs := make([]chan interface{}, len(b.subscribers[topic]))
    copy(subs, b.subscribers[topic])
    b.mu.RUnlock()

    for _, ch := range subs {
        select {
        case ch <- payload:
        case <-b.done:
            return
        default:
            // Drop if subscriber is slow (non-blocking)
        }
    }
}

func (b *EventBus) Unsubscribe(topic string, ch <-chan interface{}) {
    b.mu.Lock()
    defer b.mu.Unlock()
    subs := b.subscribers[topic]
    for i, s := range subs {
        if s == ch {
            close(s)
            b.subscribers[topic] = append(subs[:i], subs[i+1:]...)
            return
        }
    }
}

func (b *EventBus) Close() {
    close(b.done)
```

```go
    b.mu.Lock()
    defer b.mu.Unlock()
    for _, subs := range b.subscribers {
        for _, ch := range subs { close(ch) }
    }
}
```

Q104

Design a Token Bucket rate limiter.

Difficulty: Hard

```go
type TokenBucket struct {
    mu         sync.Mutex
    tokens     float64
    maxTokens  float64
    refillRate float64  // tokens per second
    lastRefill time.Time
}

func NewTokenBucket(maxTokens, refillRate float64) *TokenBucket {
    return &TokenBucket{
        tokens:     maxTokens,
        maxTokens:  maxTokens,
        refillRate: refillRate,
        lastRefill: time.Now(),
    }
}

func (tb *TokenBucket) Allow() bool {
    return tb.AllowN(1)
}

func (tb *TokenBucket) AllowN(n float64) bool {
    tb.mu.Lock()
    defer tb.mu.Unlock()

    now := time.Now()
    elapsed := now.Sub(tb.lastRefill).Seconds()
    tb.tokens = math.Min(tb.maxTokens, tb.tokens + elapsed*tb.refillRate)
    tb.lastRefill = now

    if tb.tokens >= n {
        tb.tokens -= n
        return true
    }
    return false
}

func (tb *TokenBucket) Wait(ctx context.Context) error {
    for {
        if tb.Allow() { return nil }
        select {
        case <-ctx.Done(): return ctx.Err()
        case <-time.After(10 * time.Millisecond):
        }
    }
}

// Per-user rate limiting
type RateLimiterMap struct {
    mu       sync.RWMutex
    buckets  map[string]*TokenBucket
    rate     float64
    capacity float64
```

```go
}

func (m *RateLimiterMap) Allow(userID string) bool {
    m.mu.RLock()
    if b, ok := m.buckets[userID]; ok {
        m.mu.RUnlock()
        return b.Allow()
    }
    m.mu.RUnlock()

    m.mu.Lock()
    defer m.mu.Unlock()
    if _, ok := m.buckets[userID]; !ok {
        m.buckets[userID] = NewTokenBucket(m.capacity, m.rate)
    }
    return m.buckets[userID].Allow()
}
```

Q105

Design a connection pool in Go.

Difficulty: Hard

```go
type Connection interface {
    Close() error
    IsAlive() bool
}

type Pool struct {
    mu          sync.Mutex
    conns       chan Connection
    factory     func() (Connection, error)
    maxSize     int
    currentSize int
    closed      bool
}

func NewPool(factory func() (Connection, error), maxSize int) *Pool {
    return &Pool{
        conns:   make(chan Connection, maxSize),
        factory: factory,
        maxSize: maxSize,
    }
}

func (p *Pool) Get(ctx context.Context) (Connection, error) {
    select {
    case conn := <-p.conns:
        if conn.IsAlive() {
            return conn, nil
        }
        // Stale connection: create new
        p.mu.Lock()
        p.currentSize--
        p.mu.Unlock()
        return p.newConn()
    default:
        return p.newConn()
    }
}

func (p *Pool) newConn() (Connection, error) {
    p.mu.Lock()
    defer p.mu.Unlock()
    if p.currentSize >= p.maxSize {
        return nil, errors.New("pool exhausted")
    }
    conn, err := p.factory()
    if err != nil { return nil, err }
    p.currentSize++
    return conn, nil
}

func (p *Pool) Put(conn Connection) {
    if !conn.IsAlive() {
        p.mu.Lock()
```

```go
        p.currentSize--
        p.mu.Unlock()
        conn.Close()
        return
    }
    select {
    case p.conns <- conn:  // return to pool
    default:
        // Pool full: close connection
        p.mu.Lock()
        p.currentSize--
        p.mu.Unlock()
        conn.Close()
    }
}

func (p *Pool) Close() {
    p.mu.Lock()
    defer p.mu.Unlock()
    p.closed = true
    close(p.conns)
    for conn := range p.conns {
        conn.Close()
    }
}
```

Q106

Design a Trie data structure in Go.

Difficulty: Hard

```go
type TrieNode struct {
    children [26]*TrieNode
    isEnd    bool
    word     string
}

type Trie struct {
    root *TrieNode
}

func NewTrie() *Trie {
    return &Trie{root: &TrieNode{}}
}

func (t *Trie) Insert(word string) {
    node := t.root
    for _, ch := range word {
        idx := ch - 'a'
        if node.children[idx] == nil {
            node.children[idx] = &TrieNode{}
        }
        node = node.children[idx]
    }
    node.isEnd = true
    node.word = word
}

func (t *Trie) Search(word string) bool {
    node := t.find(word)
    return node != nil && node.isEnd
}

func (t *Trie) StartsWith(prefix string) bool {
    return t.find(prefix) != nil
}

func (t *Trie) find(prefix string) *TrieNode {
    node := t.root
    for _, ch := range prefix {
        idx := ch - 'a'
        if node.children[idx] == nil { return nil }
        node = node.children[idx]
    }
    return node
}

// Autocomplete: find all words with given prefix
func (t *Trie) AutoComplete(prefix string) []string {
    node := t.find(prefix)
    if node == nil { return nil }

    var results []string
    var dfs func(*TrieNode)
```

```go
    dfs = func(n *TrieNode) {
        if n.isEnd { results = append(results, n.word) }
        for _, child := range n.children {
            if child != nil { dfs(child) }
        }
    }
    dfs(node)
    return results
}

// Concurrent-safe Trie with read-write lock
type ConcurrentTrie struct {
    mu   sync.RWMutex
    trie *Trie
}
```

Q107

Design a concurrent job queue with priority.

Difficulty: Hard

```go
type Priority int

const (
    High   Priority = 0
    Medium Priority = 1
    Low    Priority = 2
)

type Job struct {
    ID       string
    Priority Priority
    Payload  interface{}
    Handler  func(interface{}) error
}

type PriorityQueue struct {
    queues  [3]chan Job  // High, Medium, Low
    workers int
    wg      sync.WaitGroup
    done    chan struct{}
}

func NewPriorityQueue(workers, bufSize int) *PriorityQueue {
    pq := &PriorityQueue{
        workers: workers,
        done:    make(chan struct{}),
    }
    for i := range pq.queues {
        pq.queues[i] = make(chan Job, bufSize)
    }
    pq.start()
    return pq
}

func (pq *PriorityQueue) start() {
    for i := 0; i < pq.workers; i++ {
        pq.wg.Add(1)
        go pq.worker()
    }
}

func (pq *PriorityQueue) worker() {
    defer pq.wg.Done()
    for {
        select {
        case job := <-pq.queues[High]:
            job.Handler(job.Payload)
        default:
            select {
            case job := <-pq.queues[High]:
                job.Handler(job.Payload)
            case job := <-pq.queues[Medium]:
                job.Handler(job.Payload)
```

```go
            default:
                select {
                case job := <-pq.queues[High]:
                    job.Handler(job.Payload)
                case job := <-pq.queues[Medium]:
                    job.Handler(job.Payload)
                case job := <-pq.queues[Low]:
                    job.Handler(job.Payload)
                case <-pq.done:
                    return
                }
            }
        }
    }
}

func (pq *PriorityQueue) Submit(job Job) {
    pq.queues[job.Priority] <- job
}
```

Q108

Design a publish-subscribe system with filtering.

Difficulty: Hard

```go
type Predicate func(interface{}) bool

type Subscription struct {
    id        string
    topic     string
    filter    Predicate
    ch        chan interface{}
    done      chan struct{}
}

type FilteredPubSub struct {
    mu            sync.RWMutex
    subscriptions map[string][]*Subscription  // topic → subs
}

func (ps *FilteredPubSub) Subscribe(topic string, filter Predicate) *Subscription {
    sub := &Subscription{
        id:     uuid.New().String(),
        topic:  topic,
        filter: filter,
        ch:     make(chan interface{}, 100),
        done:   make(chan struct{}),
    }
    ps.mu.Lock()
    ps.subscriptions[topic] = append(ps.subscriptions[topic], sub)
    ps.mu.Unlock()
    return sub
}

func (ps *FilteredPubSub) Publish(topic string, msg interface{}) {
    ps.mu.RLock()
    subs := ps.subscriptions[topic]
    ps.mu.RUnlock()

    for _, sub := range subs {
        if sub.filter == nil || sub.filter(msg) {
            select {
            case sub.ch <- msg:
            case <-sub.done:
            default:
                // Slow subscriber: drop or log
            }
        }
    }
}

func (sub *Subscription) Messages() <-chan interface{} { return sub.ch }
func (sub *Subscription) Close() { close(sub.done); close(sub.ch) }

// Usage: subscribe to orders with amount > 100
sub := ps.Subscribe("orders", func(msg interface{}) bool {
    order := msg.(Order)
    return order.Amount > 100
```

```go
})
```

Q109

Design a circuit breaker in Go.

Difficulty: Hard

```go
type State int

const (
    StateClosed   State = iota  // normal: pass through
    StateOpen                   // failing: reject all
    StateHalfOpen               // testing: allow some through
)

type CircuitBreaker struct {
    mu               sync.Mutex
    state            State
    failures         int
    successes        int
    maxFailures      int
    successThreshold int
    timeout          time.Duration
    lastFailureTime  time.Time
}

func NewCircuitBreaker(maxFailures, successThreshold int, timeout time.Duration) *CircuitBreaker {
    return &CircuitBreaker{
        state:            StateClosed,
        maxFailures:      maxFailures,
        successThreshold: successThreshold,
        timeout:          timeout,
    }
}

func (cb *CircuitBreaker) Execute(fn func() error) error {
    cb.mu.Lock()
    switch cb.state {
    case StateOpen:
        if time.Since(cb.lastFailureTime) > cb.timeout {
            cb.state = StateHalfOpen
            cb.successes = 0
        } else {
            cb.mu.Unlock()
            return ErrCircuitOpen
        }
    case StateHalfOpen:
        // Allow through (testing if service recovered)
    }
    cb.mu.Unlock()

    err := fn()

    cb.mu.Lock()
    defer cb.mu.Unlock()

    if err != nil {
        cb.failures++
        cb.lastFailureTime = time.Now()
        if cb.state == StateHalfOpen || cb.failures >= cb.maxFailures {
```

```go
            cb.state = StateOpen
            cb.failures = 0
        }
        return err
    }

    // Success
    if cb.state == StateHalfOpen {
        cb.successes++
        if cb.successes >= cb.successThreshold {
            cb.state = StateClosed
            cb.failures = 0
        }
    } else {
        cb.failures = 0
    }
    return nil
}

var ErrCircuitOpen = errors.New("circuit breaker is open")
```

Q110

Design a retry mechanism with exponential backoff.

Difficulty: Medium

```go
type RetryConfig struct {
    MaxAttempts int
    InitialWait time.Duration
    MaxWait     time.Duration
    Multiplier  float64
    Jitter      bool
    RetryIf     func(error) bool  // nil = retry all errors
}

func Retry(ctx context.Context, cfg RetryConfig, fn func() error) error {
    var err error
    wait := cfg.InitialWait

    for attempt := 0; attempt < cfg.MaxAttempts; attempt++ {
        if err = fn(); err == nil {
            return nil
        }

        if cfg.RetryIf != nil && !cfg.RetryIf(err) {
            return err  // non-retryable error
        }

        if attempt == cfg.MaxAttempts-1 {
            break  // last attempt, don't wait
        }

        // Calculate wait with jitter
        actualWait := wait
        if cfg.Jitter {
            jitter := time.Duration(rand.Int63n(int64(wait / 2)))
            actualWait = wait + jitter
        }
        actualWait = min(actualWait, cfg.MaxWait)

        select {
        case <-ctx.Done():
            return fmt.Errorf("context cancelled after %d attempts: %w", attempt+1, ctx.Err())
        case <-time.After(actualWait):
        }

        wait = time.Duration(float64(wait) * cfg.Multiplier)
    }

    return fmt.Errorf("failed after %d attempts: %w", cfg.MaxAttempts, err)
}

// Usage
err := Retry(ctx, RetryConfig{
    MaxAttempts: 5,
    InitialWait: 100 * time.Millisecond,
    MaxWait:     30 * time.Second,
    Multiplier:  2.0,
    Jitter:      true,
```

```go
    RetryIf:     func(err error) bool { return errors.Is(err, ErrTemporary) },
}, func() error {
    return callExternalService()
})
```

Q111

Design a Bloom Filter in Go.

Difficulty: Hard

```go
import (
    "math"
    "github.com/spaolacci/murmur3"
)

type BloomFilter struct {
    bits    []bool
    k       int    // number of hash functions
    m       int    // bit array size
    n       int    // approximate items added
}

func NewBloomFilter(expectedItems int, falsePositiveRate float64) *BloomFilter {
    m := optimalM(expectedItems, falsePositiveRate)
    k := optimalK(m, expectedItems)
    return &BloomFilter{
        bits: make([]bool, m),
        k:    k, m: m,
    }
}

func optimalM(n int, p float64) int {
    return int(-float64(n) * math.Log(p) / (math.Log(2) * math.Log(2)))
}

func optimalK(m, n int) int {
    return int(float64(m) / float64(n) * math.Log(2))
}

func (bf *BloomFilter) hashes(item []byte) []int {
    positions := make([]int, bf.k)
    h1 := murmur3.Sum32WithSeed(item, 0)
    h2 := murmur3.Sum32WithSeed(item, h1)
    for i := 0; i < bf.k; i++ {
        positions[i] = int((uint64(h1) + uint64(i)*uint64(h2)) % uint64(bf.m))
    }
    return positions
}

func (bf *BloomFilter) Add(item []byte) {
    for _, pos := range bf.hashes(item) {
        bf.bits[pos] = true
    }
    bf.n++
}

func (bf *BloomFilter) MightContain(item []byte) bool {
    for _, pos := range bf.hashes(item) {
        if !bf.bits[pos] { return false }
    }
    return true
}
```

```go
// Usage
bf := NewBloomFilter(1000000, 0.01)  // 1M items, 1% FP rate
bf.Add([]byte("user:42"))
if bf.MightContain([]byte("user:42")) {
    // check DB to confirm (false positives possible)
}
```

Q112

Design a bounded semaphore.

```go
type Semaphore struct {
    ch chan struct{}
}

func NewSemaphore(n int) *Semaphore {
    return &Semaphore{ch: make(chan struct{}, n)}
}

func (s *Semaphore) Acquire(ctx context.Context) error {
    select {
    case s.ch <- struct{}{}:
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}

func (s *Semaphore) Release() { <-s.ch }

func (s *Semaphore) TryAcquire() bool {
    select {
    case s.ch <- struct{}{}:
        return true
    default:
        return false
    }
}
```

Q113

Design an in-memory key-value store with TTL.

```go
type entry struct {
    value     interface{}
    expiresAt time.Time
}

type TTLCache struct {
    mu      sync.RWMutex
    items   map[string]entry
    closeCh chan struct{}
}

func NewTTLCache(cleanupInterval time.Duration) *TTLCache {
    c := &TTLCache{
        items:   make(map[string]entry),
        closeCh: make(chan struct{}),
    }
    go c.cleanup(cleanupInterval)
    return c
}

func (c *TTLCache) Set(key string, val interface{}, ttl time.Duration) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.items[key] = entry{value: val, expiresAt: time.Now().Add(ttl)}
}

func (c *TTLCache) Get(key string) (interface{}, bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()
    e, ok := c.items[key]
    if !ok || time.Now().After(e.expiresAt) { return nil, false }
    return e.value, true
}

func (c *TTLCache) cleanup(interval time.Duration) {
    ticker := time.NewTicker(interval)
    defer ticker.Stop()
    for {
        select {
        case <-ticker.C:
            c.mu.Lock()
            for k, e := range c.items {
                if time.Now().After(e.expiresAt) { delete(c.items, k) }
            }
            c.mu.Unlock()
        case <-c.closeCh: return
        }
    }
}
```

Q114

Design a sliding window rate limiter.

```go
type SlidingWindowLimiter struct {
    mu         sync.Mutex
    timestamps []time.Time
    limit      int
    window     time.Duration
}

func (l *SlidingWindowLimiter) Allow() bool {
    l.mu.Lock()
    defer l.mu.Unlock()

    now := time.Now()
    cutoff := now.Add(-l.window)

    // Remove timestamps outside window
    i := 0
    for ; i < len(l.timestamps); i++ {
        if l.timestamps[i].After(cutoff) { break }
    }
    l.timestamps = l.timestamps[i:]

    if len(l.timestamps) >= l.limit { return false }
    l.timestamps = append(l.timestamps, now)
    return true
}
```

Q115

Design a concurrent map with shard locking.

```go
const shardCount = 32

type ConcurrentMap[K comparable, V any] struct {
    shards [shardCount]shard[K, V]
}

type shard[K comparable, V any] struct {
    mu   sync.RWMutex
    data map[K]V
}

func (m *ConcurrentMap[K, V]) getShard(key K) *shard[K, V] {
    hash := fnv32(fmt.Sprint(key))
    return &m.shards[hash%shardCount]
}

func (m *ConcurrentMap[K, V]) Set(key K, val V) {
    s := m.getShard(key)
    s.mu.Lock()
    defer s.mu.Unlock()
    s.data[key] = val
}

func (m *ConcurrentMap[K, V]) Get(key K) (V, bool) {
    s := m.getShard(key)
    s.mu.RLock()
    defer s.mu.RUnlock()
    v, ok := s.data[key]
    return v, ok
}
```

Q116

Design a file-based config with hot reload.

```go
type Config struct {
    mu   sync.RWMutex
    data map[string]interface{}
    path string
}

func (c *Config) Watch(ctx context.Context) error {
    watcher, _ := fsnotify.NewWatcher()
    watcher.Add(c.path)

    for {
        select {
        case event := <-watcher.Events:
            if event.Op&fsnotify.Write != 0 {
                c.reload()
            }
        case <-ctx.Done():
            return nil
        }
    }
}

func (c *Config) reload() {
    data, err := os.ReadFile(c.path)
    if err != nil { return }

    var newData map[string]interface{}
    json.Unmarshal(data, &newData)

    c.mu.Lock()
    c.data = newData
    c.mu.Unlock()
    log.Printf("config reloaded")
}
```

Q117

Design a worker pool with graceful shutdown.

```go
type WorkerPool struct {
    jobs    chan func()
    wg      sync.WaitGroup
    once    sync.Once
    done    chan struct{}
}

func NewWorkerPool(workers, queueSize int) *WorkerPool {
    p := &WorkerPool{
        jobs: make(chan func(), queueSize),
        done: make(chan struct{}),
    }
    for i := 0; i < workers; i++ {
        p.wg.Add(1)
        go p.worker()
    }
    return p
}

func (p *WorkerPool) worker() {
    defer p.wg.Done()
    for fn := range p.jobs { fn() }
}

func (p *WorkerPool) Submit(fn func()) bool {
    select {
    case p.jobs <- fn:
        return true
    case <-p.done:
        return false
    }
}

func (p *WorkerPool) Shutdown(ctx context.Context) error {
    p.once.Do(func() {
        close(p.done)
        close(p.jobs)
    })

    done := make(chan struct{})
    go func() { p.wg.Wait(); close(done) }()

    select {
    case <-done: return nil
    case <-ctx.Done(): return ctx.Err()
    }
}
```

Q118

Design a middleware chain pattern.

```go
type HandlerFunc func(ctx context.Context, req interface{}) (interface{}, error)
type MiddlewareFunc func(HandlerFunc) HandlerFunc

func Chain(handler HandlerFunc, middlewares ...MiddlewareFunc) HandlerFunc {
    for i := len(middlewares) - 1; i >= 0; i-- {
        handler = middlewares[i](handler)
    }
    return handler
}

// Logging middleware
func LoggingMiddleware(next HandlerFunc) HandlerFunc {
    return func(ctx context.Context, req interface{}) (interface{}, error) {
        start := time.Now()
        resp, err := next(ctx, req)
        log.Printf("request completed in %v, error=%v", time.Since(start), err)
        return resp, err
    }
}

// Usage
handler := Chain(myHandler,
    LoggingMiddleware,
    AuthMiddleware,
    RateLimitMiddleware,
)
```

Q119

Design a generic pipeline in Go.

```go
func Pipeline[T, U any](
    ctx context.Context,
    input <-chan T,
    process func(T) (U, error),
) (<-chan U, <-chan error) {
    output := make(chan U)
    errs := make(chan error, 1)

    go func() {
        defer close(output)
        defer close(errs)

        for v := range input {
            result, err := process(v)
            if err != nil { errs <- err; return }
            select {
            case output <- result:
            case <-ctx.Done():
                errs <- ctx.Err()
                return
            }
        }
    }()

    return output, errs
}
```

Q120

Design a distributed ID generator (Snowflake).

```go
// Snowflake ID: 64-bit unique, time-sortable, no coordination
// Layout: [41 bits timestamp][10 bits machine ID][12 bits sequence]

type Snowflake struct {
    mu        sync.Mutex
    epoch     int64  // custom epoch (ms)
    machineID int64  // 0-1023
    sequence  int64  // 0-4095
    lastTime  int64
}

const (
    workerBits  = 10
    sequenceBits = 12
    maxSequence = 1<<sequenceBits - 1  // 4095
)

func (s *Snowflake) Next() (int64, error) {
    s.mu.Lock()
    defer s.mu.Unlock()

    now := time.Now().UnixMilli() - s.epoch

    if now == s.lastTime {
        s.sequence = (s.sequence + 1) & maxSequence
        if s.sequence == 0 {
            for now <= s.lastTime {
                now = time.Now().UnixMilli() - s.epoch
            }
        }
    } else {
        s.sequence = 0
    }
    s.lastTime = now

    id := (now << (workerBits + sequenceBits)) |
          (s.machineID << sequenceBits) |
          s.sequence
    return id, nil
}
```

Q121–Q150: Additional LLD Topics

| Q | Topic |

|---|---|

| Q121 | Design Observer pattern with generics |

| Q122 | Design Decorator pattern for HTTP handlers |

| Q123 | Design State machine for order workflow |

| Q124 | Design Command pattern with undo/redo |

| Q125 | Design Composite pattern for file system |

| Q126 | Design Proxy pattern for lazy loading |

| Q127 | Design Flyweight pattern for character rendering |

| Q128 | Design Template Method for data export |

| Q129 | Design Chain of Responsibility for validation |

| Q130 | Design Iterator pattern for custom collections |

| Q131 | Design Visitor pattern for AST traversal |

| Q132 | Design Mediator pattern for chat room |

| Q133 | Design Interpreter pattern for query parser |

| Q134 | Design Memento pattern for game save state |

| Q135 | Design generic Result type (Ok/Err) |

| Q136 | Design functional options vs config struct |

| Q137 | Design interface segregation for user repository |

| Q138 | Design dependency injection container |

| Q139 | Design event-driven order processing |

| Q140 | Design idempotent API handler |

| Q141 | Design health check aggregator |

| Q142 | Design graceful shutdown handler |

| Q143 | Design distributed tracing context propagation |

| Q144 | Design metric collector with sampling |

| Q145 | Design structured logger with hooks |

| Q146 | Design feature flag evaluator |

| Q147 | Design API response pagination helper |

| Q148 | Design multi-level fallback chain |

| Q149 | LLD interview: parking lot system |

| Q150 | LLD interview: elevator controller |

Extended Questions (Q122–Q150)

Q122

Design a thread-safe LRU cache in Go.

Difficulty: Hard

```go
type LRUCache struct {
    capacity int
    mu       sync.RWMutex
    cache    map[int]*list.Element
    list     *list.List
}

type entry struct {
    key   int
    value int
}

func NewLRUCache(capacity int) *LRUCache {
    return &LRUCache{
        capacity: capacity,
        cache:    make(map[int]*list.Element),
        list:     list.New(),
    }
}

func (c *LRUCache) Get(key int) (int, bool) {
    c.mu.Lock()
    defer c.mu.Unlock()

    if elem, ok := c.cache[key]; ok {
        c.list.MoveToFront(elem)
        return elem.Value.(*entry).value, true
    }
    return 0, false
}

func (c *LRUCache) Put(key, value int) {
    c.mu.Lock()
    defer c.mu.Unlock()

    if elem, ok := c.cache[key]; ok {
        c.list.MoveToFront(elem)
        elem.Value.(*entry).value = value
        return
    }

    if c.list.Len() == c.capacity {
        // Evict LRU (back of list)
        back := c.list.Back()
        c.list.Remove(back)
        delete(c.cache, back.Value.(*entry).key)
    }

    elem := c.list.PushFront(&entry{key, value})
    c.cache[key] = elem
}

// Time: O(1) for Get and Put
```

```go
// Space: O(capacity)
```

Q123

Design a pub/sub event bus in Go.

Difficulty: Hard

```go
type EventBus struct {
    mu          sync.RWMutex
    subscribers map[string][]chan interface{}
    closed      bool
}

func NewEventBus() *EventBus {
    return &EventBus{
        subscribers: make(map[string][]chan interface{}),
    }
}

func (b *EventBus) Subscribe(topic string, bufferSize int) (<-chan interface{}, func()) {
    ch := make(chan interface{}, bufferSize)

    b.mu.Lock()
    b.subscribers[topic] = append(b.subscribers[topic], ch)
    b.mu.Unlock()

    // Return unsubscribe function
    unsubscribe := func() {
        b.mu.Lock()
        defer b.mu.Unlock()
        subs := b.subscribers[topic]
        for i, s := range subs {
            if s == ch {
                b.subscribers[topic] = append(subs[:i], subs[i+1:]...)
                close(ch)
                return
            }
        }
    }

    return ch, unsubscribe
}

func (b *EventBus) Publish(topic string, event interface{}) {
    b.mu.RLock()
    subs := b.subscribers[topic]
    b.mu.RUnlock()

    for _, ch := range subs {
        select {
        case ch <- event:
        default:
            // Slow subscriber: drop message (or log)
        }
    }
}

func (b *EventBus) Close() {
    b.mu.Lock()
    defer b.mu.Unlock()
```

```go
    for _, subs := range b.subscribers {
        for _, ch := range subs {
            close(ch)
        }
    }
    b.subscribers = make(map[string][]chan interface{})
}
```

Q124

Design a connection pool in Go.

Difficulty: Hard

```go
type Pool struct {
    mu          sync.Mutex
    conns       chan net.Conn
    factory     func() (net.Conn, error)
    maxOpen     int
    numOpen     int
    maxIdleTime time.Duration
}

func NewPool(maxOpen int, maxIdleTime time.Duration, factory func() (net.Conn, error)) *Pool {
    return &Pool{
        conns:       make(chan net.Conn, maxOpen),
        factory:     factory,
        maxOpen:     maxOpen,
        maxIdleTime: maxIdleTime,
    }
}

func (p *Pool) Get(ctx context.Context) (net.Conn, error) {
    select {
    case conn := <-p.conns:
        return conn, nil  // reuse idle connection
    default:
    }

    p.mu.Lock()
    if p.numOpen < p.maxOpen {
        p.numOpen++
        p.mu.Unlock()
        conn, err := p.factory()
        if err != nil {
            p.mu.Lock(); p.numOpen--; p.mu.Unlock()
            return nil, err
        }
        return conn, nil
    }
    p.mu.Unlock()

    // Wait for available connection
    select {
    case conn := <-p.conns:
        return conn, nil
    case <-ctx.Done():
        return nil, ctx.Err()
    }
}

func (p *Pool) Put(conn net.Conn) {
    select {
    case p.conns <- conn:
    default:
        conn.Close()
        p.mu.Lock(); p.numOpen--; p.mu.Unlock()
```

```go
    }
}

func (p *Pool) Close() {
    close(p.conns)
    for conn := range p.conns {
        conn.Close()
    }
}
```

Q125

Design a task scheduler in Go.

Difficulty: Hard

```go
type Task struct {
    ID       string
    RunAt    time.Time
    Fn       func(ctx context.Context) error
    Interval time.Duration  // 0 = one-time
}

type Scheduler struct {
    mu     sync.Mutex
    tasks  map[string]*Task
    queue  *PriorityQueue  // min-heap by RunAt
    ctx    context.Context
    cancel context.CancelFunc
}

func (s *Scheduler) Schedule(task *Task) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.tasks[task.ID] = task
    heap.Push(s.queue, task)
}

func (s *Scheduler) Run() {
    timer := time.NewTimer(time.Hour)
    defer timer.Stop()

    for {
        s.mu.Lock()
        var d time.Duration
        if s.queue.Len() > 0 {
            next := (*s.queue)[0].(*Task)
            d = time.Until(next.RunAt)
            if d <= 0 {
                task := heap.Pop(s.queue).(*Task)
                s.mu.Unlock()

                go func() {
                    if err := task.Fn(s.ctx); err != nil {
                        log.Printf("task %s failed: %v", task.ID, err)
                    }
                    if task.Interval > 0 {
                        task.RunAt = time.Now().Add(task.Interval)
                        s.Schedule(task)
                    }
                }()
                continue
            }
        } else {
            d = time.Hour
        }
        s.mu.Unlock()

        timer.Reset(d)
```

```go
        select {
        case <-s.ctx.Done(): return
        case <-timer.C:
        }
    }
}
```

Q126

Design a semaphore using channels in Go.

Difficulty: Medium

```go
// Semaphore: limit concurrent operations

type Semaphore struct {
    ch chan struct{}
}

func NewSemaphore(n int) *Semaphore {
    return &Semaphore{ch: make(chan struct{}, n)}
}

func (s *Semaphore) Acquire(ctx context.Context) error {
    select {
    case s.ch <- struct{}{}:
        return nil
    case <-ctx.Done():
        return ctx.Err()
    }
}

func (s *Semaphore) Release() {
    <-s.ch
}

func (s *Semaphore) TryAcquire() bool {
    select {
    case s.ch <- struct{}{}:
        return true
    default:
        return false
    }
}

// Usage: limit concurrent HTTP requests
sem := NewSemaphore(10)

var wg sync.WaitGroup
for _, url := range urls {
    wg.Add(1)
    if err := sem.Acquire(ctx); err != nil {
        wg.Done(); break
    }
    go func(u string) {
        defer wg.Done()
        defer sem.Release()
        fetch(u)
    }(url)
}
wg.Wait()
```

Q127

Design a circuit breaker in Go.

Difficulty: Hard

```go
type State int
const (
    Closed   State = iota  // normal, allows calls
    Open                   // tripped, blocks calls
    HalfOpen               // testing recovery
)

type CircuitBreaker struct {
    mu              sync.Mutex
    state           State
    failures        int
    successes       int
    maxFailures     int
    timeout         time.Duration
    lastOpenTime    time.Time
    maxHalfOpenReqs int
    halfOpenReqs    int
}

func (cb *CircuitBreaker) Execute(fn func() error) error {
    cb.mu.Lock()
    switch cb.state {
    case Open:
        if time.Since(cb.lastOpenTime) > cb.timeout {
            cb.state = HalfOpen
            cb.halfOpenReqs = 0
        } else {
            cb.mu.Unlock()
            return ErrCircuitOpen
        }
    case HalfOpen:
        if cb.halfOpenReqs >= cb.maxHalfOpenReqs {
            cb.mu.Unlock()
            return ErrCircuitOpen
        }
        cb.halfOpenReqs++
    }
    cb.mu.Unlock()

    err := fn()

    cb.mu.Lock()
    defer cb.mu.Unlock()

    if err != nil {
        cb.failures++
        if cb.state == HalfOpen || cb.failures >= cb.maxFailures {
            cb.state = Open
            cb.lastOpenTime = time.Now()
            cb.failures = 0
        }
        return err
    }
```

```go
    if cb.state == HalfOpen {
        cb.successes++
        if cb.successes >= cb.maxHalfOpenReqs {
            cb.state = Closed
            cb.failures = 0
            cb.successes = 0
        }
    } else {
        cb.failures = 0
    }
    return nil
}
```

Q128

Design an in-memory key-value store with TTL.

Difficulty: Hard

```go
type KVStore struct {
    mu      sync.RWMutex
    data    map[string]item
    done    chan struct{}
}

type item struct {
    value     interface{}
    expiresAt time.Time
    hasExpiry bool
}

func NewKVStore() *KVStore {
    s := &KVStore{
        data: make(map[string]item),
        done: make(chan struct{}),
    }
    go s.cleanupLoop()
    return s
}

func (s *KVStore) Set(key string, value interface{}, ttl time.Duration) {
    s.mu.Lock()
    defer s.mu.Unlock()
    i := item{value: value}
    if ttl > 0 {
        i.hasExpiry = true
        i.expiresAt = time.Now().Add(ttl)
    }
    s.data[key] = i
}

func (s *KVStore) Get(key string) (interface{}, bool) {
    s.mu.RLock()
    defer s.mu.RUnlock()
    i, ok := s.data[key]
    if !ok { return nil, false }
    if i.hasExpiry && time.Now().After(i.expiresAt) {
        return nil, false  // expired (lazy deletion)
    }
    return i.value, true
}

func (s *KVStore) cleanupLoop() {
    ticker := time.NewTicker(time.Minute)
    defer ticker.Stop()
    for {
        select {
        case <-s.done: return
        case <-ticker.C:
            s.mu.Lock()
            now := time.Now()
            for k, i := range s.data {
```

```go
                if i.hasExpiry && now.After(i.expiresAt) {
                    delete(s.data, k)
                }
            }
            s.mu.Unlock()
        }
    }
}
```

Q129

Design a retry mechanism with jitter in Go.

Difficulty: Medium

```go
type RetryConfig struct {
    MaxAttempts int
    InitialDelay time.Duration
    MaxDelay     time.Duration
    Multiplier   float64
    Jitter       bool
}

func Retry(ctx context.Context, cfg RetryConfig, fn func() error) error {
    delay := cfg.InitialDelay

    for attempt := 1; attempt <= cfg.MaxAttempts; attempt++ {
        err := fn()
        if err == nil { return nil }

        if attempt == cfg.MaxAttempts { return err }

        // Exponential backoff
        wait := delay
        if cfg.Jitter {
            // Full jitter: random between 0 and delay
            wait = time.Duration(rand.Int63n(int64(delay)))
        }

        select {
        case <-ctx.Done(): return ctx.Err()
        case <-time.After(wait):
        }

        // Increase delay for next attempt
        delay = time.Duration(float64(delay) * cfg.Multiplier)
        if delay > cfg.MaxDelay { delay = cfg.MaxDelay }
    }
    return nil
}

// Usage
err := Retry(ctx, RetryConfig{
    MaxAttempts:  5,
    InitialDelay: 100 * time.Millisecond,
    MaxDelay:     10 * time.Second,
    Multiplier:   2.0,
    Jitter:       true,
}, func() error {
    return callExternalService()
})
```

Q130

Design a concurrent map reduce in Go.

Difficulty: Hard

```go
func MapReduce[T, U, V any](
    ctx context.Context,
    input []T,
    mapFn func(T) U,
    reduceFn func(V, U) V,
    initialValue V,
    workers int,
) V {
    // Map phase: parallel
    mapped := make(chan U, len(input))

    g, ctx := errgroup.WithContext(ctx)
    sem := make(chan struct{}, workers)

    for _, item := range input {
        item := item
        g.Go(func() error {
            sem <- struct{}{}
            defer func() { <-sem }()
            select {
            case <-ctx.Done(): return ctx.Err()
            default:
                mapped <- mapFn(item)
                return nil
            }
        })
    }

    go func() {
        g.Wait()
        close(mapped)
    }()

    // Reduce phase: sequential (for determinism)
    result := initialValue
    for u := range mapped {
        result = reduceFn(result, u)
    }
    return result
}

// Usage: count words
wordCount := MapReduce(ctx, documents,
    func(doc string) map[string]int {
        counts := make(map[string]int)
        for _, w := range strings.Fields(doc) { counts[w]++ }
        return counts
    },
    func(acc, m map[string]int) map[string]int {
        for k, v := range m { acc[k] += v }
        return acc
    },
    make(map[string]int),
```

```go
    runtime.NumCPU(),
)
```

Q131–Q150: Final LLD Questions

Q131

Design a generic Stack with generics.

```go
type Stack[T any] struct {
    items []T
}
func (s *Stack[T]) Push(v T)       { s.items = append(s.items, v) }
func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 { return zero, false }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}
func (s *Stack[T]) Peek() (T, bool) {
    var zero T
    if len(s.items) == 0 { return zero, false }
    return s.items[len(s.items)-1], true
}
func (s *Stack[T]) Size() int { return len(s.items) }
```

Q132

Design a middleware pipeline for HTTP.

```go
type Middleware func(http.Handler) http.Handler

func Chain(h http.Handler, ms ...Middleware) http.Handler {
    for i := len(ms) - 1; i >= 0; i-- { h = ms[i](h) }
    return h
}

func Logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Printf("%s %s %v", r.Method, r.URL.Path, time.Since(start))
    })
}

mux := http.NewServeMux()
handler := Chain(mux, Logging, Auth, RateLimit)
```

Q133

Design a generic priority queue.

```go
type PQ[T any] struct {
    items []T
    less  func(T, T) bool
}
func (pq *PQ[T]) Len() int           { return len(pq.items) }
func (pq *PQ[T]) Push(v T)           { pq.items = append(pq.items, v); up(pq, len(pq.items)-1) }
func (pq *PQ[T]) Pop() T             { n := len(pq.items)-1; pq.swap(0, n); down(pq, 0, n); v := pq.items[n
func (pq *PQ[T]) swap(i, j int)      { pq.items[i], pq.items[j] = pq.items[j], pq.items[i] }
// (heap operations: up/down standard binary heap)
```

Q134

Design a debouncer in Go.

```go
type Debouncer struct {
    mu      sync.Mutex
    timer   *time.Timer
    delay   time.Duration
}
func NewDebouncer(delay time.Duration) *Debouncer { return &Debouncer{delay: delay} }
func (d *Debouncer) Do(fn func()) {
    d.mu.Lock(); defer d.mu.Unlock()
    if d.timer != nil { d.timer.Stop() }
    d.timer = time.AfterFunc(d.delay, fn)
}
```

Q135

Design a throttler (rate limiter) in Go.

```go
type Throttler struct{ ticker *time.Ticker }
func NewThrottler(rps int) *Throttler {
    return &Throttler{ticker: time.NewTicker(time.Second / time.Duration(rps))}
}
func (t *Throttler) Wait(ctx context.Context) error {
    select {
    case <-t.ticker.C: return nil
    case <-ctx.Done(): return ctx.Err()
    }
}
```

Q136

Design a trie (prefix tree) for autocomplete.

```go
type TrieNode struct {
    children map[rune]*TrieNode
    isEnd    bool
    word     string
}
type Trie struct{ root *TrieNode }
func (t *Trie) Insert(word string) {
    node := t.root
    for _, ch := range word {
        if _, ok := node.children[ch]; !ok {
            node.children[ch] = &TrieNode{children: make(map[rune]*TrieNode)}
        }
        node = node.children[ch]
    }
    node.isEnd = true; node.word = word
}
func (t *Trie) Search(prefix string) []string {
    node := t.root
    for _, ch := range prefix {
        if _, ok := node.children[ch]; !ok { return nil }
        node = node.children[ch]
    }
    var results []string
    t.collect(node, &results)
    return results
}
func (t *Trie) collect(node *TrieNode, results *[]string) {
    if node.isEnd { *results = append(*results, node.word) }
    for _, child := range node.children { t.collect(child, results) }
}
```

Q137

Design a message queue (in-memory).

```go
type Queue[T any] struct {
    mu   sync.Mutex
    cond *sync.Cond
    items []T
    closed bool
}
func NewQueue[T any]() *Queue[T] { q := &Queue[T]{}; q.cond = sync.NewCond(&q.mu); return q }
func (q *Queue[T]) Enqueue(v T) {
    q.mu.Lock(); q.items = append(q.items, v); q.cond.Signal(); q.mu.Unlock()
}
func (q *Queue[T]) Dequeue() (T, bool) {
    q.mu.Lock(); defer q.mu.Unlock()
    for len(q.items) == 0 && !q.closed { q.cond.Wait() }
    if len(q.items) == 0 { var z T; return z, false }
    v := q.items[0]; q.items = q.items[1:]; return v, true
}
```

Q138

Design a read-write lock from scratch.

```go
type RWLock struct {
    mu      sync.Mutex
    cond    *sync.Cond
    readers int
    writing bool
}
func NewRWLock() *RWLock { l := &RWLock{}; l.cond = sync.NewCond(&l.mu); return l }
func (l *RWLock) RLock() {
    l.mu.Lock(); for l.writing { l.cond.Wait() }
    l.readers++; l.mu.Unlock()
}
func (l *RWLock) RUnlock() {
    l.mu.Lock(); l.readers--; if l.readers == 0 { l.cond.Broadcast() }; l.mu.Unlock()
}
func (l *RWLock) Lock() {
    l.mu.Lock(); for l.writing || l.readers > 0 { l.cond.Wait() }
    l.writing = true; l.mu.Unlock()
}
func (l *RWLock) Unlock() {
    l.mu.Lock(); l.writing = false; l.cond.Broadcast(); l.mu.Unlock()
}
```

Q139

Design a consistent hash ring.

```go
type HashRing struct {
    mu       sync.RWMutex
    ring     map[uint32]string
    sorted   []uint32
    replicas int
}
func (r *HashRing) Add(node string) {
    r.mu.Lock(); defer r.mu.Unlock()
    for i := 0; i < r.replicas; i++ {
        h := crc32.ChecksumIEEE([]byte(fmt.Sprintf("%s:%d", node, i)))
        r.ring[h] = node; r.sorted = append(r.sorted, h)
    }
    sort.Slice(r.sorted, func(i, j int) bool { return r.sorted[i] < r.sorted[j] })
}
func (r *HashRing) Get(key string) string {
    r.mu.RLock(); defer r.mu.RUnlock()
    h := crc32.ChecksumIEEE([]byte(key))
    idx := sort.Search(len(r.sorted), func(i int) bool { return r.sorted[i] >= h })
    if idx == len(r.sorted) { idx = 0 }
    return r.ring[r.sorted[idx]]
}
```

Q140-Q150: Final LLD Questions

| Q | Topic |

|---|---|

| Q140 | Design a JSON serializer/deserializer |

| Q141 | Design a workflow engine with DAG |

| Q142 | Design a feature flag system |

| Q143 | Design a distributed counter |

| Q144 | Design an event sourcing store |

| Q145 | Design a generic object pool |

| Q146 | Design an HTTP router (trie-based) |

| Q147 | Design a saga orchestrator |

| Q148 | Design a pipeline with stages and backpressure |

| Q149 | Design a bloom filter implementation |

| Q150 | LLD production readiness checklist |

Q142

Design a generic object pool in Go.

```go
type Pool[T any] struct {
    mu      sync.Mutex
    items   []T
    factory func() T
    reset   func(T) T
    maxSize int
}
func NewPool[T any](maxSize int, factory func() T, reset func(T) T) *Pool[T] {
    return &Pool[T]{factory: factory, reset: reset, maxSize: maxSize}
}
func (p *Pool[T]) Get() T {
    p.mu.Lock(); defer p.mu.Unlock()
    if len(p.items) == 0 { return p.factory() }
    item := p.items[len(p.items)-1]
    p.items = p.items[:len(p.items)-1]
    return item
}
func (p *Pool[T]) Put(item T) {
    p.mu.Lock(); defer p.mu.Unlock()
    if len(p.items) >= p.maxSize { return }  // discard if full
    p.items = append(p.items, p.reset(item))
}
// Usage: buffers, DB connections, serializers
```

Q143

Design an HTTP router with trie-based path matching.

```go
type node struct {
    children map[string]*node
    handler  http.Handler
    param    string  // e.g., "id" for /:id
}
type Router struct{ root *node }
func NewRouter() *Router { return &Router{root: &node{children: map[string]*node{}}} }
func (r *Router) Add(method, path string, h http.Handler) {
    parts := strings.Split(strings.Trim(path, "/"), "/")
    cur := r.root
    for _, p := range parts {
        key := p
        if strings.HasPrefix(p, ":") { key = ":" }
        if cur.children[key] == nil { cur.children[key] = &node{children: map[string]*node{}} }
        if key == ":" { cur.children[key].param = p[1:] }
        cur = cur.children[key]
    }
    cur.handler = h
}
func (r *Router) Match(path string) (http.Handler, map[string]string) {
    parts := strings.Split(strings.Trim(path, "/"), "/")
    params := map[string]string{}
    cur := r.root
    for _, p := range parts {
        if child, ok := cur.children[p]; ok { cur = child; continue }
        if param, ok := cur.children[":"]; ok { params[param.param] = p; cur = param; continue }
        return nil, nil
    }
    return cur.handler, params
}
```

Q144

Design a saga orchestrator in Go.

```go
type Step struct {
    Name       string
    Execute    func(ctx context.Context) error
    Compensate func(ctx context.Context) error
}

type Saga struct {
    steps []Step
}

func (s *Saga) Run(ctx context.Context) error {
    executed := make([]Step, 0, len(s.steps))

    for _, step := range s.steps {
        if err := step.Execute(ctx); err != nil {
            // Compensate in reverse order
            for i := len(executed) - 1; i >= 0; i-- {
                if cerr := executed[i].Compensate(ctx); cerr != nil {
                    log.Printf("compensation failed for %s: %v", executed[i].Name, cerr)
                }
            }
            return fmt.Errorf("step %s failed: %w", step.Name, err)
        }
        executed = append(executed, step)
    }
    return nil
}

// Usage
saga := &Saga{steps: []Step{
    {Name: "reserve-inventory",
     Execute: func(ctx context.Context) error { return inventory.Reserve(ctx, itemID) },
     Compensate: func(ctx context.Context) error { return inventory.Release(ctx, itemID) }},
    {Name: "charge-payment",
     Execute: func(ctx context.Context) error { return payment.Charge(ctx, amount) },
     Compensate: func(ctx context.Context) error { return payment.Refund(ctx, amount) }},
}}
```

Q145

Design a pipeline with stages and back-pressure.

```go
func Stage[In, Out any](ctx context.Context, in <-chan In, workers int, fn func(In) Out) <-chan Out {
    out := make(chan Out, workers)  // buffer = back-pressure control
    var wg sync.WaitGroup
    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for item := range in {
                select {
                case out <- fn(item):
                case <-ctx.Done(): return
                }
            }
        }()
    }
    go func() { wg.Wait(); close(out) }()
    return out
}

// Usage: multi-stage pipeline
source := produce(ctx, data)
stage1 := Stage(ctx, source, 4, parseRecord)
stage2 := Stage(ctx, stage1, 8, enrichRecord)
stage3 := Stage(ctx, stage2, 2, writeToDB)

for result := range stage3 {
    log.Println("processed:", result)
}
```

Q146

Design a bloom filter in Go.

```go
type BloomFilter struct {
    bitset    []uint64
    hashFuncs int
    size      uint
}

func NewBloomFilter(size, hashFuncs int) *BloomFilter {
    return &BloomFilter{
        bitset:    make([]uint64, (size+63)/64),
        hashFuncs: hashFuncs,
        size:      uint(size),
    }
}

func (bf *BloomFilter) hash(data []byte, seed uint) uint {
    h := fnv.New64a()
    binary.Write(h, binary.LittleEndian, uint64(seed))
    h.Write(data)
    return uint(h.Sum64()) % bf.size
}

func (bf *BloomFilter) Add(data []byte) {
    for i := 0; i < bf.hashFuncs; i++ {
        pos := bf.hash(data, uint(i))
        bf.bitset[pos/64] |= 1 << (pos % 64)
    }
}

func (bf *BloomFilter) Contains(data []byte) bool {
    for i := 0; i < bf.hashFuncs; i++ {
        pos := bf.hash(data, uint(i))
        if bf.bitset[pos/64]&(1<<(pos%64)) == 0 {
            return false  // definitely not present
        }
    }
    return true  // probably present
}
```

Q147

Design a distributed ID generator (Snowflake) in Go.

```go
// Snowflake ID: 64-bit = 1 unused + 41 timestamp + 10 node + 12 sequence
const (
    epoch     = int64(1609459200000) // 2021-01-01 in ms
    nodeBits  = 10
    seqBits   = 12
    nodeMax   = -1 ^ (-1 << nodeBits)
    seqMax    = -1 ^ (-1 << seqBits)
    timeShift = nodeBits + seqBits
    nodeShift = seqBits
)

type Snowflake struct {
    mu        sync.Mutex
    nodeID    int64
    sequence  int64
    lastStamp int64
}

func (s *Snowflake) NextID() int64 {
    s.mu.Lock(); defer s.mu.Unlock()
    now := time.Now().UnixMilli()
    if now == s.lastStamp {
        s.sequence = (s.sequence + 1) & seqMax
        if s.sequence == 0 {
            for now <= s.lastStamp { now = time.Now().UnixMilli() }
        }
    } else {
        s.sequence = 0
    }
    s.lastStamp = now
    return (now-epoch)<<timeShift | (s.nodeID << nodeShift) | s.sequence
}
```

Q148

Design an event sourcing store in Go.

```go
type Event struct {
    ID          int64
    AggregateID string
    Type        string
    Data        []byte
    OccurredAt  time.Time
    Version     int
}

type EventStore struct {
    mu     sync.RWMutex
    events map[string][]Event  // aggregateID → events
}

func (es *EventStore) Append(aggID string, events []Event, expectedVersion int) error {
    es.mu.Lock(); defer es.mu.Unlock()
    existing := es.events[aggID]
    if len(existing) != expectedVersion {
        return fmt.Errorf("optimistic concurrency conflict: expected version %d, got %d", expectedVersion,
    }
    for i := range events {
        events[i].Version = len(existing) + i + 1
        events[i].OccurredAt = time.Now()
    }
    es.events[aggID] = append(existing, events...)
    return nil
}

func (es *EventStore) Load(aggID string) []Event {
    es.mu.RLock(); defer es.mu.RUnlock()
    return append([]Event{}, es.events[aggID]...)
}

// Rebuild aggregate from events
func RebuildOrder(events []Event) *Order {
    o := &Order{}
    for _, e := range events {
        switch e.Type {
        case "OrderPlaced": json.Unmarshal(e.Data, &o.Items)
        case "OrderPaid": o.Status = "paid"
        }
    }
    return o
}
```

Q149

Design a rate limiter using token bucket in Go.

```go
type TokenBucket struct {
    mu         sync.Mutex
    tokens     float64
    maxTokens  float64
    refillRate float64  // tokens per second
    lastRefill time.Time
}

func NewTokenBucket(maxTokens, refillRate float64) *TokenBucket {
    return &TokenBucket{
        tokens:     maxTokens,
        maxTokens:  maxTokens,
        refillRate: refillRate,
        lastRefill: time.Now(),
    }
}

func (tb *TokenBucket) Allow(cost float64) bool {
    tb.mu.Lock(); defer tb.mu.Unlock()

    now := time.Now()
    elapsed := now.Sub(tb.lastRefill).Seconds()
    tb.lastRefill = now

    // Add tokens based on elapsed time
    tb.tokens = math.Min(tb.maxTokens, tb.tokens + elapsed * tb.refillRate)

    if tb.tokens >= cost {
        tb.tokens -= cost
        return true
    }
    return false
}

func (tb *TokenBucket) Wait(ctx context.Context, cost float64) error {
    for {
        if tb.Allow(cost) { return nil }
        select {
        case <-ctx.Done(): return ctx.Err()
        case <-time.After(10 * time.Millisecond):
        }
    }
}
```

Q150

What is LLD production readiness checklist?

```go
Code Quality:
  ■ SOLID principles followed
  ■ Interfaces used for dependencies (testability)
  ■ Error types defined (not just string errors)
  ■ Exported types/functions documented
  ■ No global mutable state

Concurrency:
  ■ All shared data protected (mutex or channels)
  ■ No data races: go test -race passes
  ■ Goroutine lifecycle managed (no goroutine leaks)
  ■ Context propagated throughout (cancellation)
  ■ Sync primitives zero-value usable (no init required for Mutex)

Reliability:
  ■ All errors handled (errcheck passes)
  ■ Timeouts on all external calls
  ■ Circuit breakers for unreliable dependencies
  ■ Retry logic with jitter
  ■ Graceful shutdown implemented

Testing:
  ■ Unit tests for all business logic
  ■ Table-driven tests for edge cases
  ■ Interfaces mocked for unit isolation
  ■ Integration tests for DB/cache/queue
  ■ Benchmarks for hot paths
  ■ go test -race -coverprofile=coverage.out

Performance:
  ■ No unnecessary allocations in hot paths
  ■ Object pooling for high-allocation types
  ■ Profiled with pprof (no obvious bottlenecks)
  ■ sync.Pool for buffers
  ■ Appropriate data structures (slice vs map vs list)
```